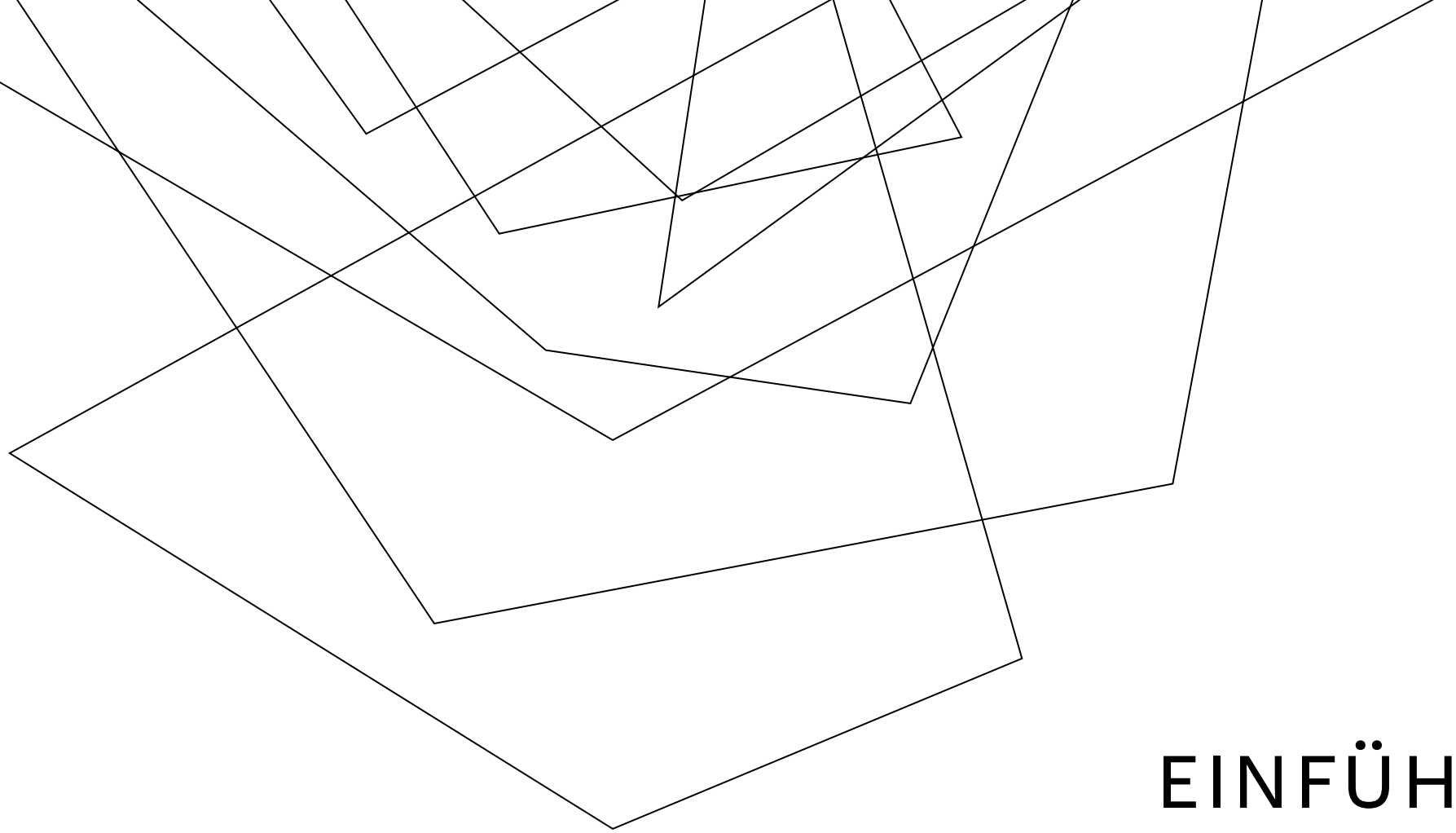


EINFÜHRUNG

ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

GLIEDERUNG

- Projektbeschreibung
- Architektur
- Verlauf
- Tätigkeiten als Projektleiter

PROJEKTBESCHREIBUNG

PROJEKTBESCHREIBUNG

Todo

- Roboter, der ein gegebenes Puzzle automatisiert lösen soll
- Robot Operating System (ROS)
- 7x7 Puzzle

PROJEKTBESCHREIBUNG

Digitalisierung

- jedes einzelne Puzzleteil wird systematisch und möglichst einheitlich abfotografiert
- in ein geeignetes Dateiformat umgewandelt
- manueller, einmaliger Prozess

PROJEKTBESCHREIBUNG

Visualisierung

- KUKA KR6 Roboter aus 7 Komponenten
- Object Files liegen vor
- Szenegraph-System Coin3D

PROJEKTBESCHREIBUNG

Bewegung

- kontinuerliche und abgerundete Trajektorie
- Durch Neuanordnung der Roboterarme realisiert
- Durch Robotics Toolbox von Peter Corke unterstützt

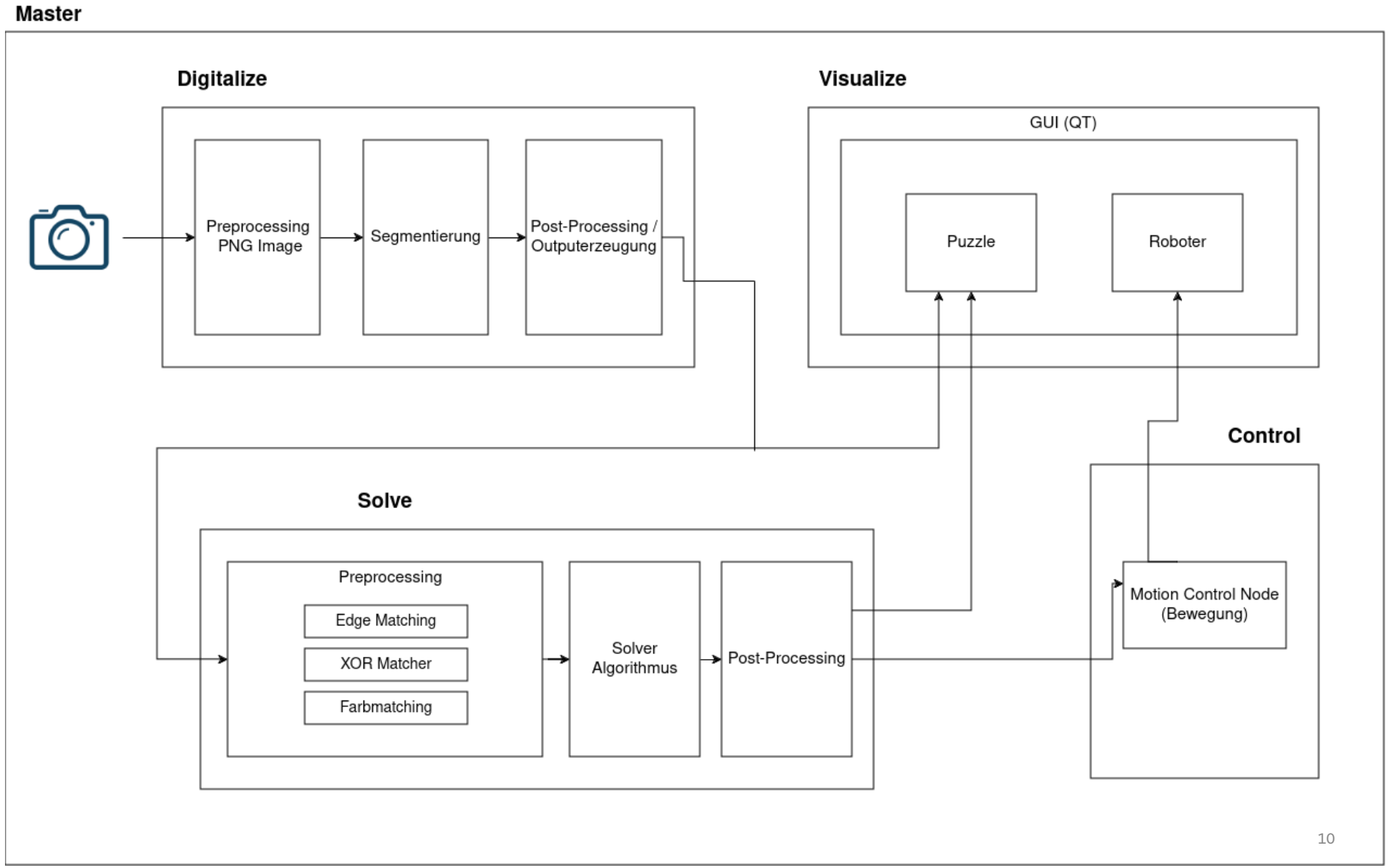
PROJEKTBESCHREIBUNG

Lösen des Puzzles

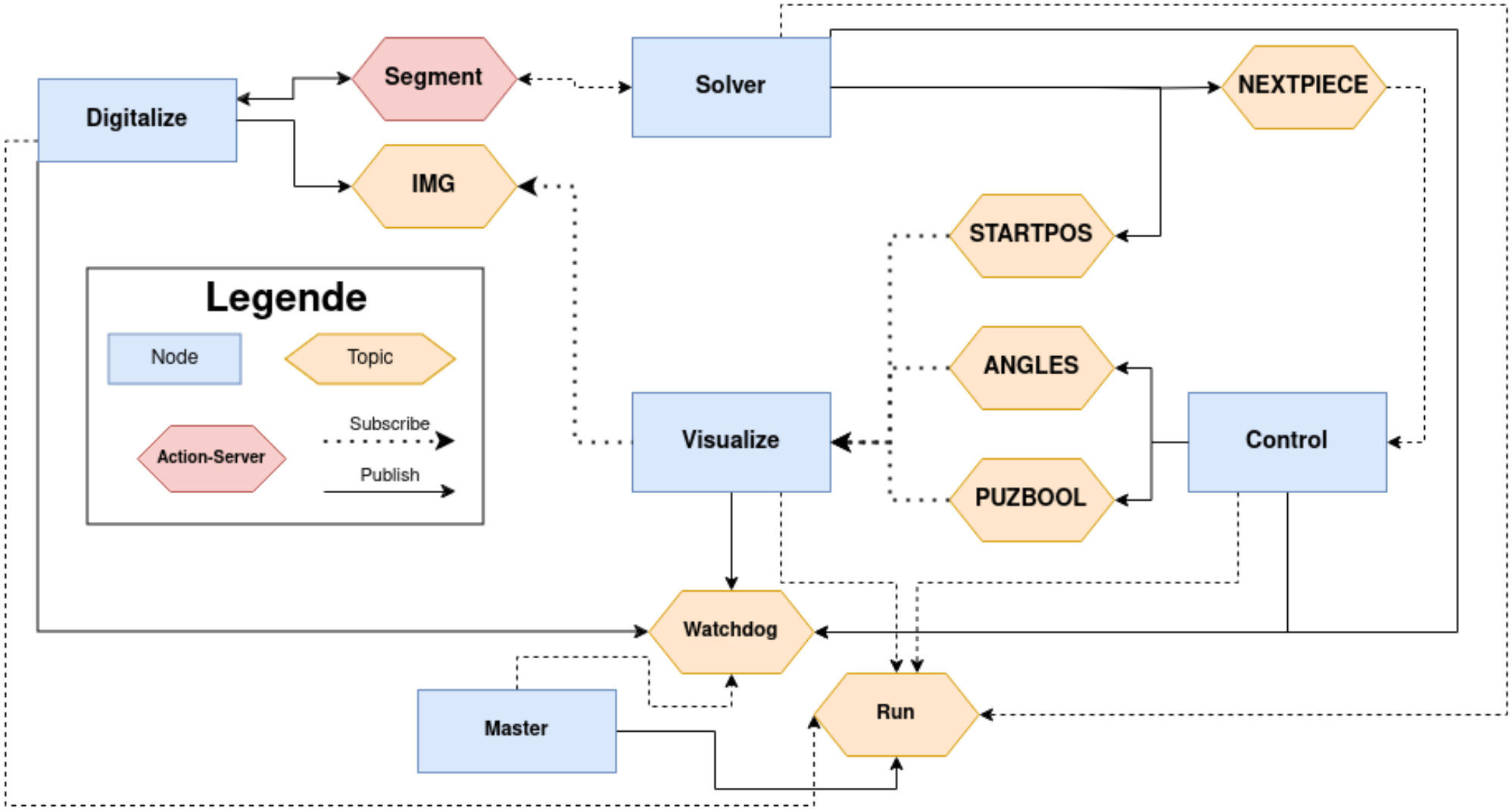
- Keine Vorgabe
- Hauptsache, die korrekte Lösung wird gefunden

ARCHITEKTUR

ARCHITEKTUR

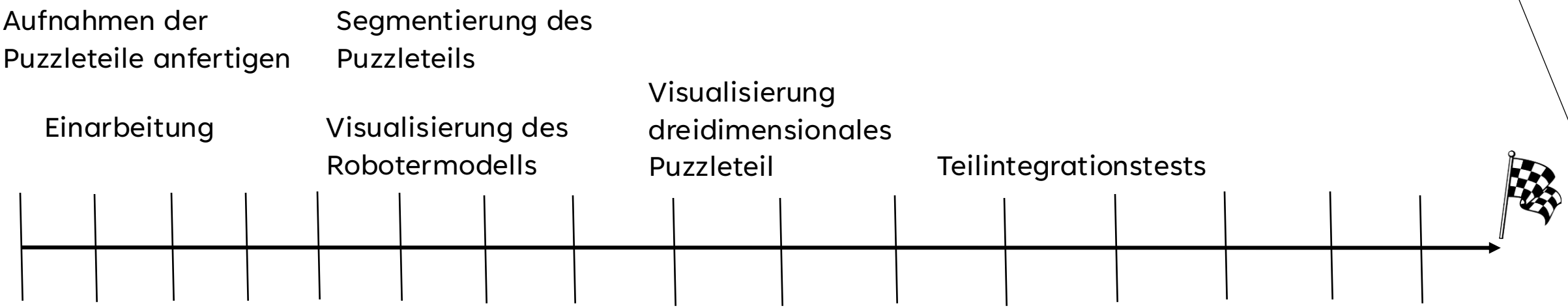


ARCHITEKTUR



VERLAUF

VERLAUF


Aufnahmen der Puzzleteile anfertigen
Segmentierung des Puzzleteils
Einarbeitung
Visualisierung des Robotermodells
Visualisierung dreidimensionales Puzzleteil
Teilintegrationstests
Erstellung einer gemeinsamen Entwicklungsumgebung
Erster Ansatz für Solveralgorithmus
Farb- und XOR Matcher
Integration aller ROS Nodes in ein Gesamtprodukt
Kantenvergleich als Wahrscheinlichkeitsmaß
Postprocessing: Berechnung
Bugfixes
Einfache Roboterbewegung

PROJEKTDURCHFÜHRUNG

ORGANISATION

- von den Teilnehmern des Kurses weitestgehend eigenverantwortlich durchgeführt
- anfängliche Vorgaben - Lastenheft - müssen erfüllt werden
- Umsetzung den Studenten überlassen
- Festlegung von Themengebieten und zugehörigen Aufgaben
- -> Anhand dieser Gebiete Teams eingeteilt

ORGANISATION

- Arbeitspakete in Form von Epics
- Laufzeit von zwei bis vier Wochen
- agile Methoden wie Scrum
- Wöchentliche Meetings mit allen Mitarbeitern + Stakeholder
- -> Continuous Feedback

PROJEKTLEITUNG

- enge Absprache mit dem Kunden
- Fortschritt des Projektes überwachen
- Eingreifen, falls der Zeitplan in Gefahr gerät
- Stakeholder in geeigneter Form den Fortschritt des Projektes vermitteln

PROJEKTLEITUNG

Methoden:

- Aufbereiten und Visualisieren dieser Informationen für Stakeholder
- Regelmäßige Besprechungen mit den Team
- Anfänglich Risikoanalyse mit den Gruppen
- Planung, Terminierung und Aufwandsabschätzung über Epics

PROJEKTLEITUNG

Methoden:

- Umverteilung der ”Mitarbeiter” dynamisch nach Workload und Bedarf
- Definition von Teilintegrationen
- Festlegen von Standards um gemeinsame Arbeit zu ermöglichen

TOP-DOWN BOTTOM-UP

- Initiative bei der Gestaltung und Ausführung neuer Projektabschnitte ging oft vom Projektleiter aus
- (Ausgearbeitete) Vorschläge und Ideen initial einbringen
  -> Top-Down
- Von den Studenten akzeptiert oder optimiert
  -> Bottom-Up
- auch als ”bipolare Strategie” bezeichnet [1]

TOP-DOWN BOTTOM-UP
PROJEKTDOKUMENTATION

- Vorgestellt:
  - was eine Projektdokumentation ist
  - welchen Zweck sie erfüllt
  - wie sie aufgebaut ist
  - Welche Fehler in Zusammenhang mit dieser auftreten können

TOP-DOWN BOTTOM-UP
PROJEKTDOKUMENTATION

- Bestandteile einer Projektdokumentation aufgelistet

1. Projektvorfeld (IST-Analyse) 2. Aufgabenstellung (SOLL-Konzept) / Anforderungen
3. Projektziele (SOLL-IST-VERGLEICH) 4. Zeitplanung / Meilensteine
5. Wirtschaftlichkeit 6. Projektverlauf
7. Ausblick / Folgeaktivitäten 8. Pflichtenheft
9. Angebot 10. Preiskalkulation / Nutzwertanalyse
11. Kundeneinweisung 12. Abnahme- und Testprotokoll
13. Rechnung

TOP-DOWN BOTTOM-UP
PROJEKTDOKUMENTATION

- Top-Down Ansatz im Sinne der Modellierung
- Überbegriff Projektdokumentation in seine Bestandteile heruntergebrochen
- Nun weiter verfeintert
- Alle für unser Projekt nicht relevanten Bestandteile entfernt

1. Aufgabenstellung (SOLL-Konzept) / Anforderungen
2. Zeitplanung / Meilensteine
3. Projektverlauf
4. Kundeneinweisung
5. Abnahme- und Testprotokoll

TOP-DOWN BOTTOM-UP
PROJEKTDOKUMENTATION

- geprüft welche Teile bereits (in einer anderen Form) vorhanden sind

1. Aufgabenstellung (SOLL-Konzept) / Anforderungen
2. Zeitplanung / Meilensteine
3. Projektverlauf
4. Kundeneinweisung
5. Abnahme- und Testprotokoll

TOP-DOWN BOTTOM-UP
PROJEKTDOKUMENTATION

Dokumentation schon fertig?

- Technische Dokumentation zur Übergabe an den Kunden
- Projekt weiterentwickeln oder warten
- von den Studenten nur noch der individuelle Projektverlauf sowie die technische Dokumentation benötigt
- Verfassen in Latex wurde vom Projektleiter vorgeschrieben

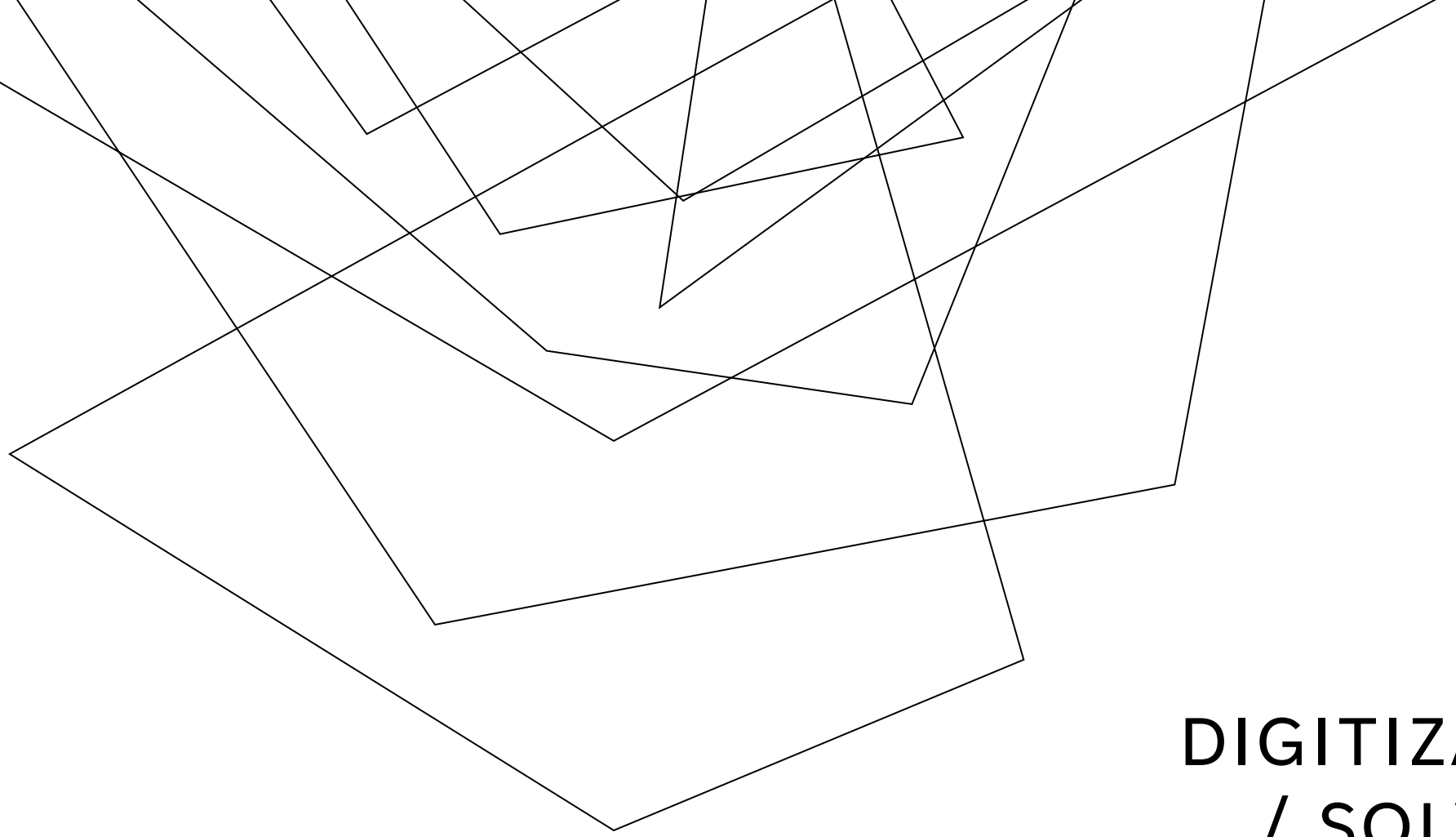
VIELEN DANK FÜR EURE AUFMERKSAMKEIT!

Max Mustermann

@mcautom

Quellen:

[1] S. Kauffeld, Arbeits-, Organisations- und Personalpsychologie für Bachelor. Springer-Verlag: Berlin Heidelberg, 2014



DIGITIZATION / SOLVER

ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

GLIEDERUNG

- Digitalisierung
- GeoMatcher
- Matcher Evaluation

DIGITALISIERUNG

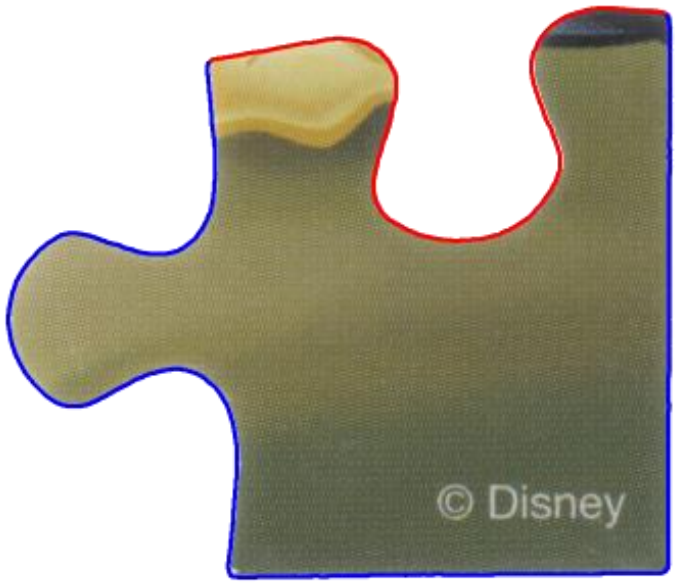






GEOMATCHER

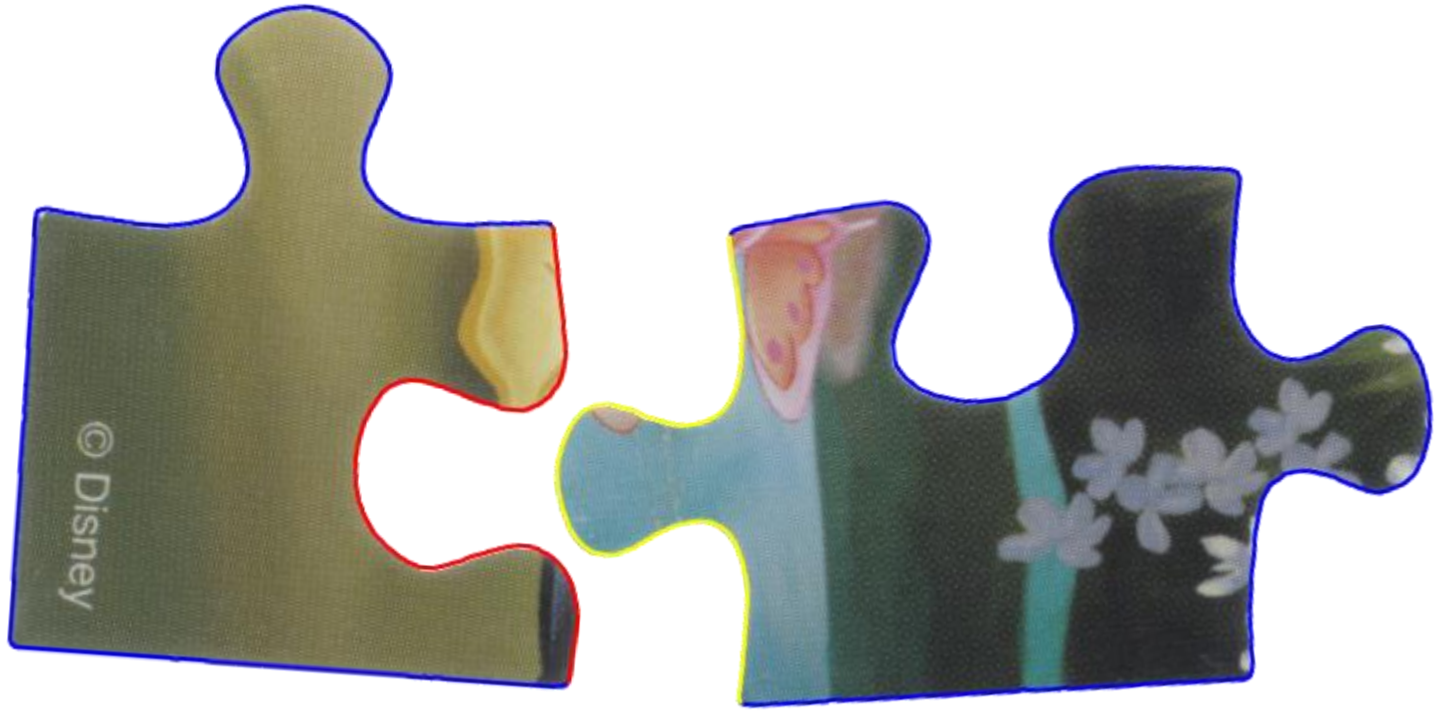
GEOMATCHER / MATCHER-INTERFACE





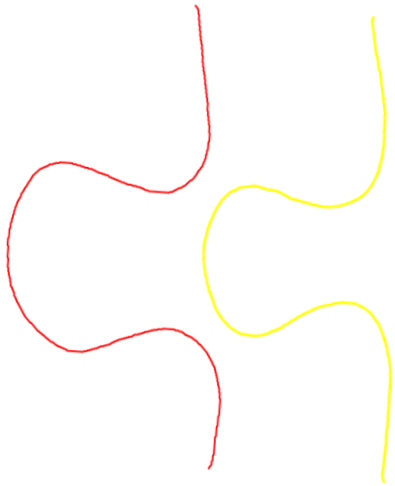
Wie gut passen diese Puzzleteile auf einer Skala von 0 (gar nicht) bis 1 (perfekt) zusammen?

GEOMATCHER / IDEE



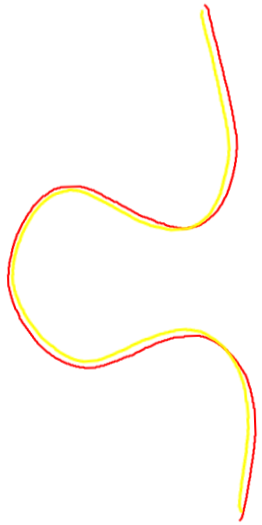
Vergleichbar anordnen

GEOMATCHER / IDEE

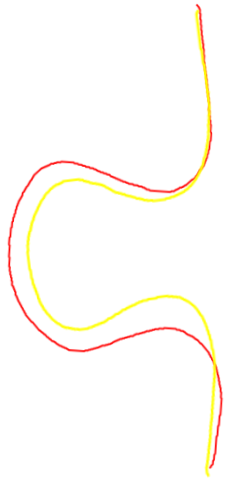


Betrachtung der Kantenform

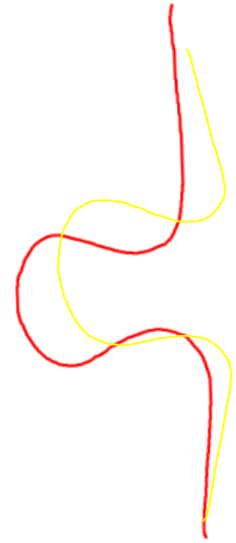
GEOMATCHER / IDEE



Sehr ähnlich
1



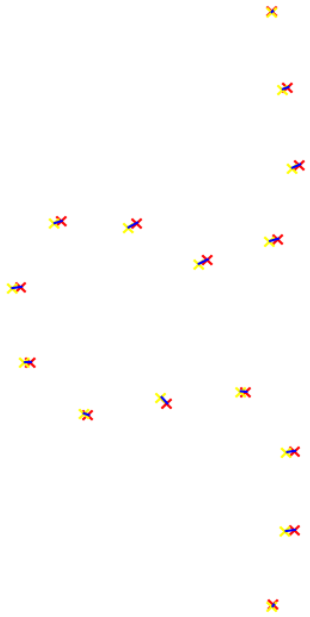
Mittelmäßig



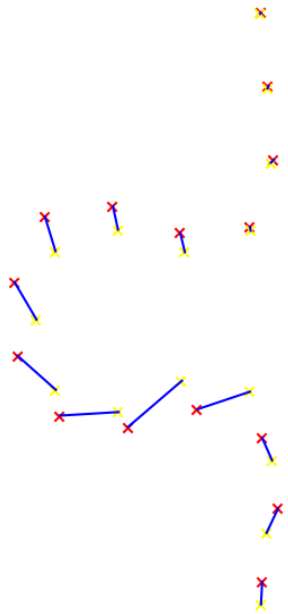
Sehr unähnlich
0

Abschätzen der Ähnlichkeit - intuitiv

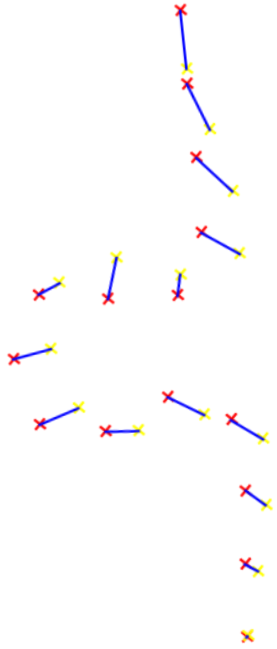
GEOMATCHER / IDEE



Sehr ähnlich
102.97



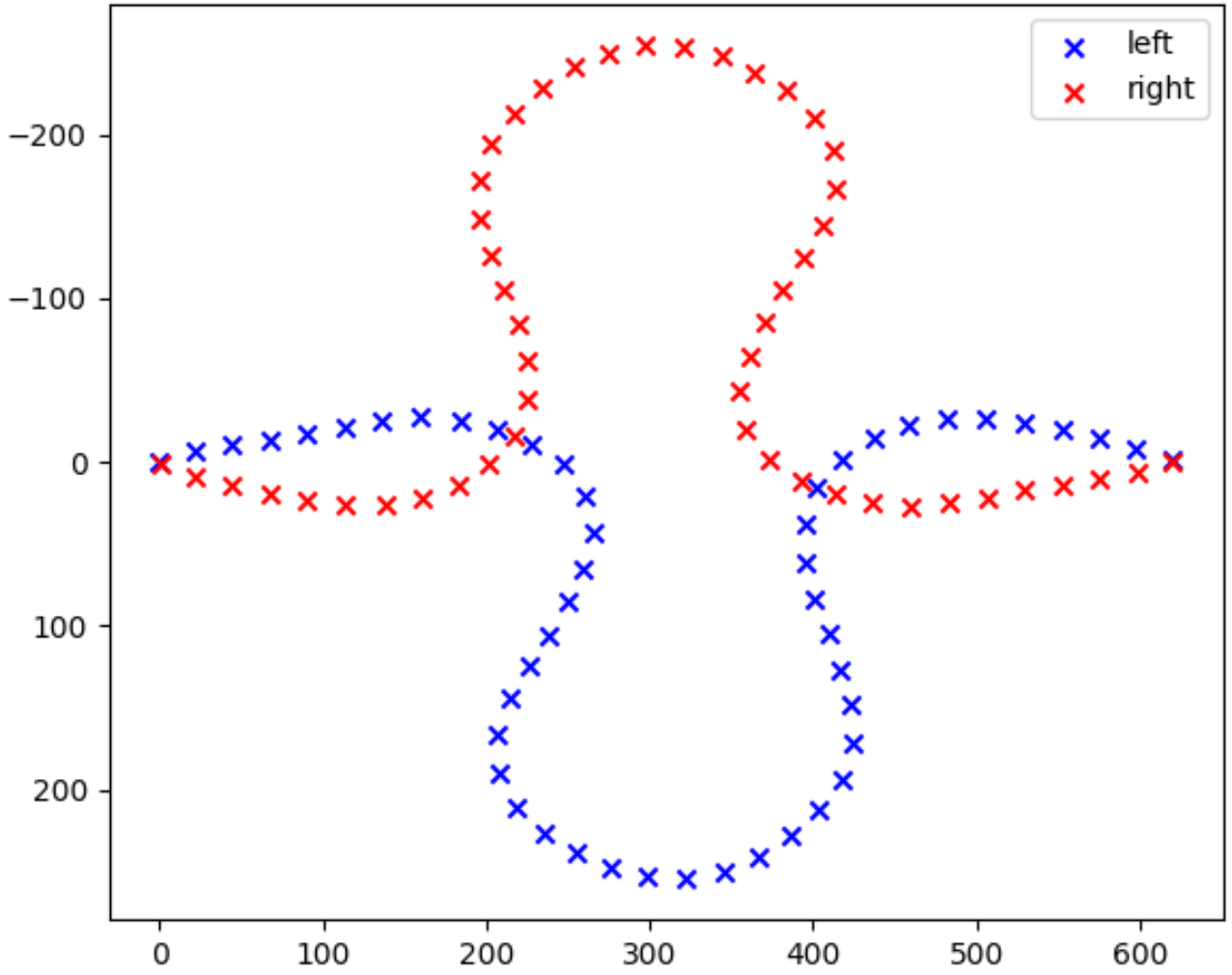
Mittelmäßig
417.64



Sehr unähnlich
553.45

Abschätzen der Ähnlichkeit - rechnerisch

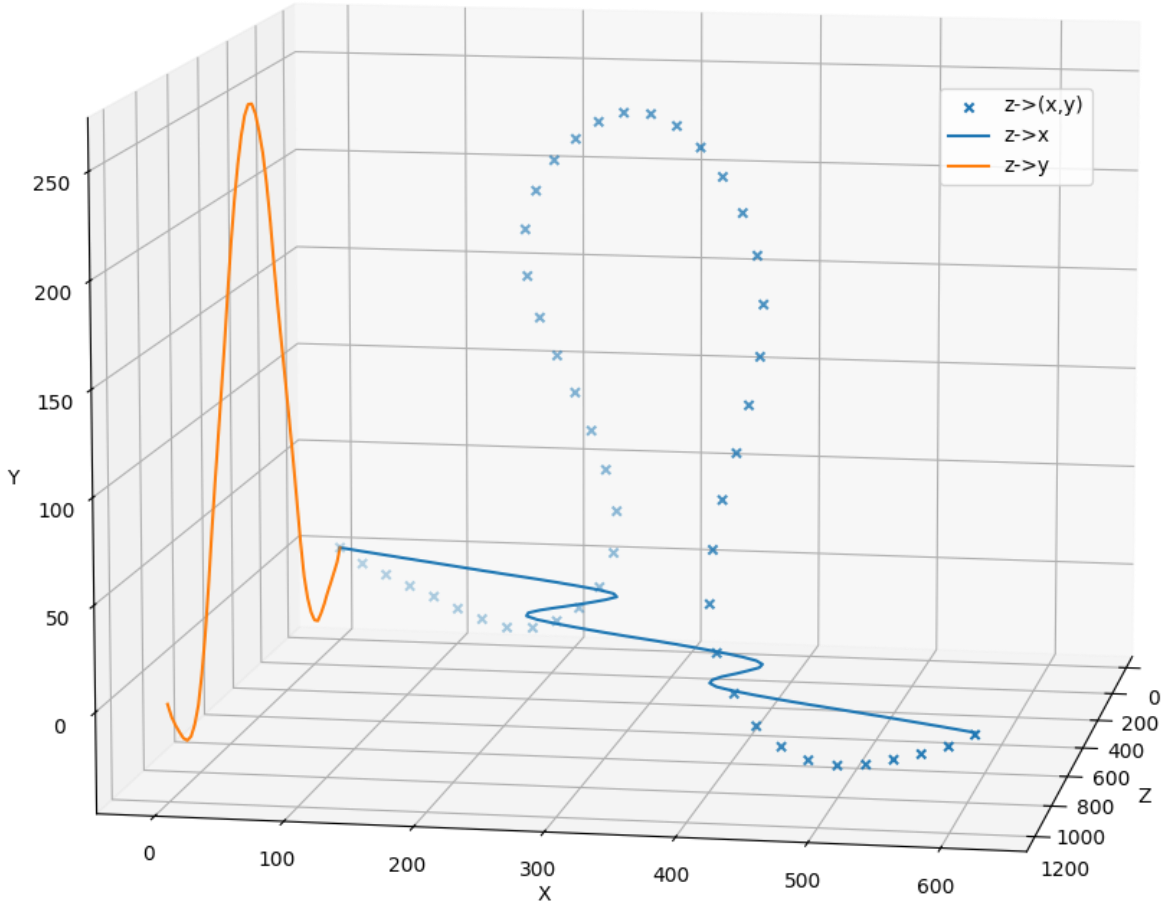
GEOMATCHER / PREPROCESSING



- Problem: langsame Transformation
- Außenseiten der Puzzleteile müssen sich beim Vergleich gegenüberliegen
  ➔ Zwei Versionen nötig
- Nur left - right sind vergleichbar!

GEOMATCHER / DISKRETISIERUNG

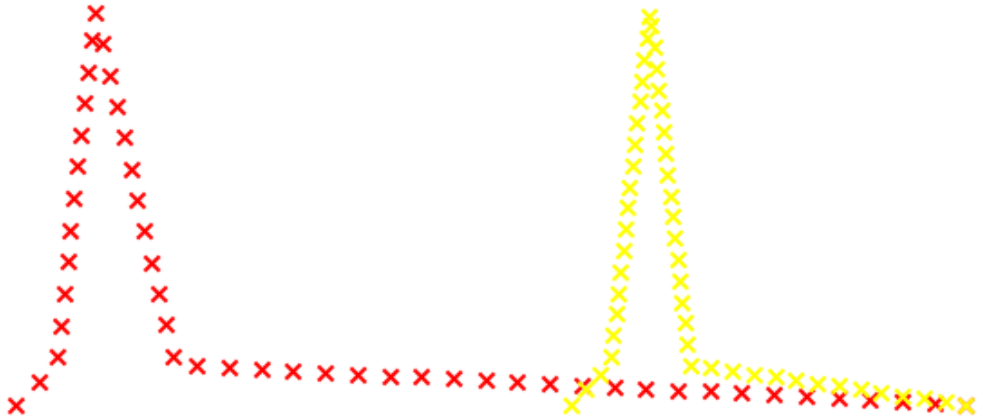
- Problem: nicht-funktionelle Kanten
- Einführung 3. Dimension: kumulative Kurvenlänge Z
- Interpolation von z nach x, y
- linspace auf z sorgt für gleichmäßige Verteilung der Abtastung auf der Kante



GEOMATCHER / DETERMINISMUS



Fehler: 320.44



Fehler: 466.11

Welche left-right-Zuweisung ist die "richtige"?

➔ Berechnung und Mittelung beider Möglichkeiten

MATCHER EVALUATION

MATCHER EVALUATION / BINÄRE KLASSIFIKATION

- Interpretation als binärer Klassifikator
- Besondere Form: fp = fn

	Geo	XOR	GeoRGB	Color
Accuracy	99.83%	97.71%	100%	99.94%
Precision Recall F1-Score	92.86%	5.95%	100%	97.62%
Specificity	99.91%	98.84%	100%	99.97%

➔ GeoMatcherRGB leistungsstärkster Einzelmatcher

➔ Farbmatcher > Formmatcher

➔ GeoMatcher > XorMatcher

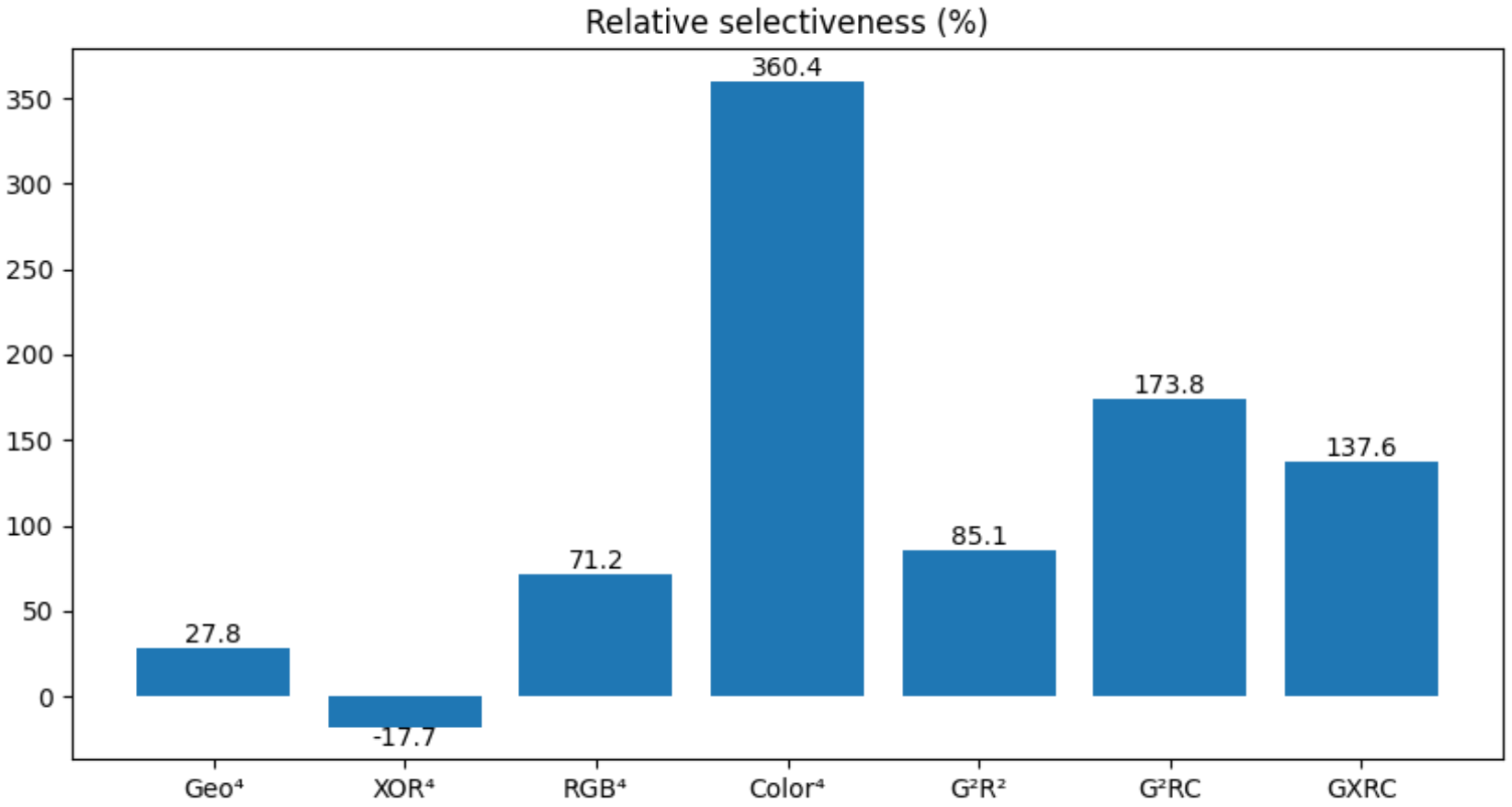
MATCHER EVALUATION / BINÄRE KLASSIFIKATION

- Kombination durch Multiplikation der Einzelscores
- Vergleich von vier kombinierten Matchern

	$Geo^2 \times RGB^2$	$Geo^2 \times RGB \times Color$	$Geo \times XOR \times RGB \times Color$
Accuracy	100%	100%	100%
Precision Recall F1-Score	100%	100%	100%
Specificity	100%	100%	100%

➔ Kombination von Farb- und Formmatchern vielversprechend

MATCHER EVALUATION / TRENNSCHÄRFE

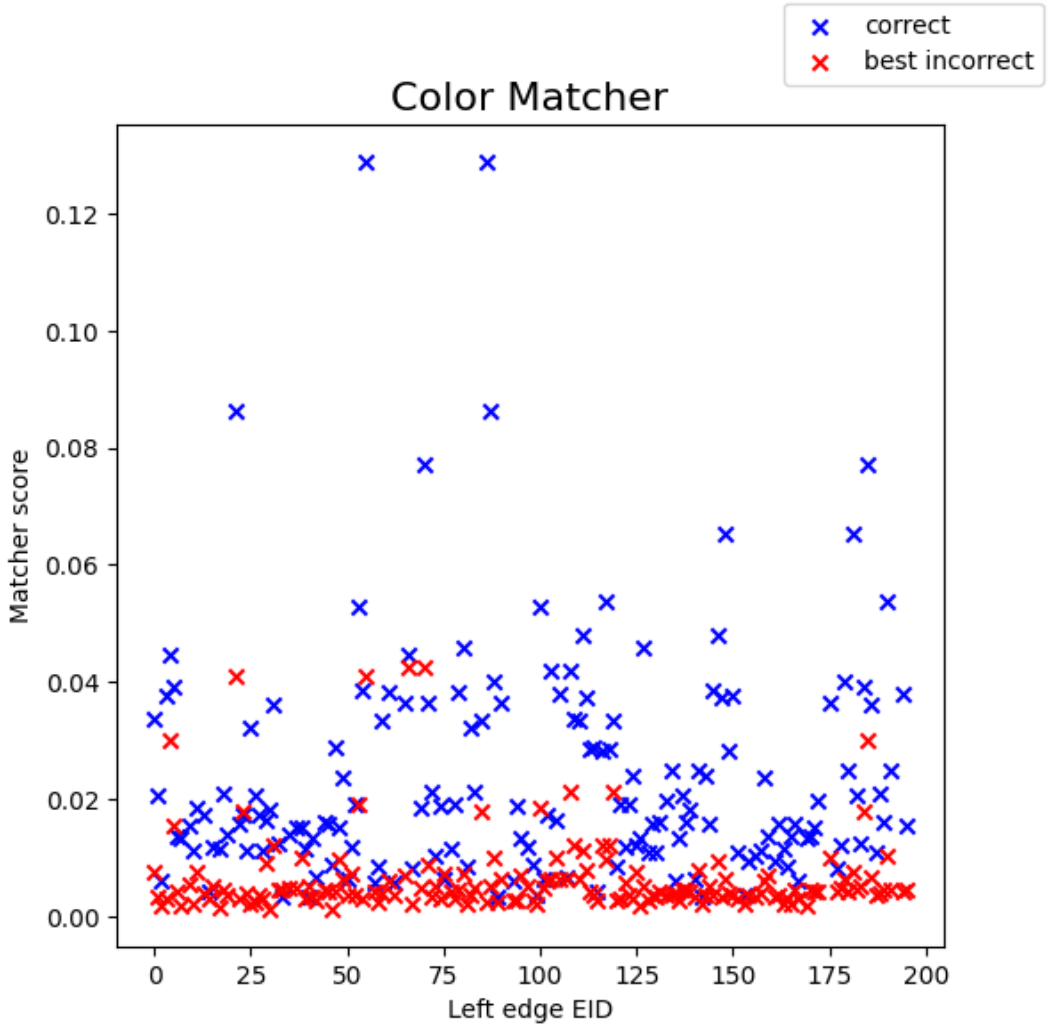


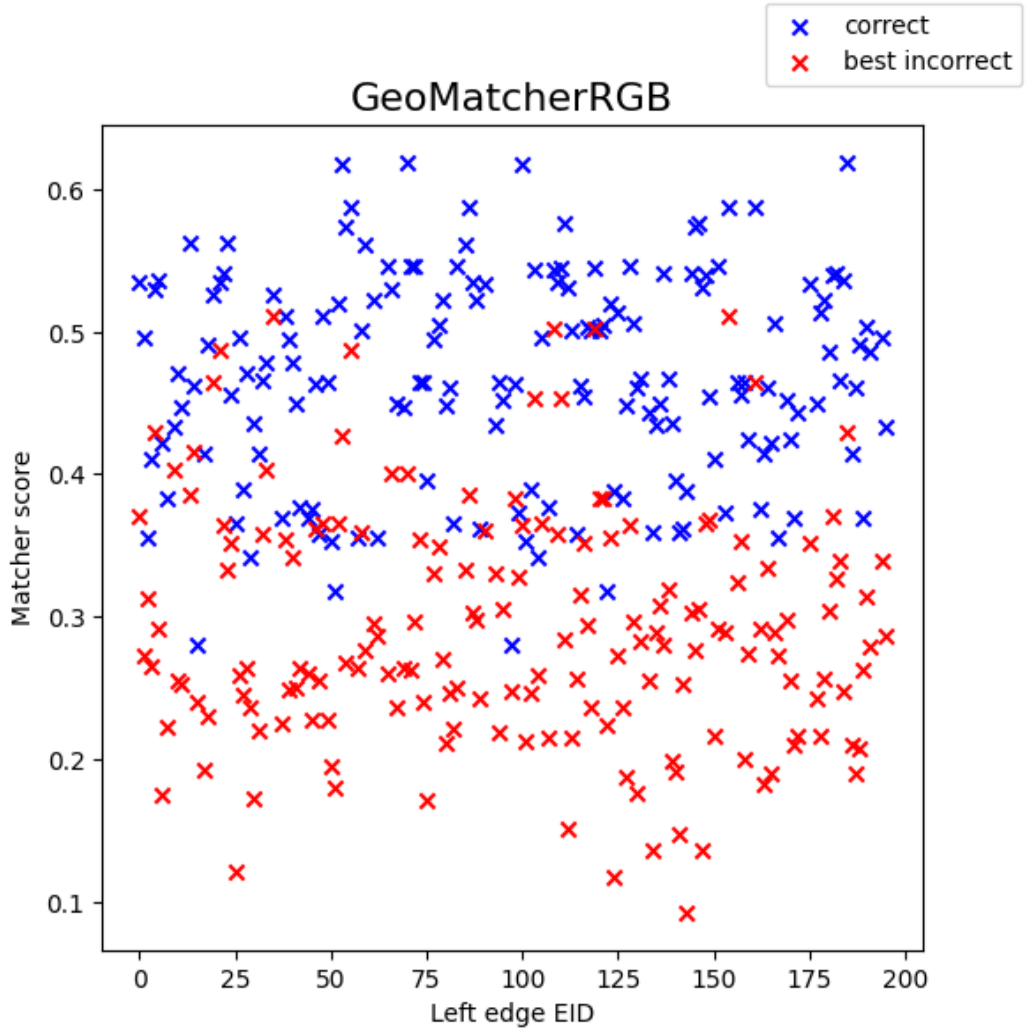
➔ ColorMatcher sehr trennscharf, aber in der binären Klassifikation schlechter?

➔ Extrem niedrige Scores (0,000)

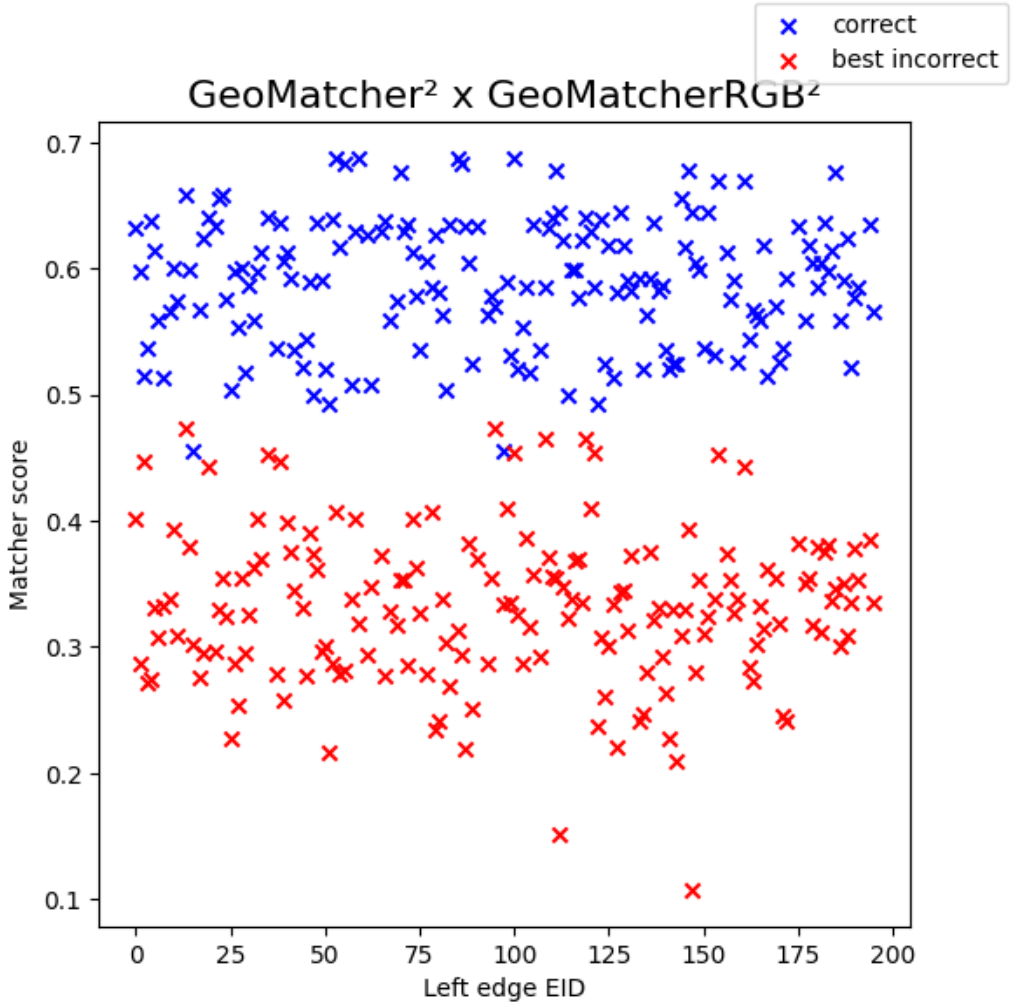
➔ Nur der Durchschnitt

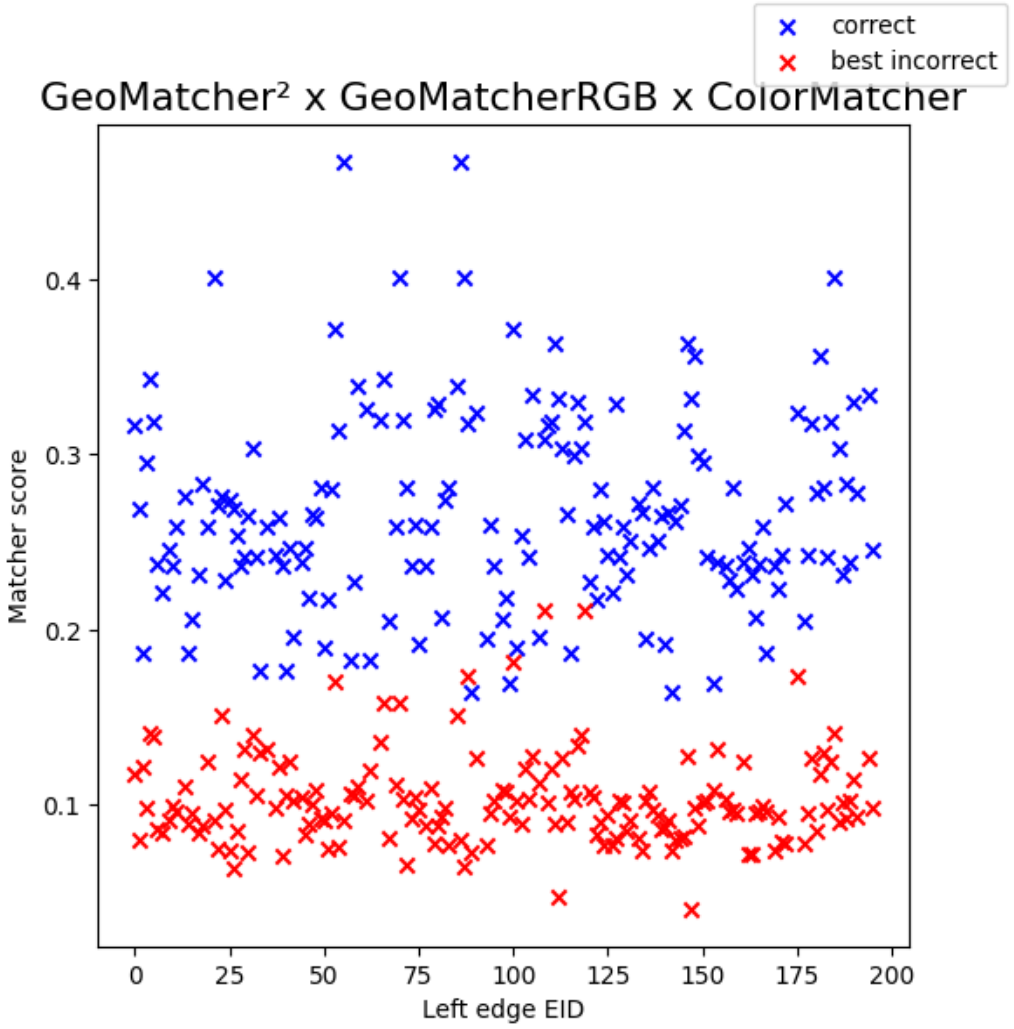
MATCHER EVALUATION / TRENNSCHÄRFE





MATCHER EVALUATION / TRENNSCHÄRFE





VIELEN DANK FÜR IHRE AUFMERKSAMKEIT!

Cedric Holzer



@hm-holzer2



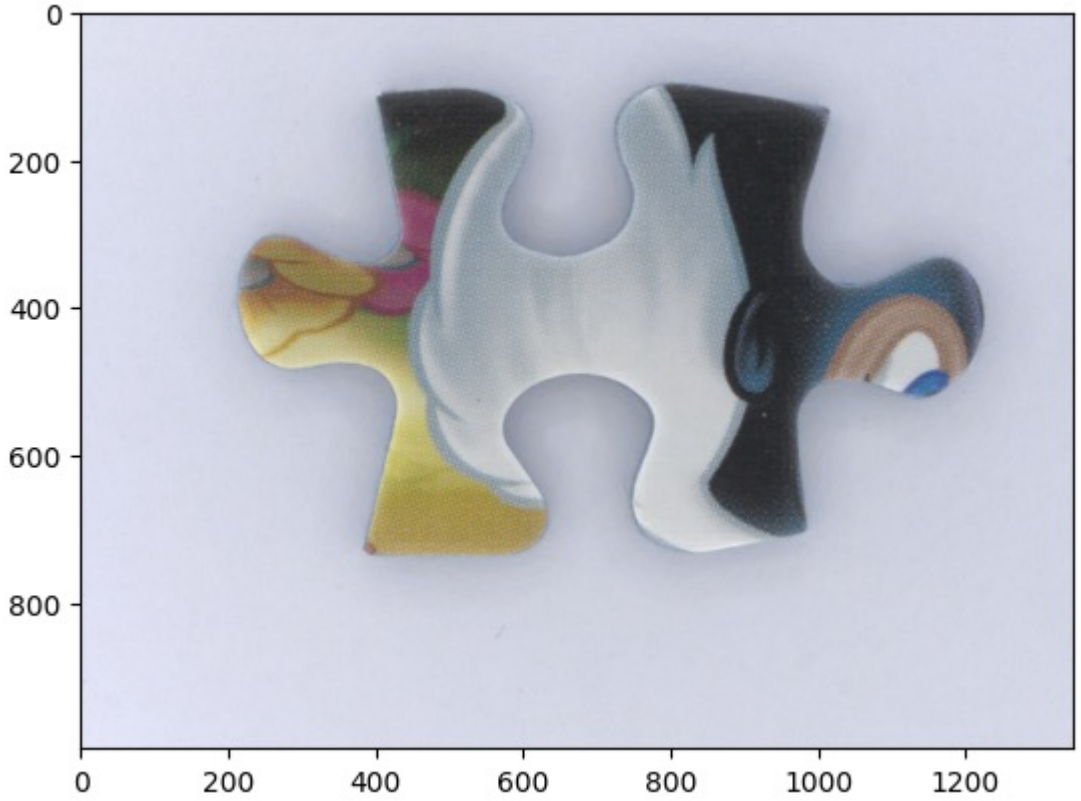
EDGE-DETECTION SEGMENTIERUNG UND ROS-NODE

ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

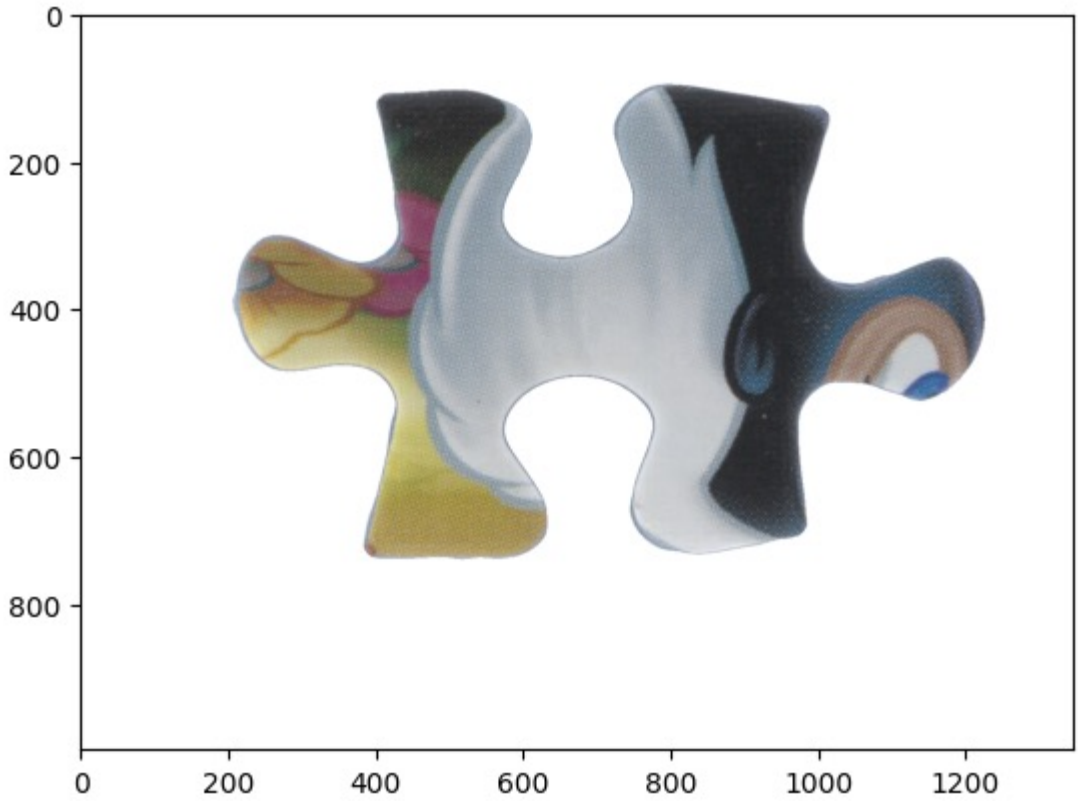
GLIEDERUNG

- Zielsetzung
- Verschiedene Verfahren
- Edge-Detection
  Segmentierung
- ROS-Node

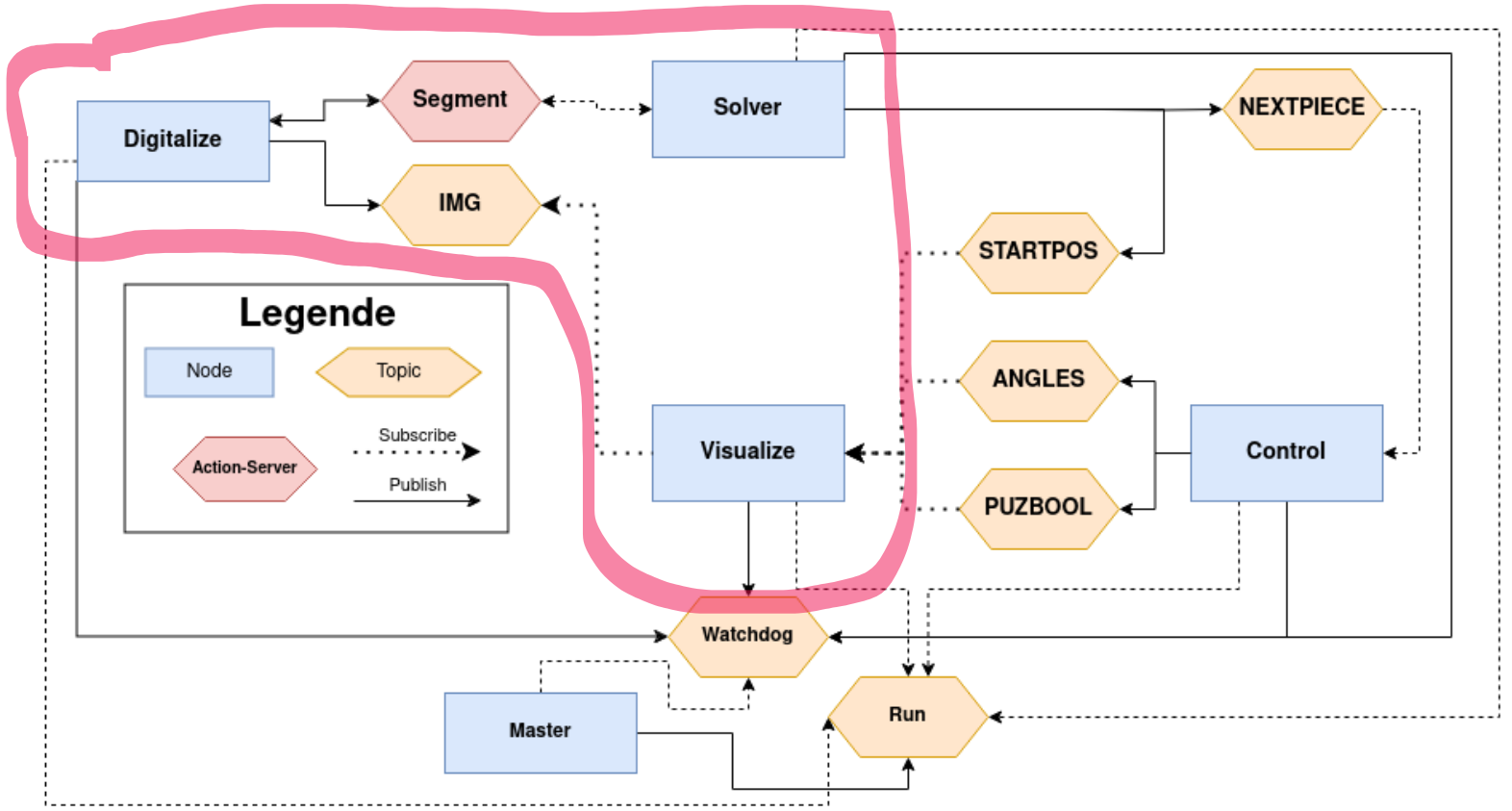
ZIELSETZUNG – SEGMENTIERUNG







ZIELSETZUNG - VERTEILUNG



VERSCHIEDENE VERFAHREN

Threshold Based – X

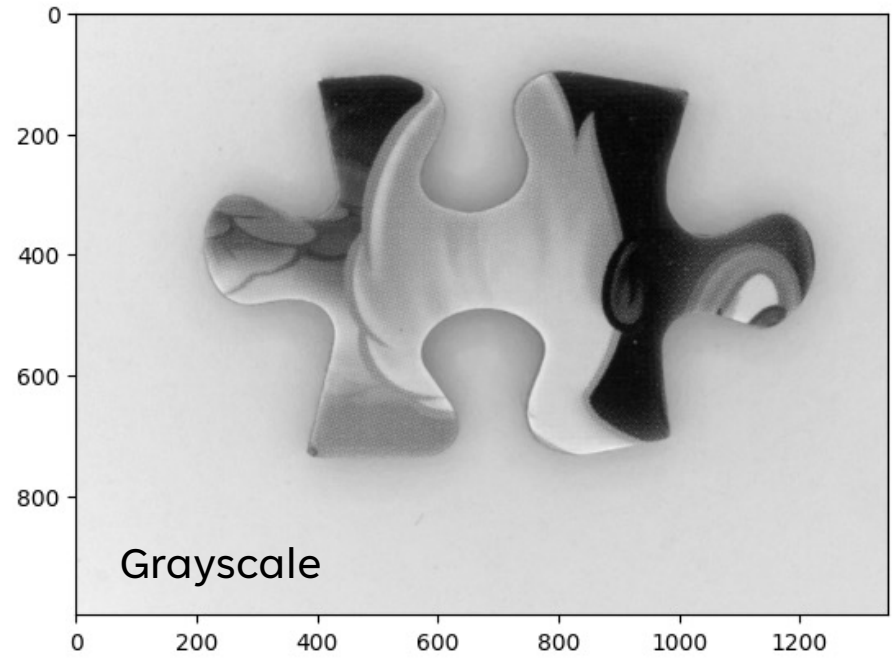
Watershed – X

Edge Detection - ✓

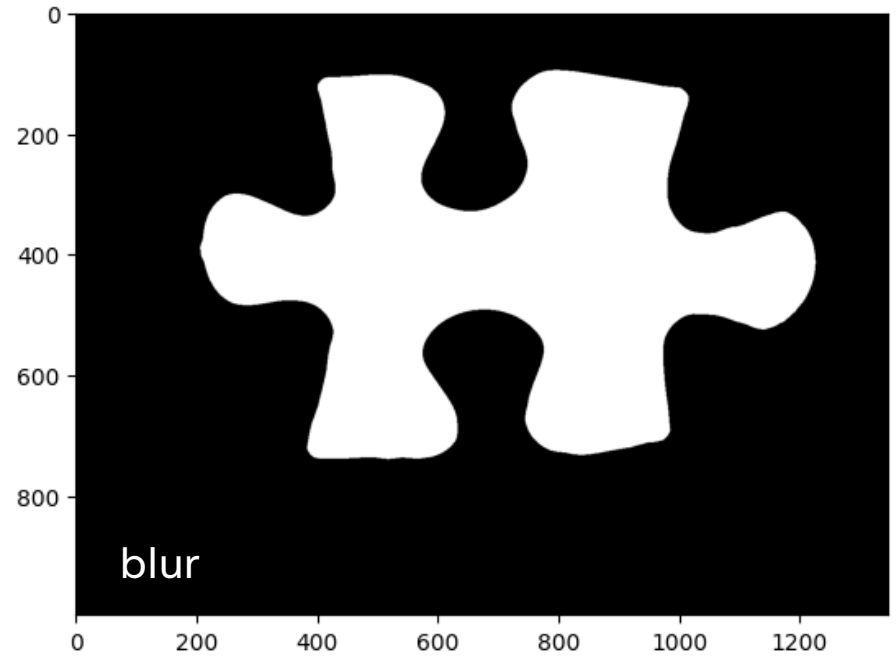
Aber, würde das Projekt noch weiter laufen…

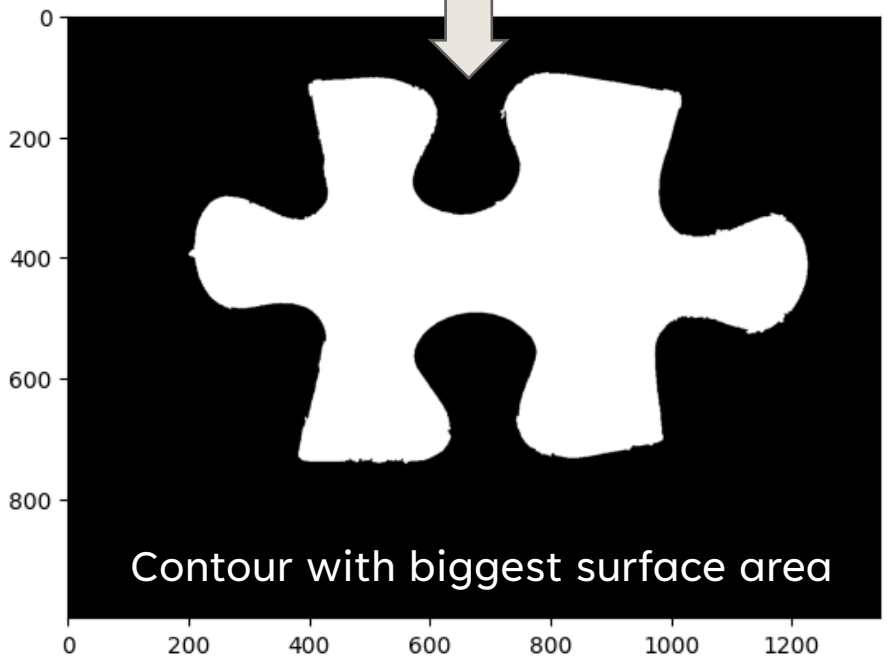
… watershed nochmal probieren!

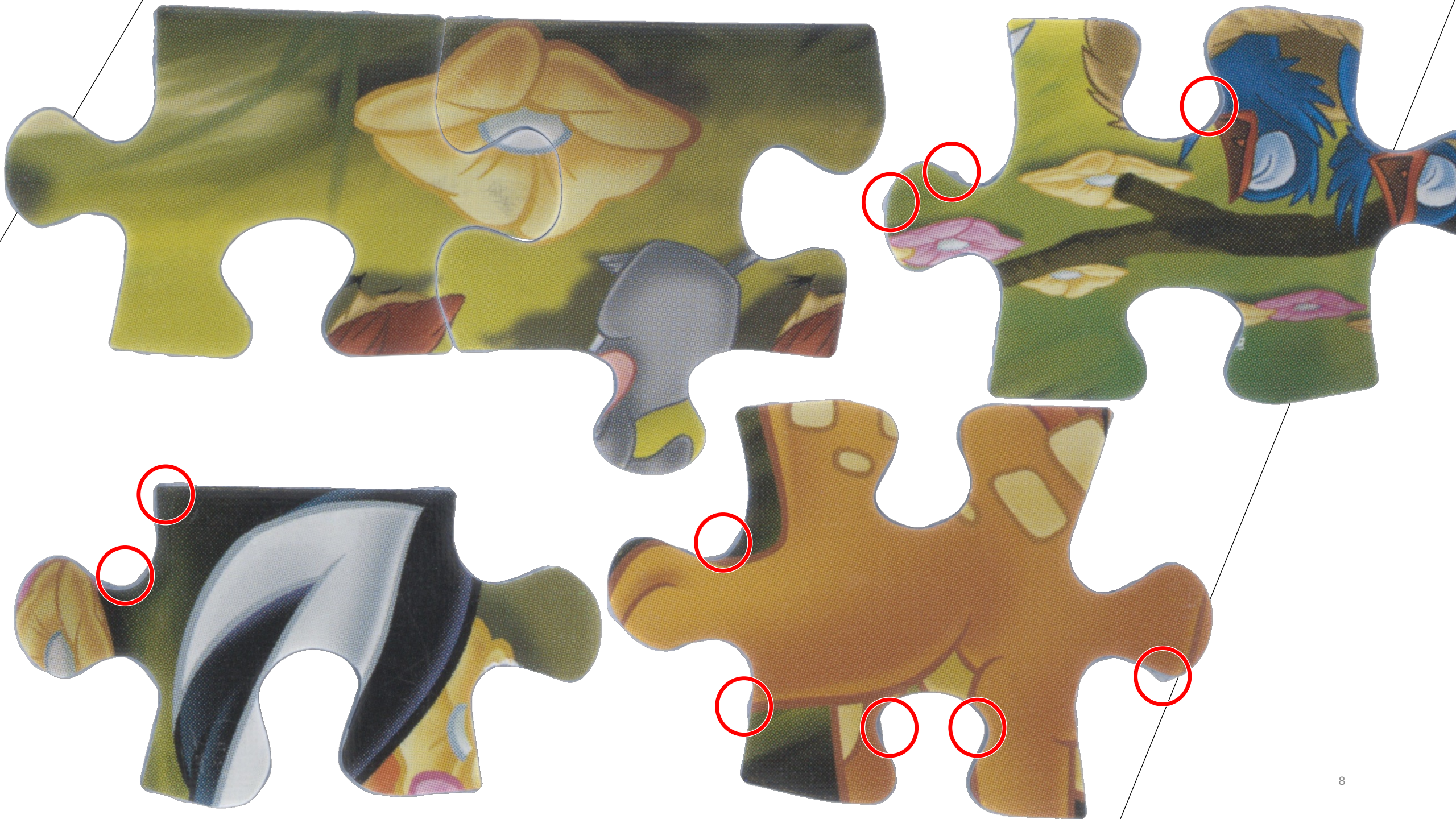
EDGE-DETECTION SEGMENTIERUNG


Grayscale


Enhanced edges with Canny


blur

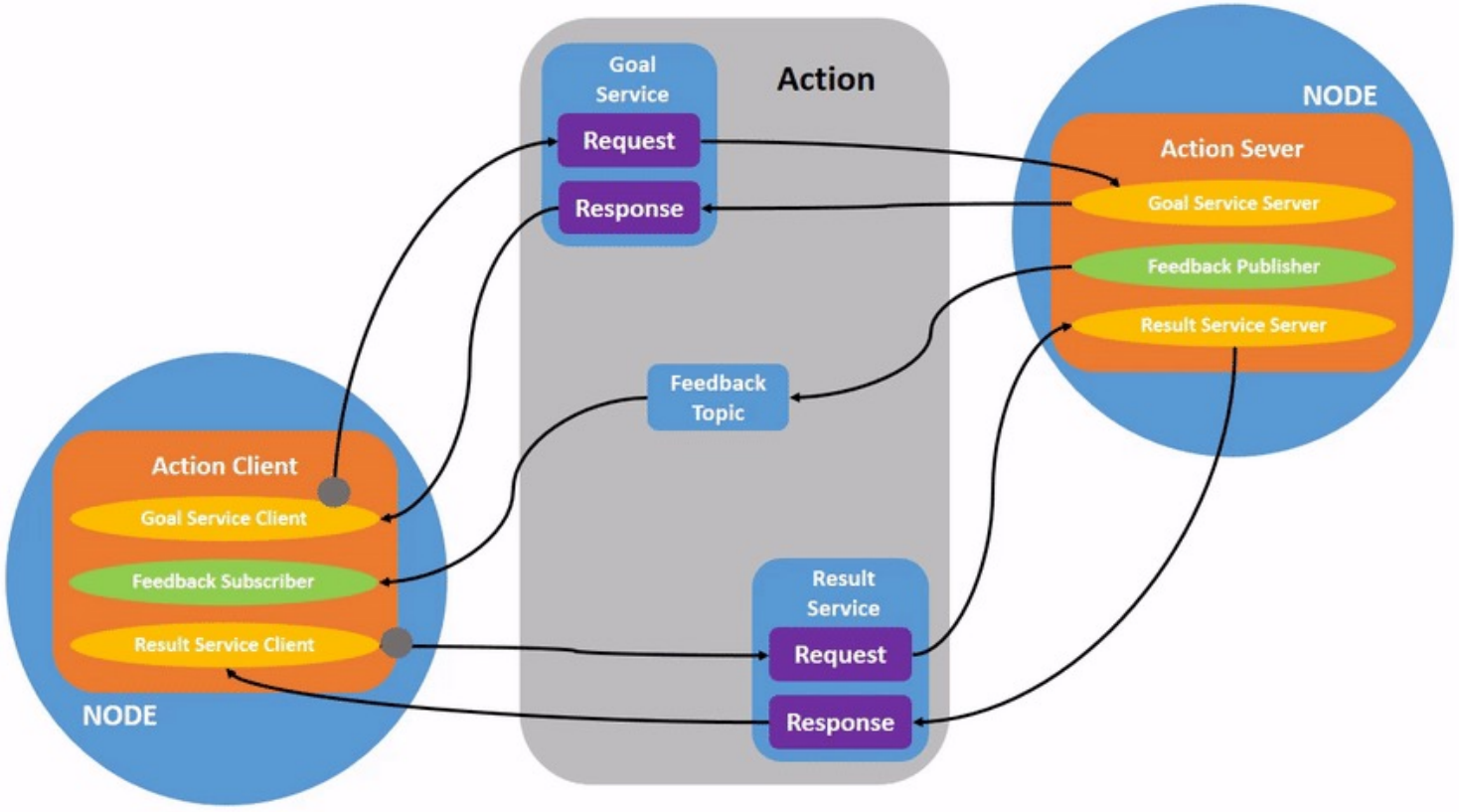

Contour with biggest surface area

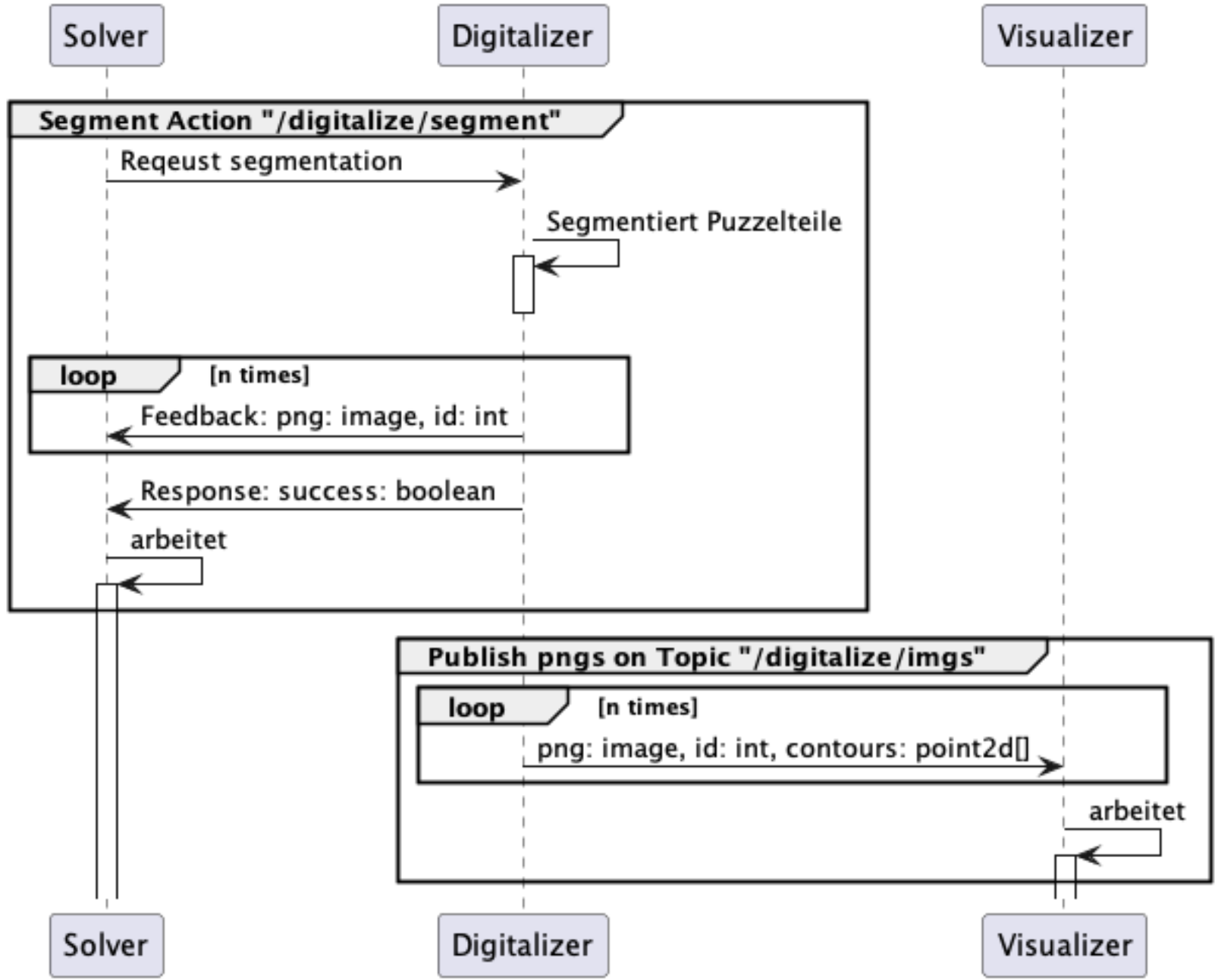




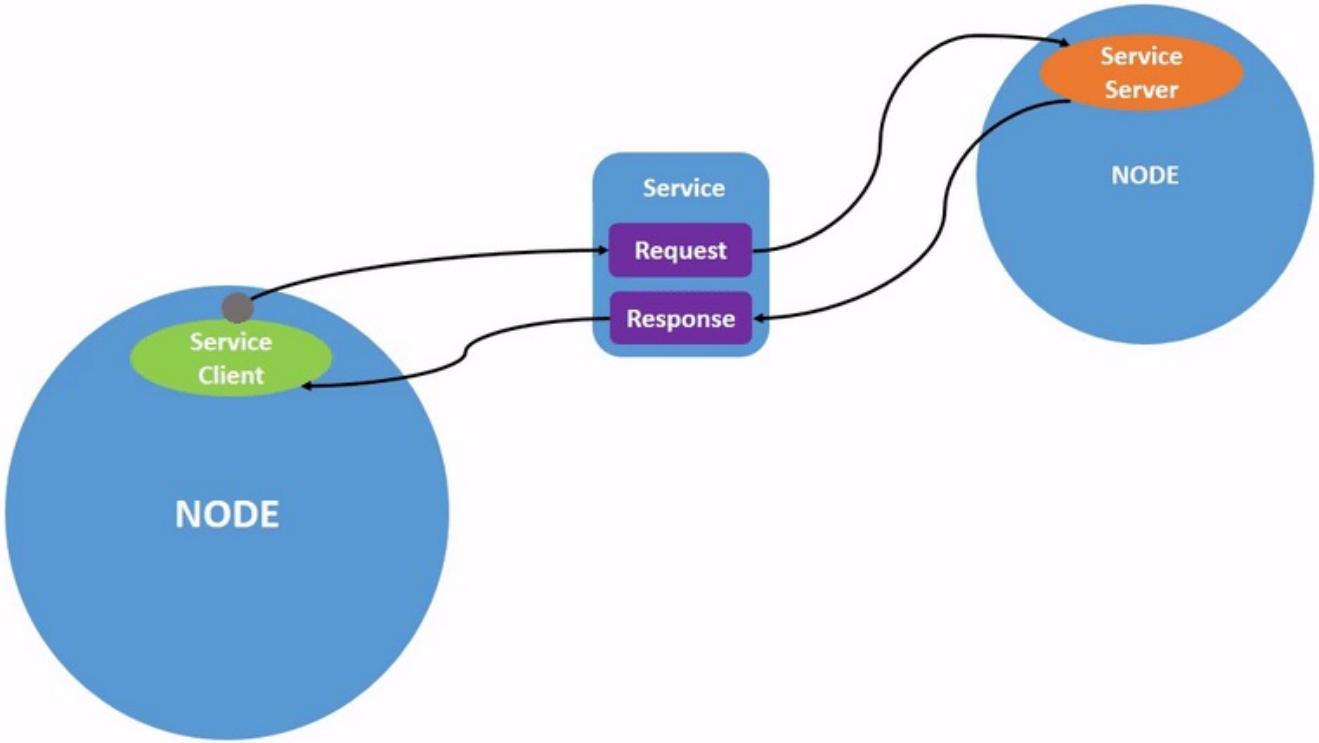
ROS-NODE

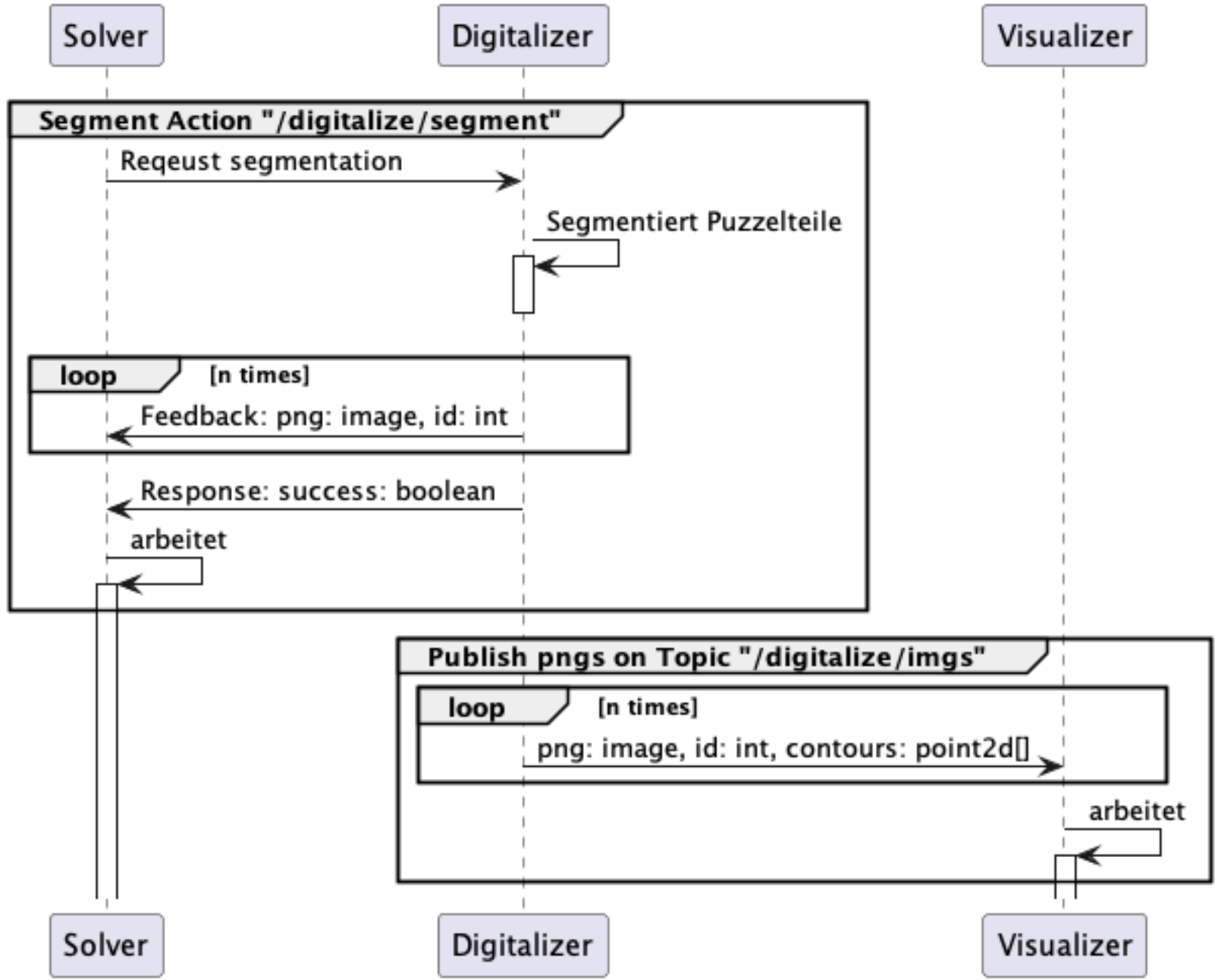
ROS NODE – KOMMUNIKATION – /DIGITALIZE/SEGMENT



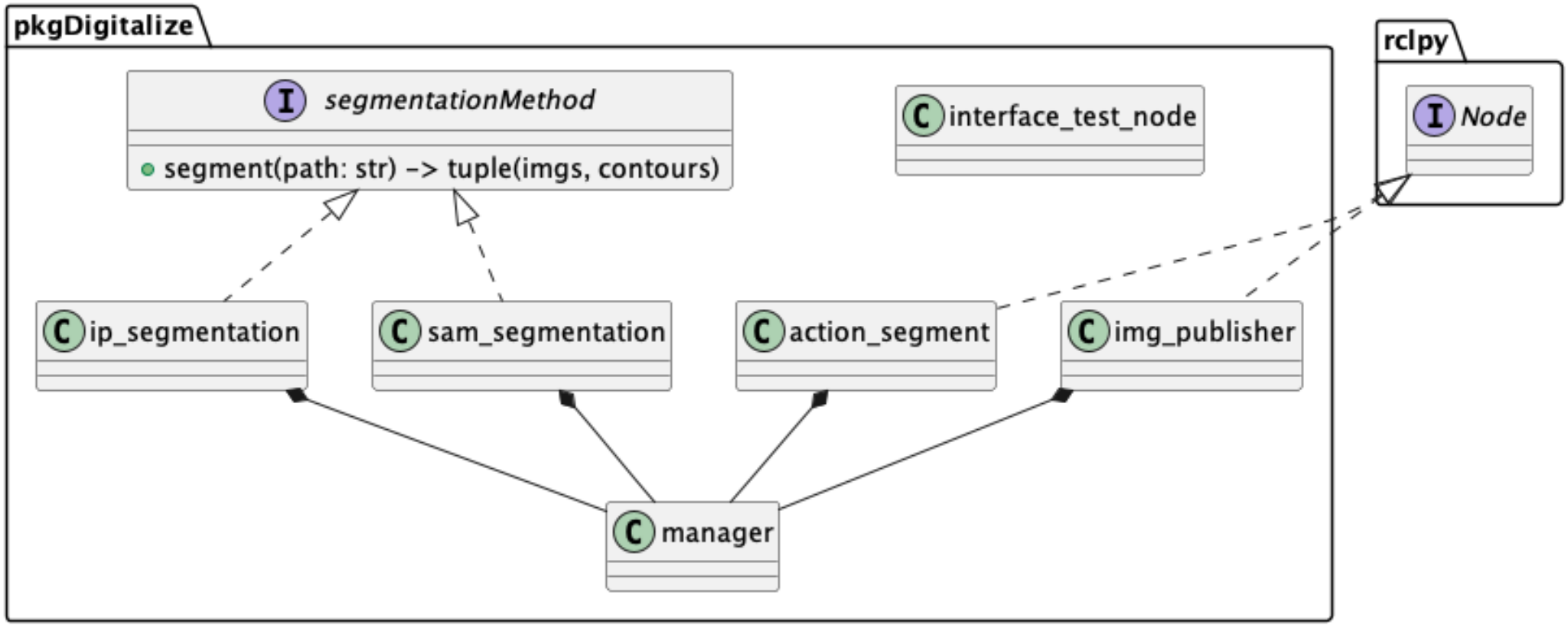

Solver
Digitalizer
Visualizer
Segment Action "/digitalize/segment"
Reqeust segmentation
Segmentiert Puzzelteile
loop
[n times]
Feedback: png: image, id: int
Response: success: boolean
arbeitet
Publish pngs on Topic "/digitalize/imgs"
loop
[n times]
png: image, id: int, contours: point2d[]
arbeitet
Solver
Digitalizer
Visualizer

ROS NODE – KOMMUNIKATION - /DIGITALIZE/IMGS



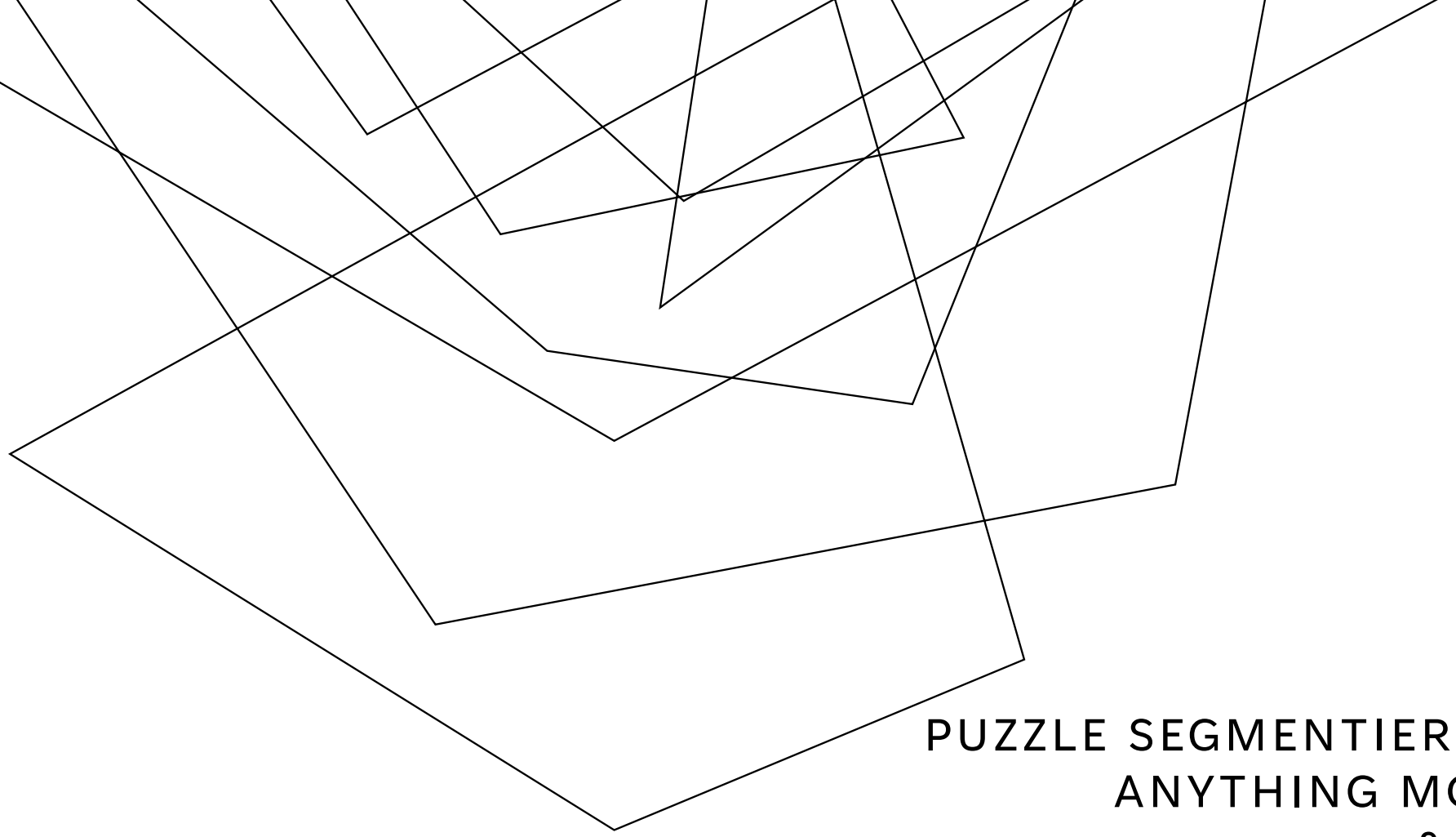

Solver
Digitalizer
Visualizer
Segment Action "/digitalize/segment"
Reqeust segmentation
Segmentiert Puzzelteile
loop
[n times]
Feedback: png: image, id: int
Response: success: boolean
arbeitet
Publish pngs on Topic "/digitalize/imgs"
loop
[n times]
png: image, id: int, contours: point2d[]
arbeitet
Solver
Digitalizer
Visualizer

ROS NODE - ARCHITEKTUR



VIELEN DANK FÜR IHRE AUFMERKSAMKEIT!

Korbinian Kipp



PUZZLE SEGMENTIERUNG MIT SEGMENT ANYTHING MODEL (SAM) & POST-PROCESSING

ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

GLIEDERUNG

- Einführung
  - Das Ziel
- SAM
  - Das Ergebnis
- Post-Processing
  - RGBA
  - SVG

GLIEDERUNG

- Einführung
  - Das Ziel
- SAM
  - Das Ergebnis
- Post-Processing
  - RGBA
  - SVG

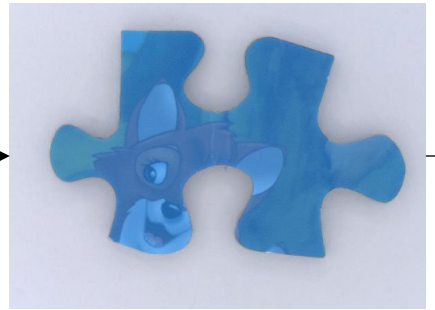
EINFÜHRUNG – DAS ZIEL

Sehr sehr sehr sehr kurz gesagt…





Mr. SAM





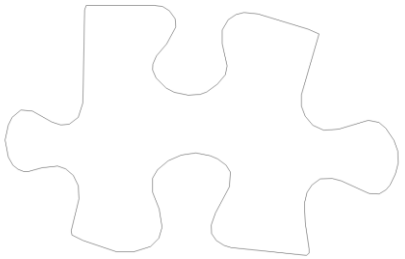


Bild Quelle : https://www.hollywoodreporter.com/business/digital/why-ai-isnt-funny-at-least-not-yet-1235503678/

GLIEDERUNG

- Einführung
  - Das Ziel
- SAM
  - Das Ergebnis
- Post-Processing
  - RGBA
  - SVG

SAM – WAS IST & WARUM SAM?

Was? SAM ist ein KI-Modell!

- Wie der Name schon sagt, liegt der Schwerpunkt auf der Segmentierung der Objekte auf den Bildern.

Warum? SAM bietet ein Zero-Shot Masterclass an!

- Beeindruckende Zero-Shot-Generalisierung Leistung.

SAM – IM EINSATZ

Schritte:

1. Eingabe des Bildes
2. Eingabe der Modellparameter
3. Warten... (Inferenz)
4. Extraktion der Segmentierungsmaske

Entzerrte Bilder als Eingabebild:



SAM – IM EINSATZ

Schritte:

1. Eingabe des Bildes
2. Eingabe der Modellparameter
3. Warten... (Inferenz)
4. Extraktion der Segmentierungsmaske

Bounding Box/Region erstellen:

```
[ ]  input_box = np.array([80, 80, 1300, 900])

     masks, scores, logits = predictor.predict(
         point_coords=None, #input_point if using option 1
         point_labels=None, #input_label if using option 1
         box=input_box[None, :],
         multimask_output=True,
     )
```

SAM – IM EINSATZ

Schritte:

1. Eingabe des Bildes
2. Eingabe der Modellparameter
3. Warten... (Inferenz)
4. Extraktion der Segmentierungsmaske

Inferenz und auf die Segmentierungsmaske warten.



GLIEDERUNG

- Einführung
  - Das Ziel
- SAM
  - Das Ergebnis
- Post-Processing
  - RGBA
  - SVG

POST-PROCESSING: GÜLTIG FÜR BEIDE METHODEN

(damit auch die anderen Teams Spaß daran haben können)

Gewünschte Outputs:

1. Puzzleteil mit transparentem Hintergrund (RGBA) – Team Solver
2. Konturen des Puzzleteils (SVG) – Team Visualizer

POST-PROCESSING

Gewünschte Outputs:

1. Puzzleteil mit transparentem Hintergrund (RGBA)
2. Konturen des Puzzleteils (SVG)

Verwendung der Segmentierungsmaske zum Ausschneiden des Region Of Interests (ROI)

Original Puzzle piece



Trimmed Puzzle piece

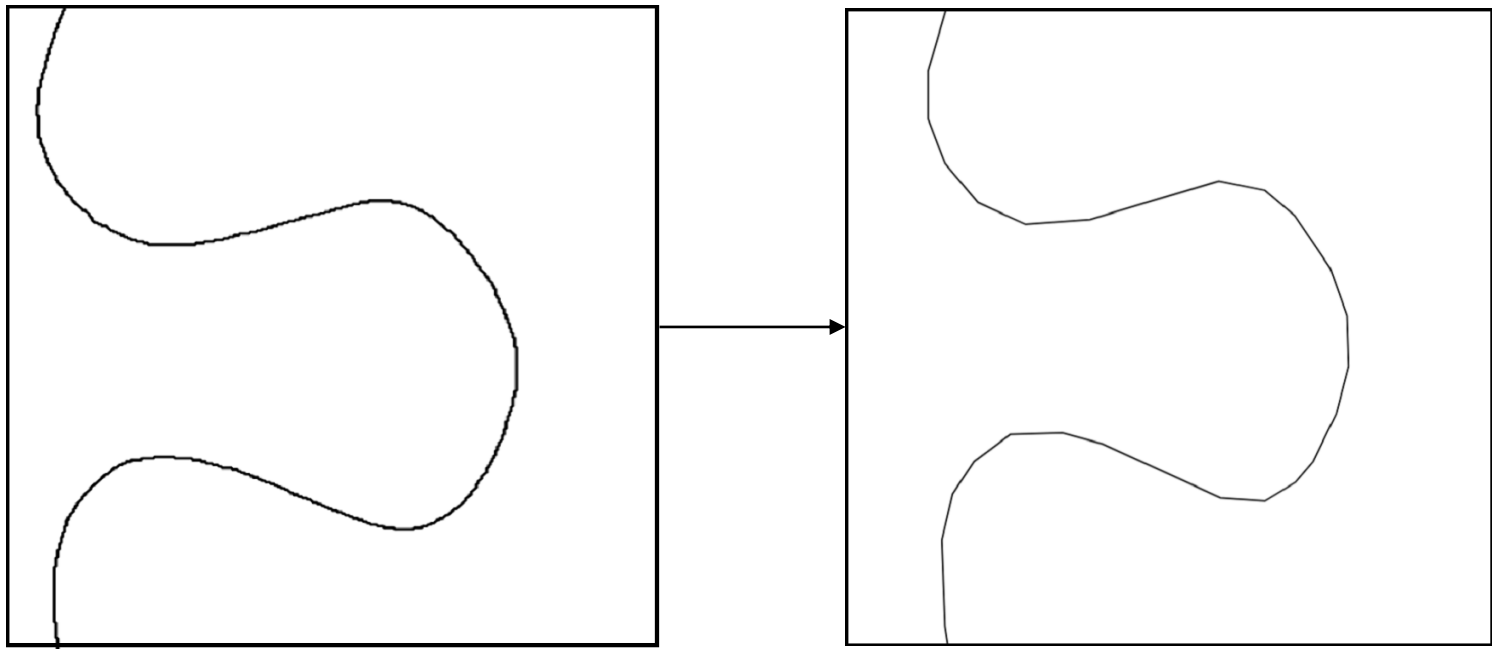


POST-PROCESSING

Gewünschte Outputs:

1. Puzzleteil mit transparentem Hintergrund (RGBA)
2. Konturen des Puzzleteils (SVG)

Verwendung des Ramer-Douglas-Peucker-Algorithmus (RDP) für die Approximierung von Kanten (epsilon = 3)

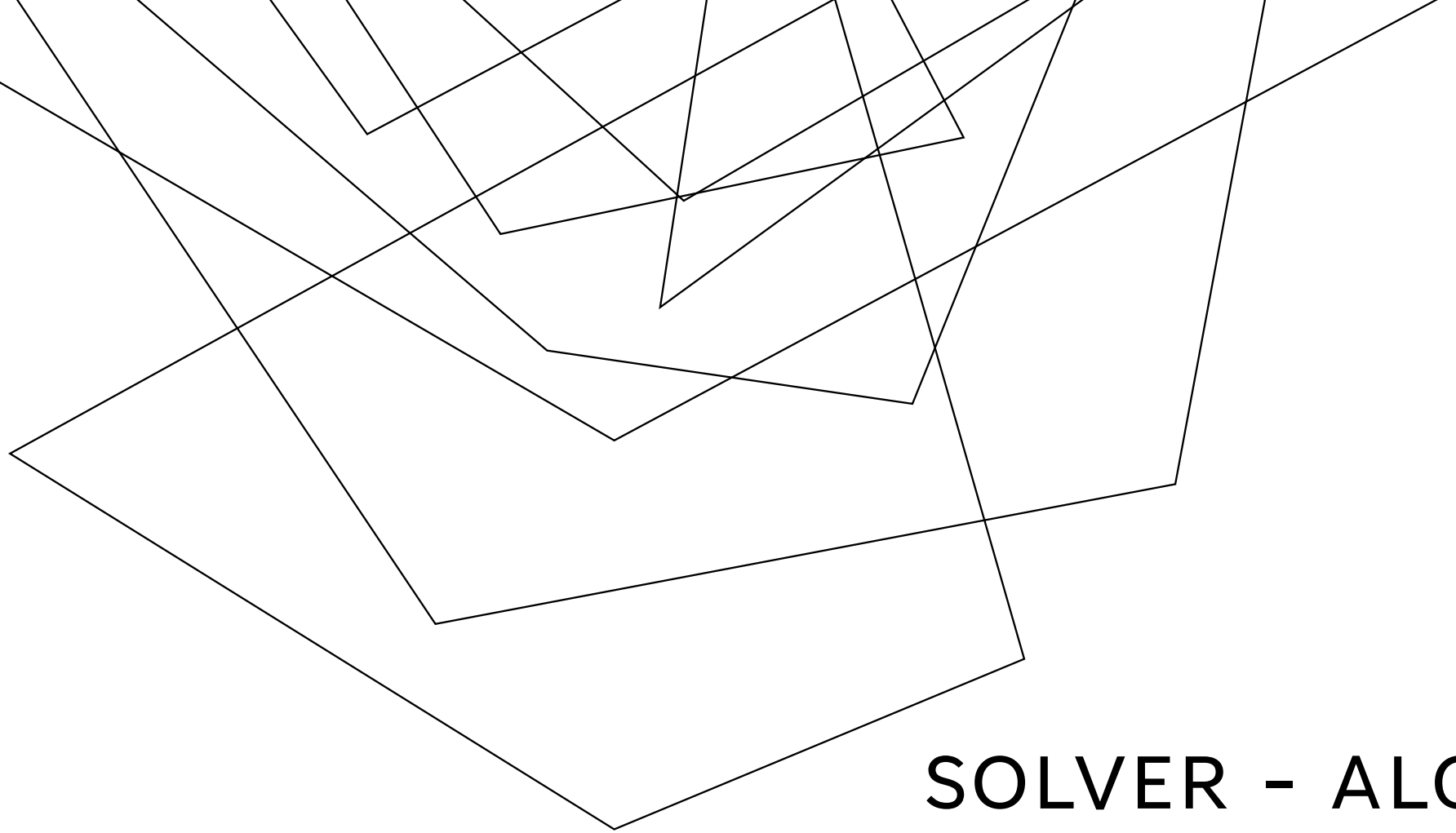


epsilon = Die Toleranz für das Herausfiltern von Ausreißern aus der vereinfachten Kurve.

VIELEN DANK FÜR IHRE AUFMERKSAMKEIT!

B. Sean Then

@then (gitlab)



SOLVER - ALGORITHMEN

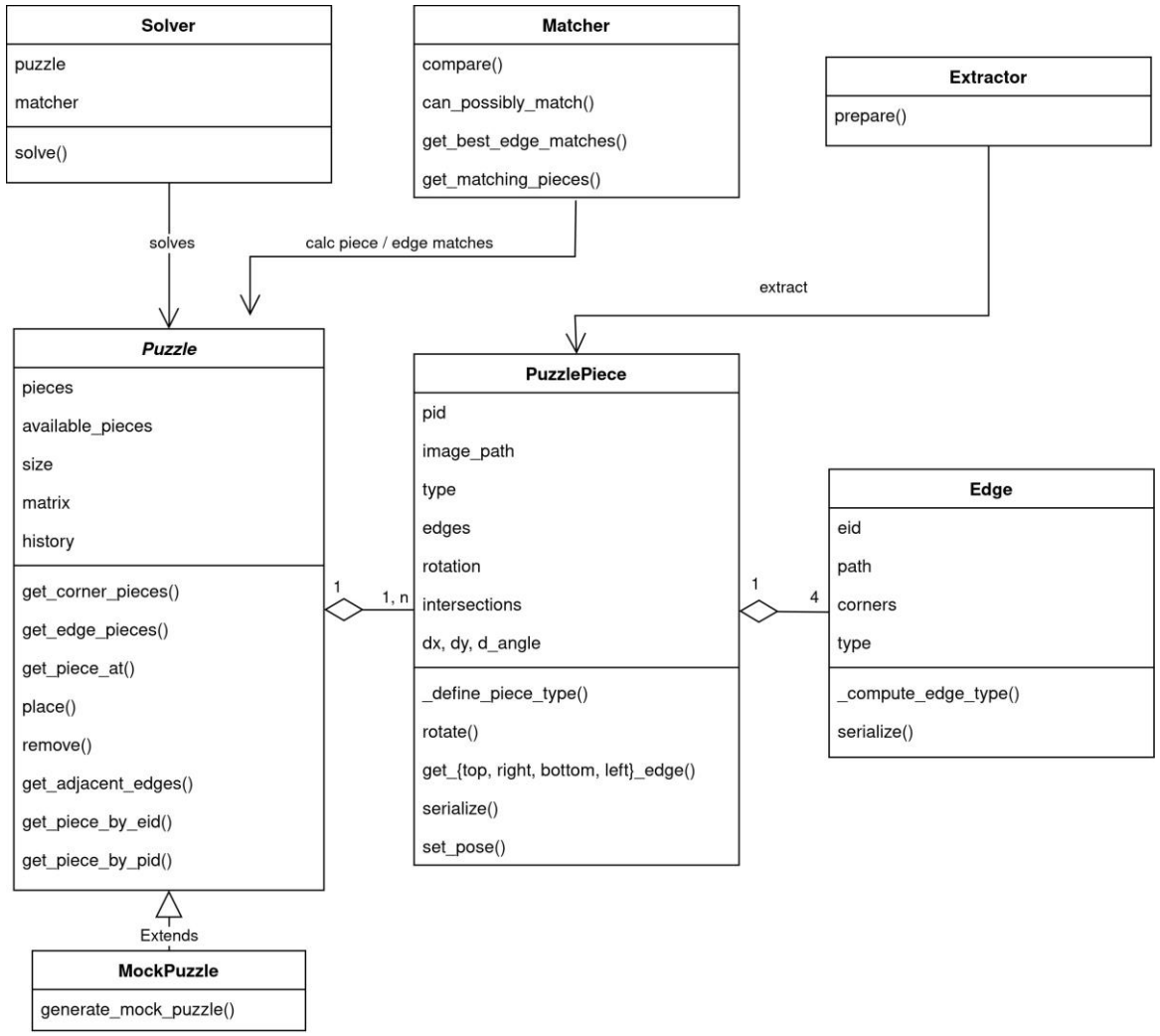
ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

GLIEDERUNG

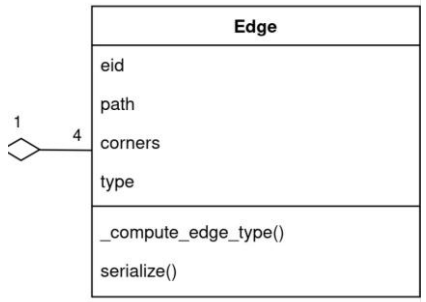
- Klassenstruktur
- Rekursive Algorithmen
  - Tiefensuche
  - Best-First-Algorithmus

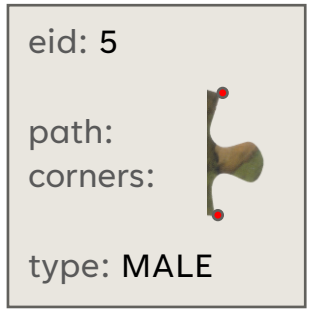
SOLVER / EINLEITUNG / KLASSENSTRUKTUR

SOLVER / EINLEITUNG / KLASSENSTRUKTUR

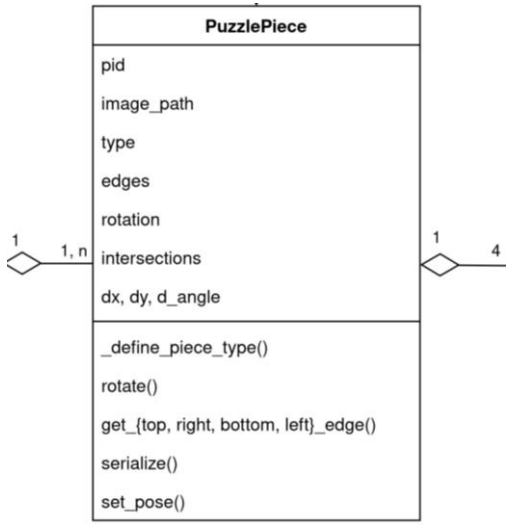


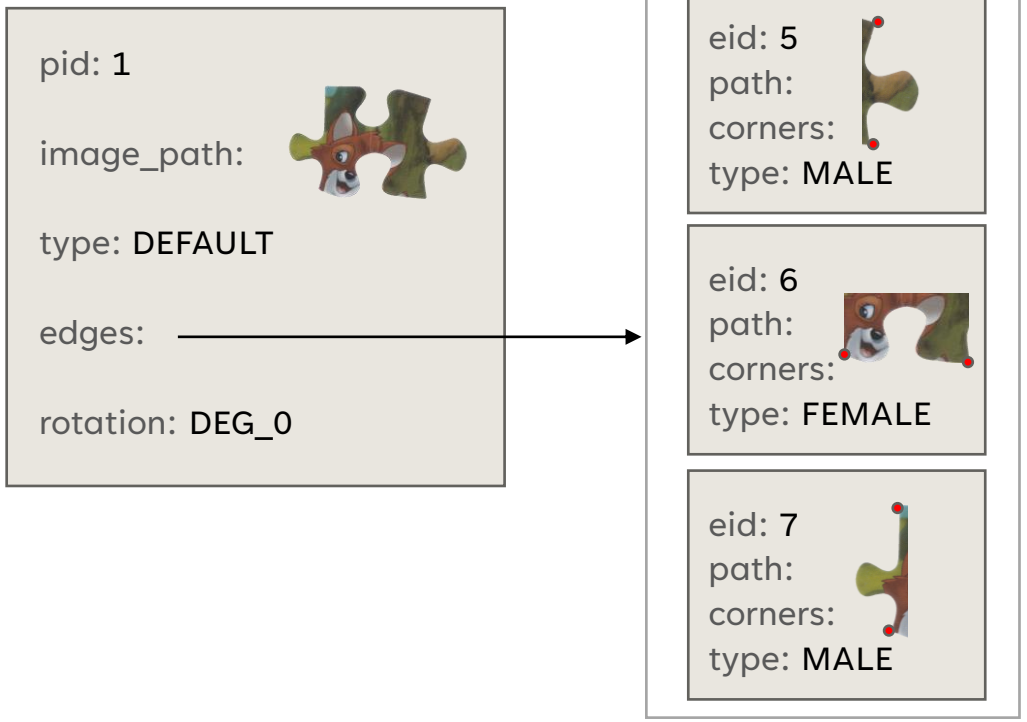
SOLVER / EINLEITUNG / KLASSENSTRUKTUR



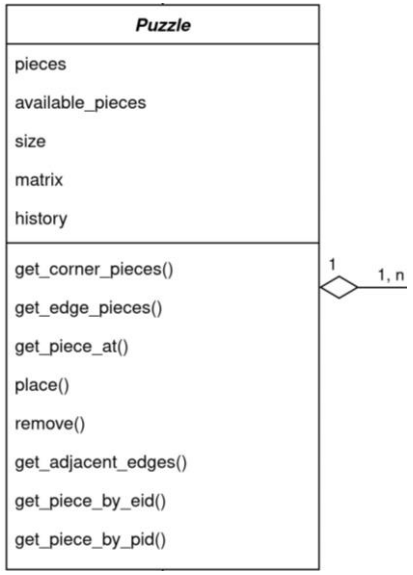


SOLVER / EINLEITUNG / KLASSENSTRUKTUR





SOLVER / EINLEITUNG / KLASSENSTRUKTUR

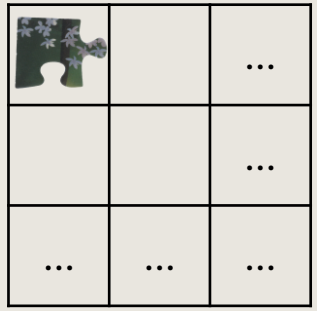


pieces: []

available_pieces: []

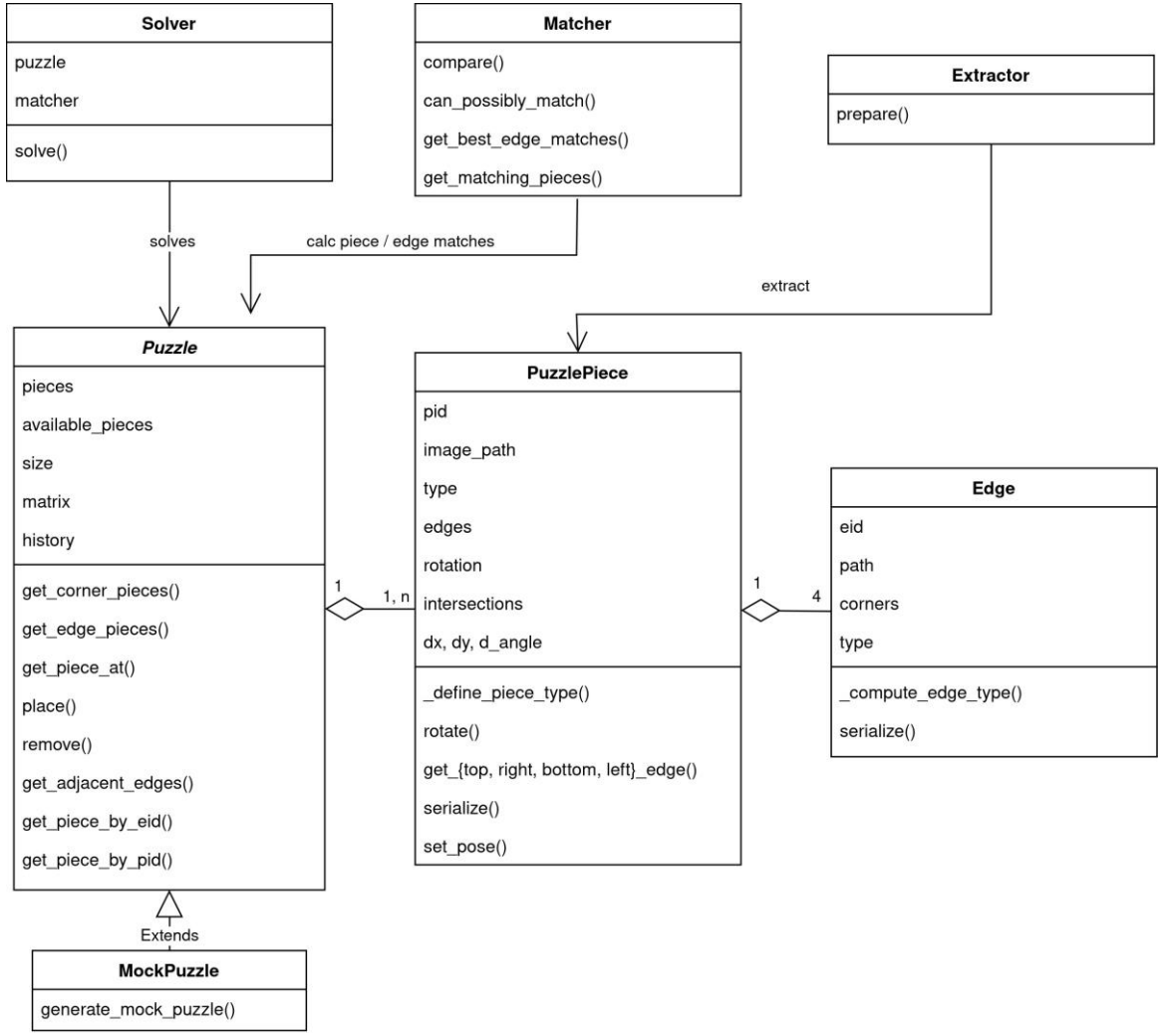
size: (7, 7)

matrix:

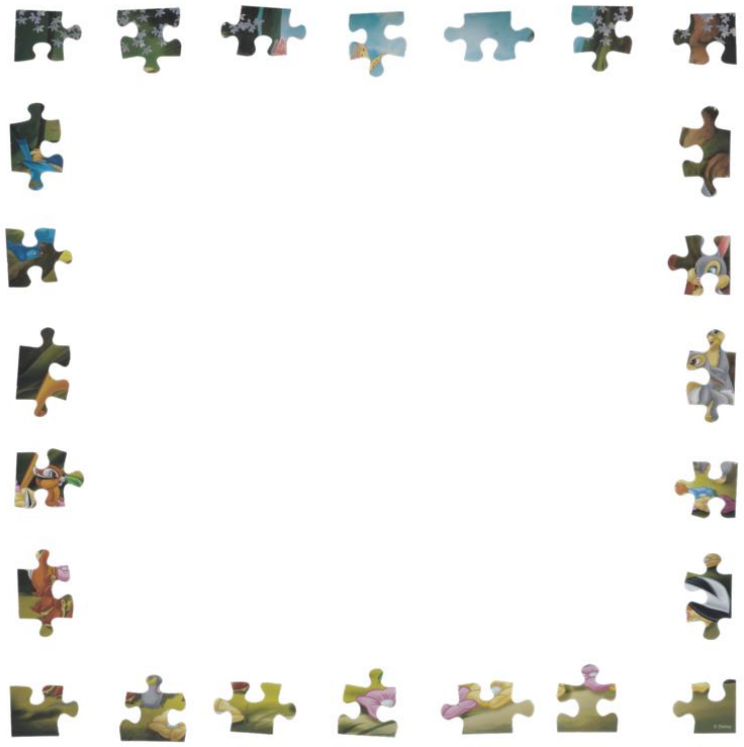


history: [(0,0), ..]

SOLVER / EINLEITUNG / KLASSENSTRUKTUR



SOLVER / ALGORITHMEN / TIEFENSUCHE

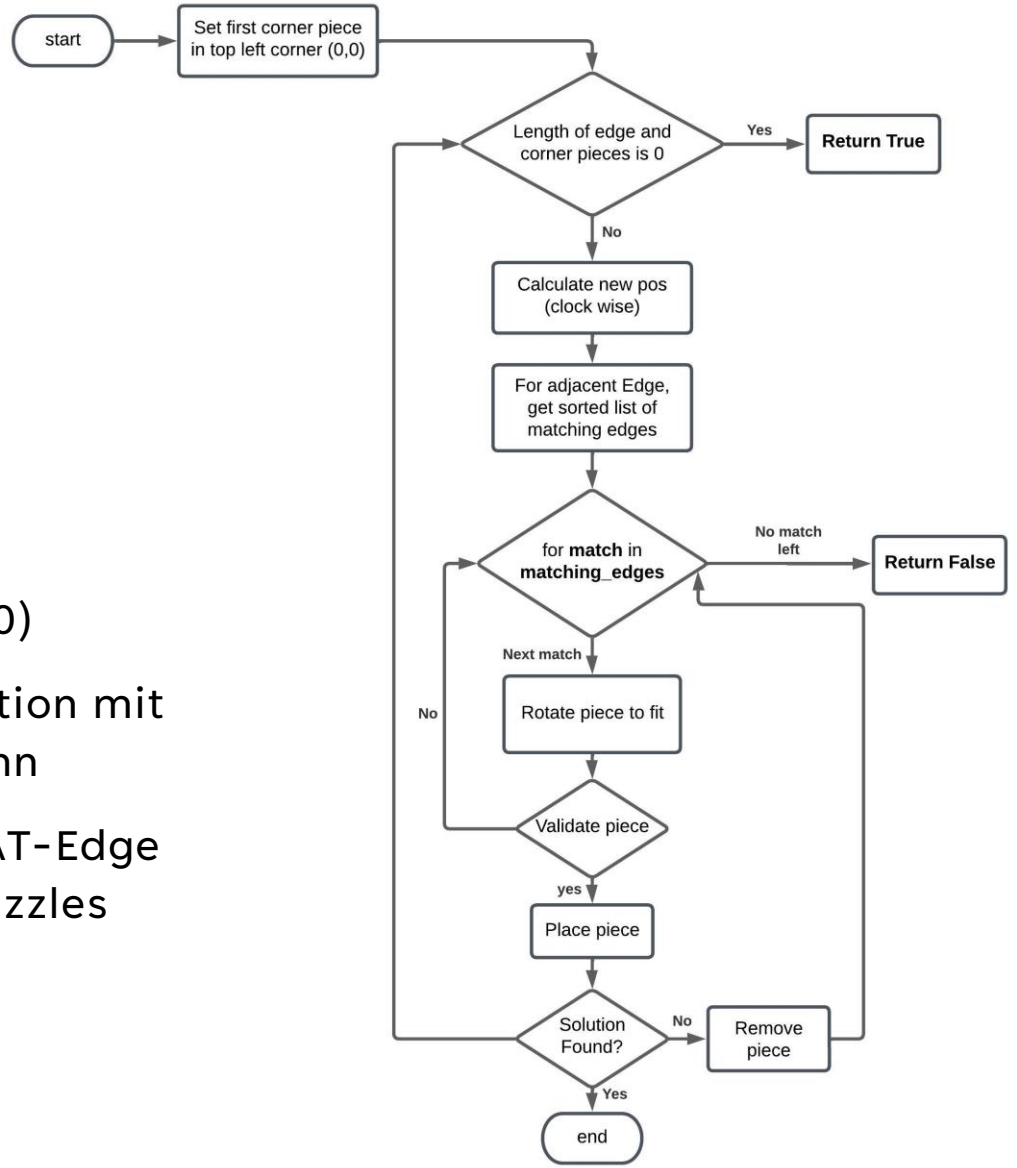


SOLVER / ALGORITHMEN /
FRAME BUILDER

- Teilproblem ist leichter zu lösen
- Betrachtung von CORNER- und EDGE-Pieces
- Löst das Dimensionsproblem

SOLVER / ALGORITHMEN / FRAME BUILDER

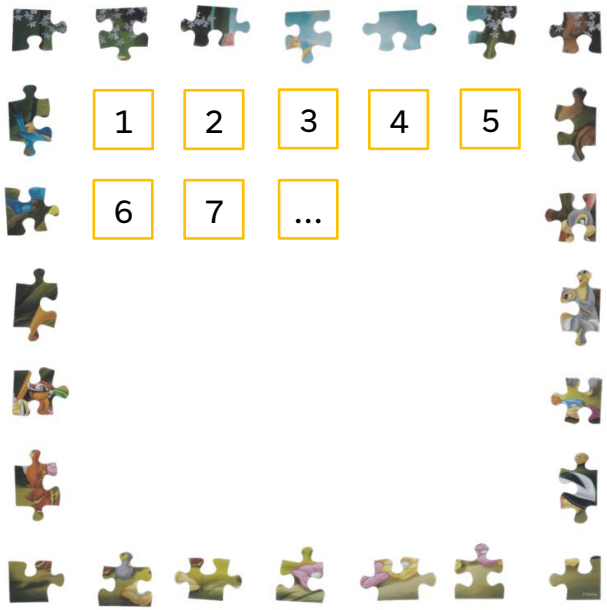
- Rekursiver Aufbau
- Initialisierung mit Ecke bei (0,0)
- Berechnung der nächsten Position mit Build Direction im Uhrzeigersinn
- Validierung möglich durch FLAT-Edge und Größeninformation des Puzzles



SOLVER / ALGORITHMEN / TIEFENSUCHE

Algorithm 3 Depth-Search-Algorithmus

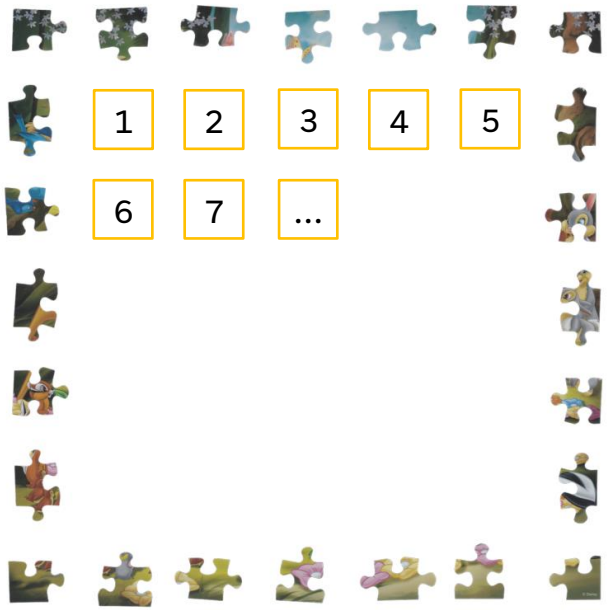
```
1:  Eingabe: Puzzle mit gelöstem Rahmen
2:  Ausgabe: gelöstes Puzzle
3:  start_pos ← (1,1)
4:  NEXT_PIECE(start_pos)
5:                                  ▷ Rekursive Funktion für jedes weitere Puzzleteil
6:  function NEXT_PIECE(pos)
7:      available_pieces ← puzzle.available_pieces
8:      if len(available_pieces) == 0 then                    ▷ Abbruchbedingung
9:          return True
10:     end if
11:     adjacent_edges ← puzzle.get_adjacent_edges(pos)
12:     matching_pieces ← matcher.get_matching_pieces(adjacent_edges)
13:     for matching_piece ∈ matching_pieces do
14:         Rotiere matching_piece
15:         Platziere matching_piece an pos
16:         new_pos ← get_next_position(pos)      ▷ Berechnet Position Reihe für Reihe
17:         if NEXT_PIECE(new_pos) then
18:             return True
19:         else
20:             Entferne matching_piece an pos
21:         end if
22:     end for
23: end function
```



SOLVER / ALGORITHMEN / TIEFENSUCHE

Algorithm 3 Depth-Search-Algorithmus

```
1:  Eingabe: Puzzle mit gelöstem Rahmen
2:  Ausgabe: gelöstes Puzzle
3:  start_pos ← (1,1)
4:  NEXT_PIECE(start_pos)
5:                                  ▷ Rekursive Funktion für jedes weitere Puzzleteil
6:  function NEXT_PIECE(pos)
7:      available_pieces ← puzzle.available_pieces
8:      if len(available_pieces) == 0 then                    ▷ Abbruchbedingung
9:          return True
10:     end if
11:     adjacent_edges ← puzzle.get_adjacent_edges(pos)
12:     matching_pieces ← matcher.get_matching_pieces(adjacent_edges)
13:     for matching_piece ∈ matching_pieces do
14:         Rotiere matching_piece
15:         Platziere matching_piece an pos
16:         new_pos ← get_next_position(pos)      ▷ Berechnet Position Reihe für Reihe
17:         if NEXT_PIECE(new_pos) then
18:             return True
19:         else
20:             Entferne matching_piece an pos
21:         end if
22:     end for
23: end function
```



SOLVER / ALGORITHMEN / TIEFENSUCHE

- *adjacent_edges* = puzzle.get_adjacent_edges((1, 1))

```
adjacent_edges = [Edge(34), None, None, Edge(16)]
```

- *matching_pieces* = matcher.get_matching_pieces(*adjacent_edges*)

1. Erstelle Best Match Listen

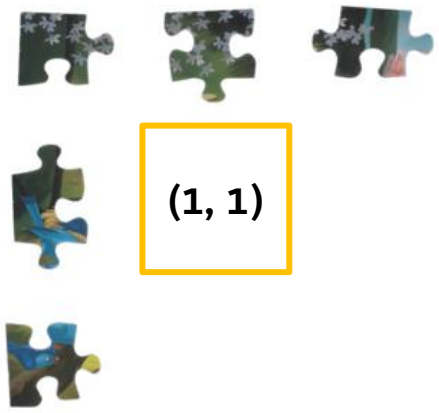
```
edge_34 = [2,8,14, … ]
edge_16 = [0,16,56,… ]
```

2. Berechne PieceID →

```
edge_34 = [0,2,3,… ]
edge_16 = [0,4,14,…]
```

3. Schnittmenge an möglichen Puzzleteilen

```
possible_pieces = [0, ..]
```



SOLVER / ALGORITHMEN / TIEFENSUCHE

4. Berechnung möglicher Piece Matches

```
possible_pieces = [0, …]
```



(0, DEG_0, 0.53)



(0, DEG_180, 0.98)

```
matching_pieces = [(0, DEG_180, 0.98), (0, DEG_0, 0.53), …]
```

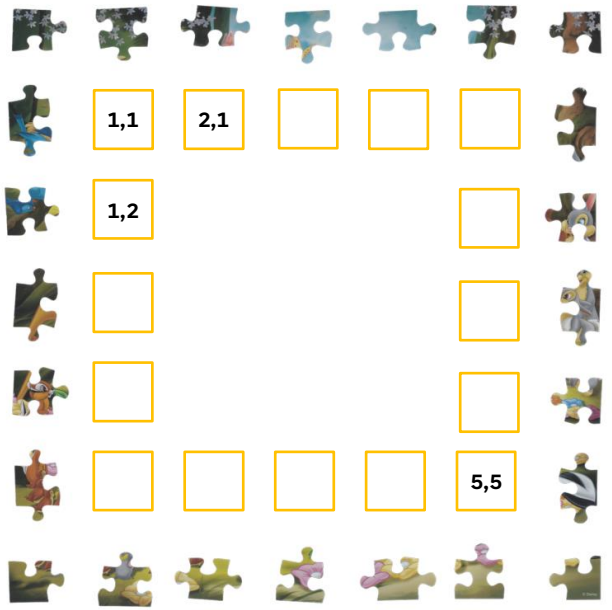


SOLVER / ALGORITHMEN / BEST-FIRST

SOLVER / ALGORITHMEN / BEST-FIRST

Algorithm 4 Best-First-Algorithmus

```
1:  Eingabe: Puzzle mit gelöstem Rahmen
2:  Ausgabe: gelöstes Puzzle
3:  exposed_edges ← Initialisiere mit puzzle          ▷ Alle inneren Kanten des Rahmens
4:  NEXT_PIECE(exposed_edges)
5:                                  ▷ Rekursive Funktion für jedes weitere Puzzleteil
6:  function NEXT_PIECE(exposed_edges)
7:      available_pieces ← puzzle.available_pieces
8:      if len(available_pieces) == 0 then                ▷ Abbruchbedingung
9:          return True
10:     end if
11:     exposed_edge_scores ← get_exposed_edge_scores()
12:     adjacent_pos_scores ← get_best_adjacent_pos(exposed_edge_scores)
13:     for pos ∈ adjacent_pos_scores do
14:         adjacent_edges ← puzzle.get_adjacent_edges(pos)
15:         matching_piece ← matcher.get_matching_pieces(adjacent_edges)[0]
16:         Rotiere matching_piece
17:         Platziere matching_piece an pos
18:         updated_exposed_edges ← Aktualisiere exposed_edges Liste
19:         if NEXT_PIECE(updated_exposed_edges) then
20:             return True
21:         else
22:             Entferne matching_piece an pos
23:         end if
24:     end for
25: end function
```



SOLVER / ALGORITHMEN / BEST-FIRST

- *exposed_edge_scores* = get_exposed_edge_scores()

```
ExEdge = (EID, ADJACENT_POS)

exposed_edge_scores = [(ExEdge(3, (1,1)), 0.97), (ExEdge(34, (3,1)), 0.95),
                       (ExEdge(8, (1,1)), 0.78), …]
```

- *adjacent_pos_scores* = get_best_adjacent_pos(*exposed_edge_scores*)

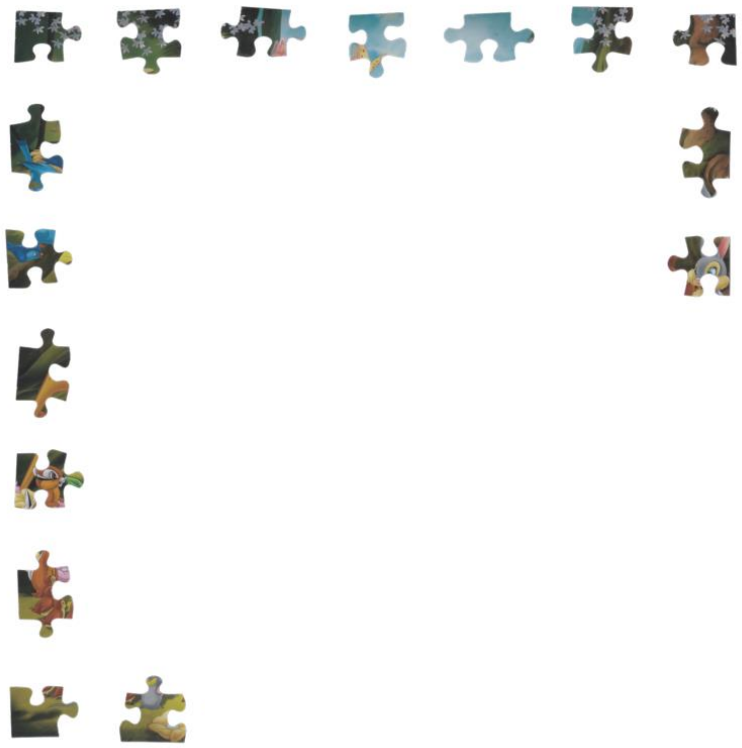
  Summiere Score pro angrenzender Position

```
adjacent_pos_scores = [((1,1), 1.75), ((3,1), 0.95) ..]
```

SOLVER / ALGORITHMEN / EASY-FIRST

- Ähnlich zu Best-First Algorithmus
- Anstatt die Möglichen Edges nach dem Besten Score zu sortieren, werden sie nach der Differenz zum nächsten Match sortiert

```
score_diff = matches[0].score - matches[1].score
```



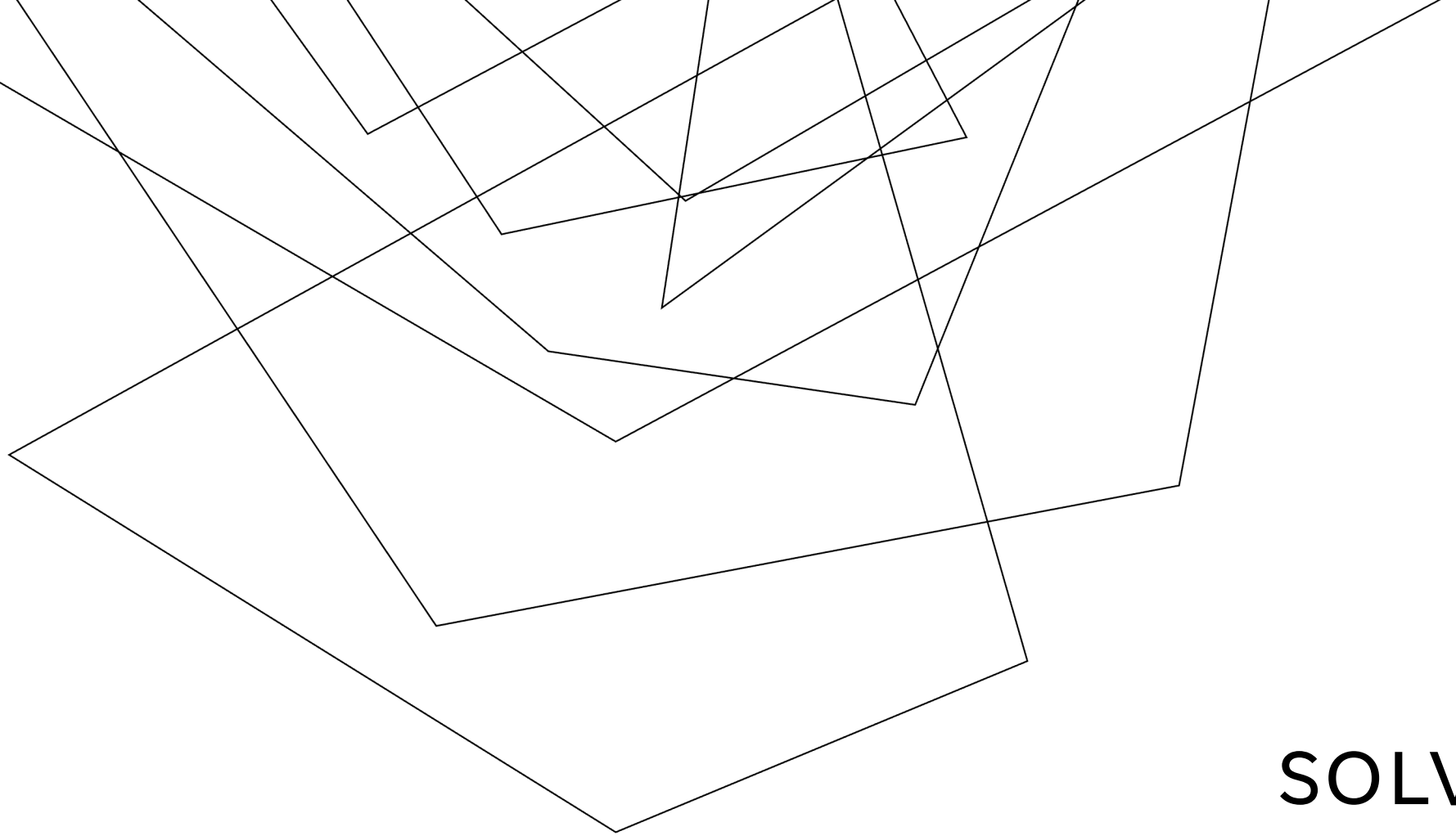
SOLVER / ALGORITHMEN /
OPTIMIERTER FRAME BUILDER

- Ähnlich zu Best-First Algorithmus
- Übergabe von Exposed Edges anstatt Position
- Bester / Leichtester Match kann zwischen zwei statt nur einer Position gewählt werden

VIELEN DANK FÜR EURE AUFMERKSAMKEIT!

Jonas Leitner

@joleitner (gitlab)

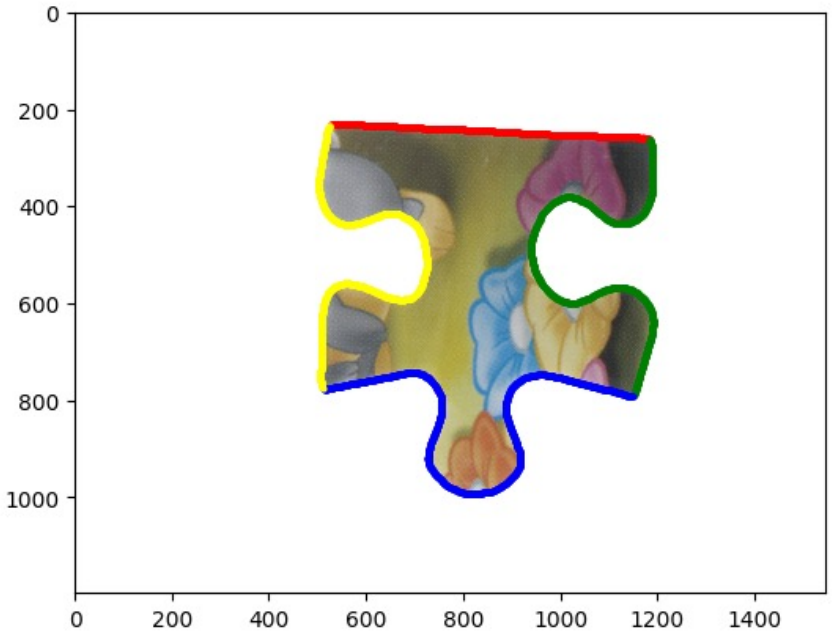


SOLVER

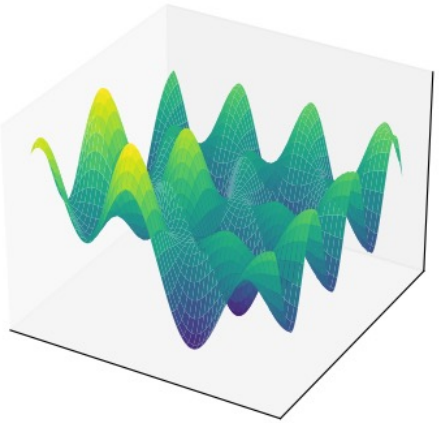
ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

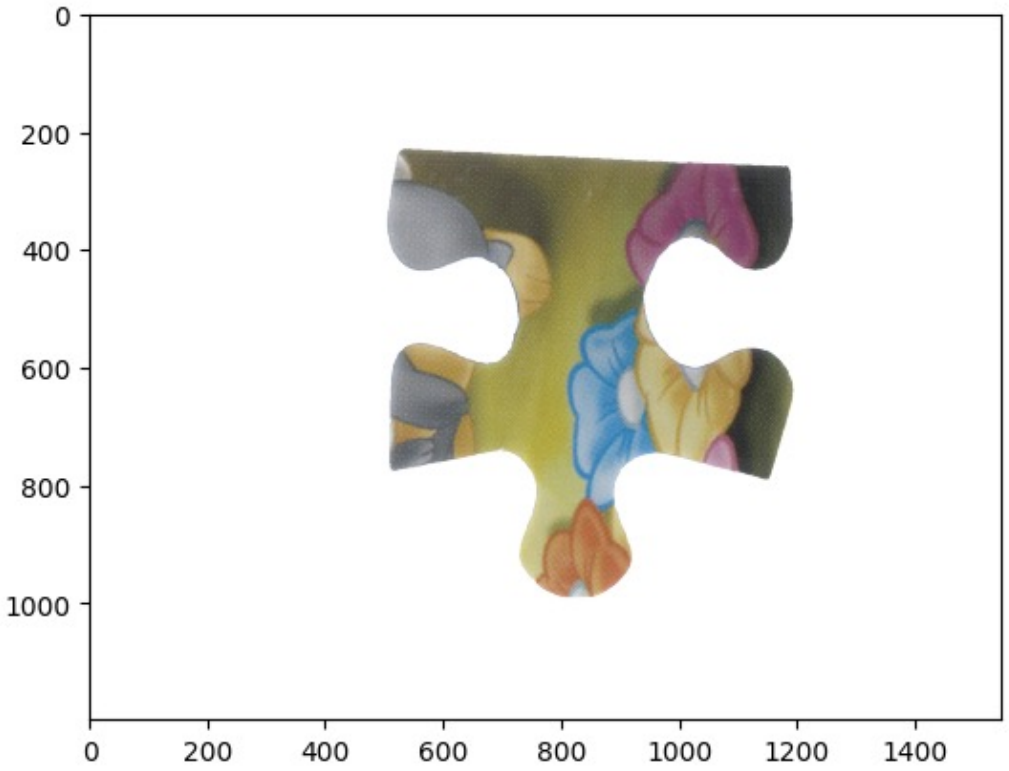
GLIEDERUNG

1. Preprocessing: Edge Extraction
2. Preprocessing: Farb Matching
3. Solving: Simulated Annealing





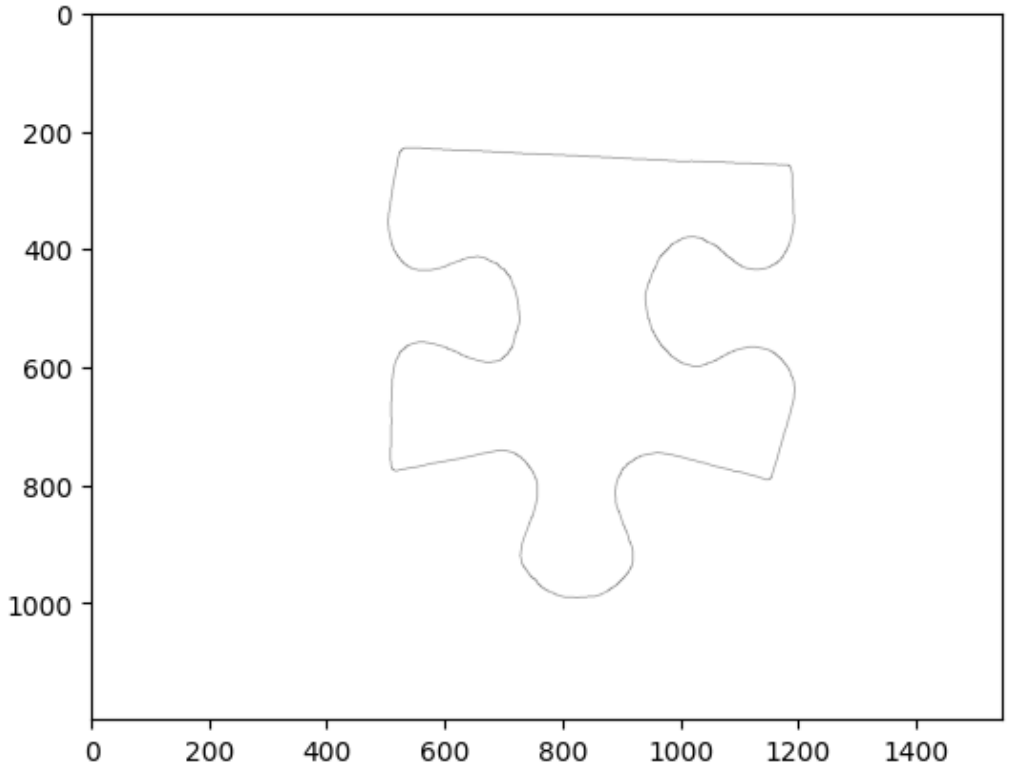




EDGE EXTRACTION

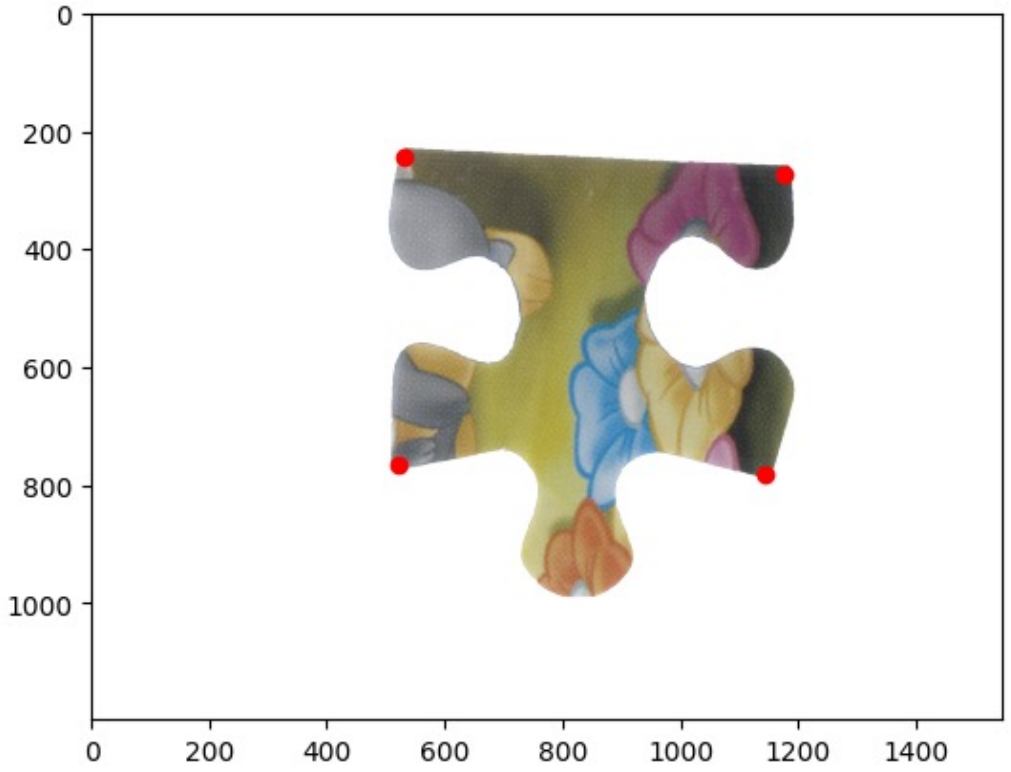
EDGE DETECTION

- Mit OpenCV wird aus dem PNG anhand des Alpha Kanals die Kante extrahiert
- Cv2.Canny()



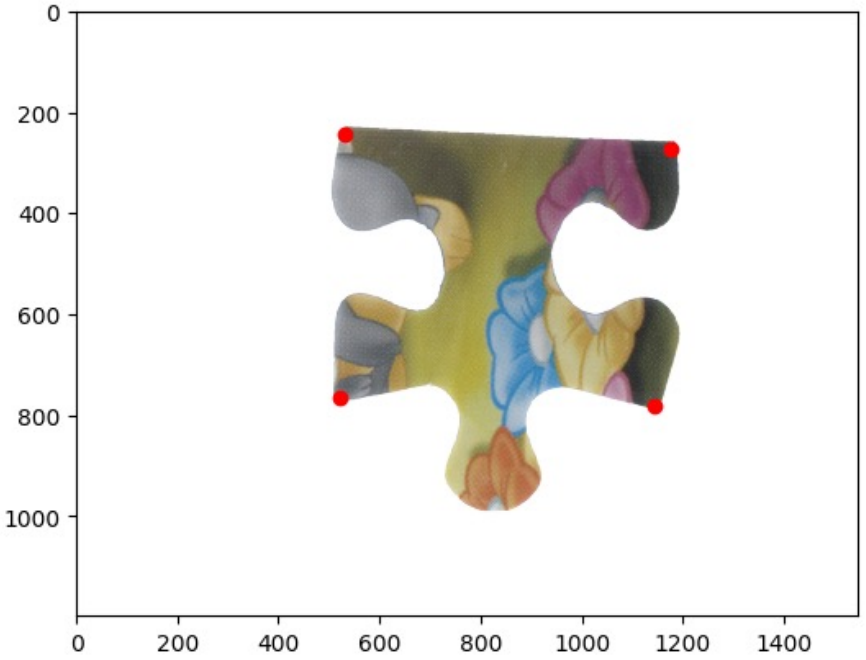
GOOD FEATURES TO TRACK

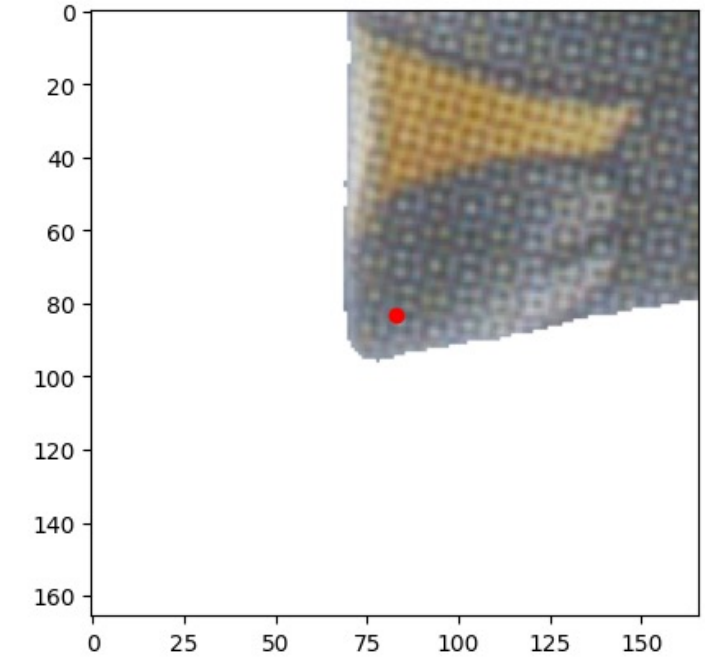
- OpenCV.goodFeaturesToTrack(alpha-Kanal)



GOOD FEATURES TO TRACK

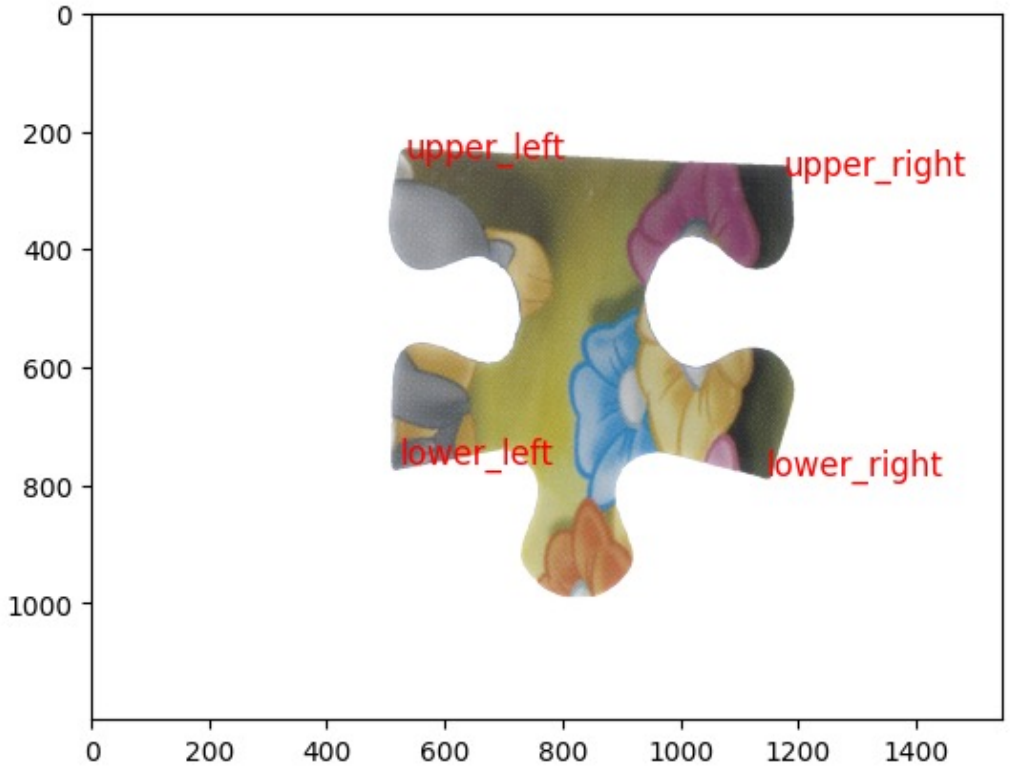
- OpenCV.goodFeaturesToTrack(alpha Kanal)





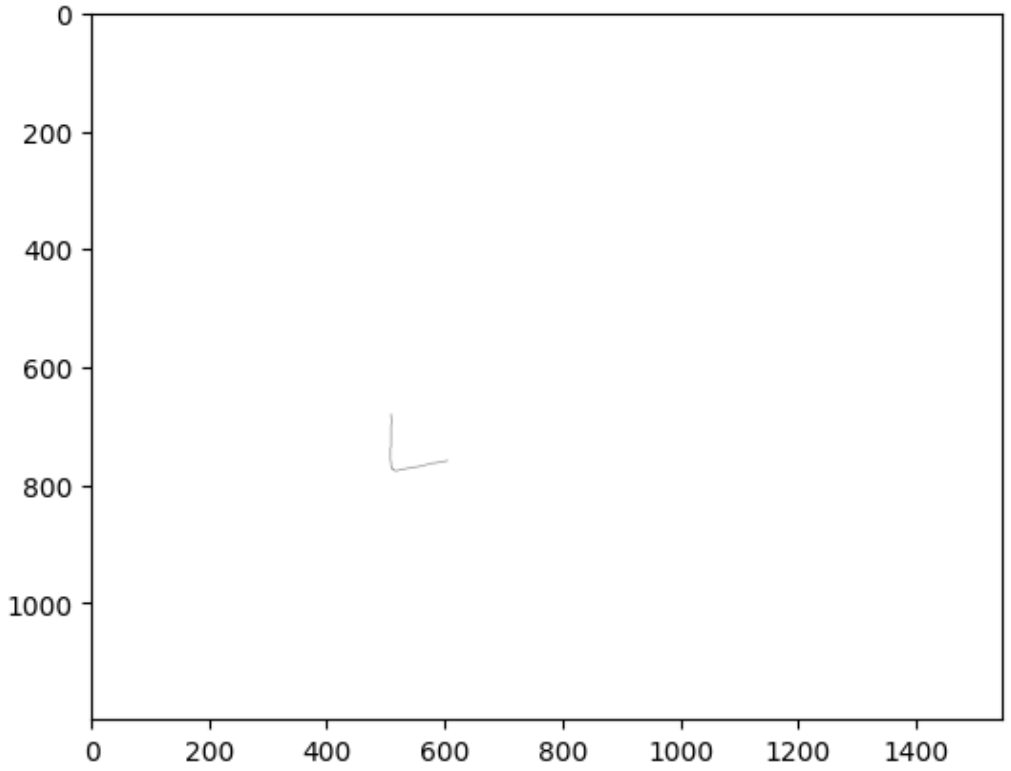
ECKEN LABELN

- Ecken im Uhrzeigersinn definiert



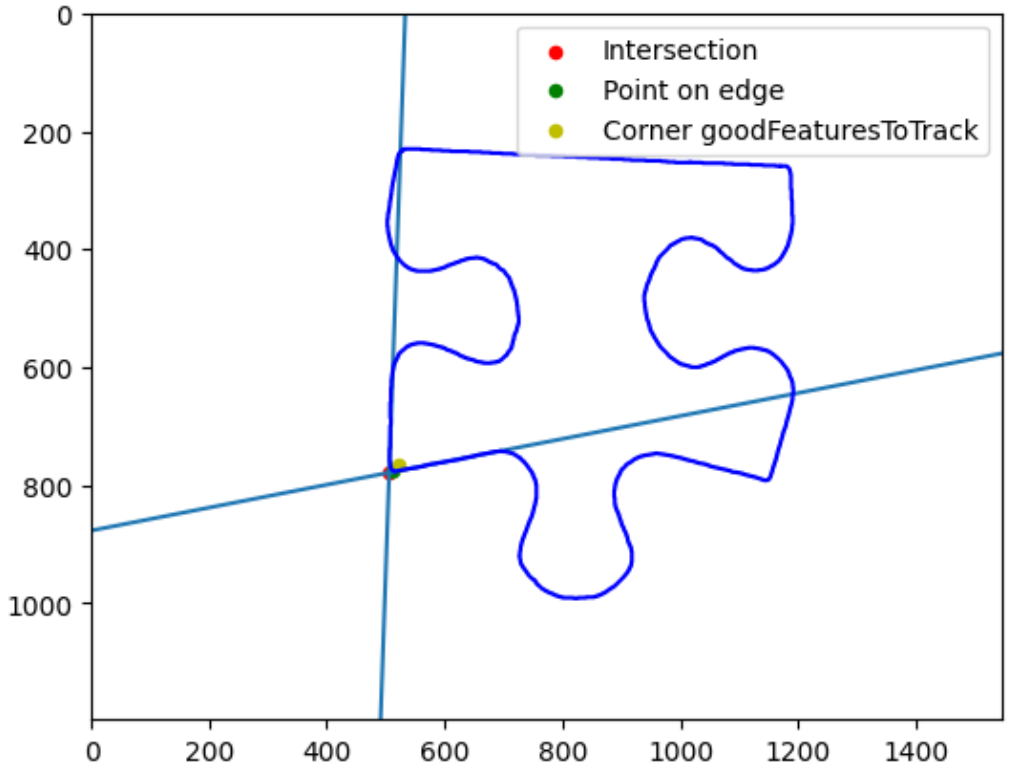
HOUGH TRANSFORMATION

- Hough Transformation erkennt präzise lokal die Kanten
- Regionen anhand der goodFeaturesToTrack Ecken



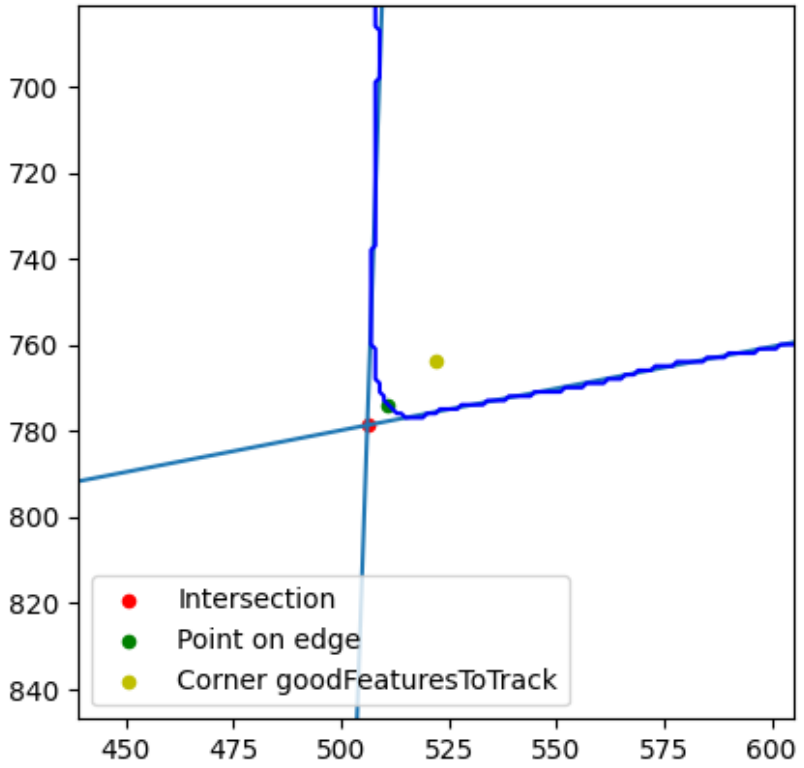
HOUGH TRANSFORMATION

- Erzeugt Geraden auf dem Kantenbild
- Die Schnittpunkte der Geraden werden für die finalen Ecken benutzt



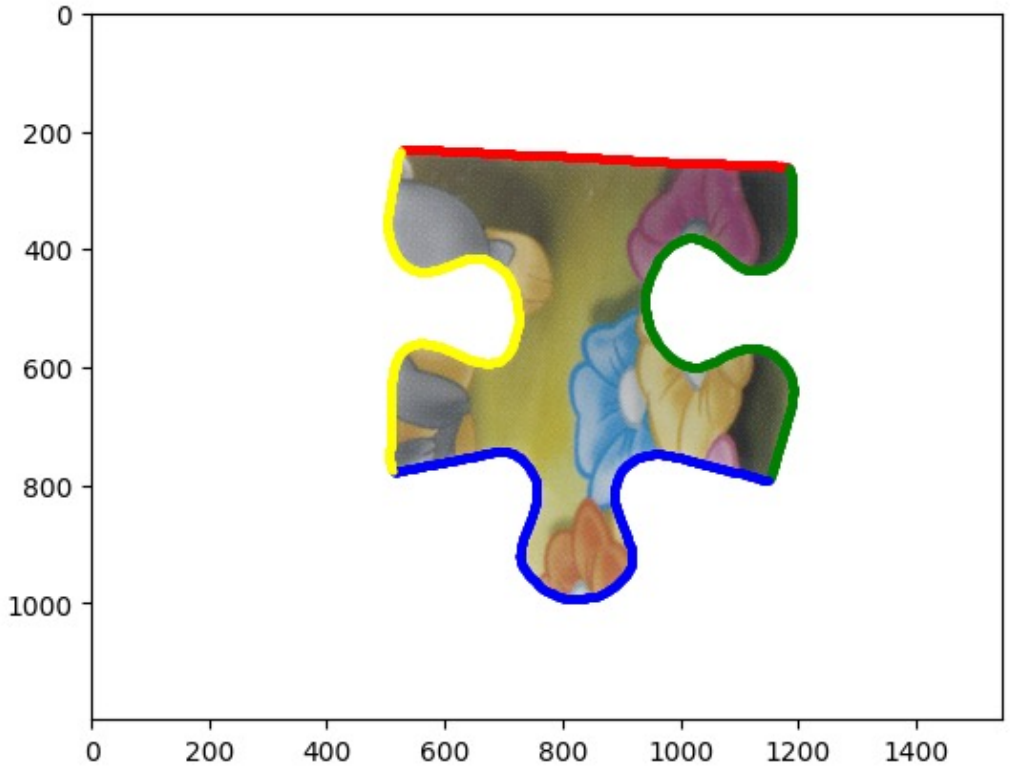
HOUGH TRANSFORMATION

- Erzeugt Geraden auf dem Kantenbild
- Die Schnittpunkte der Geraden werden für die finalen Ecken benutzt



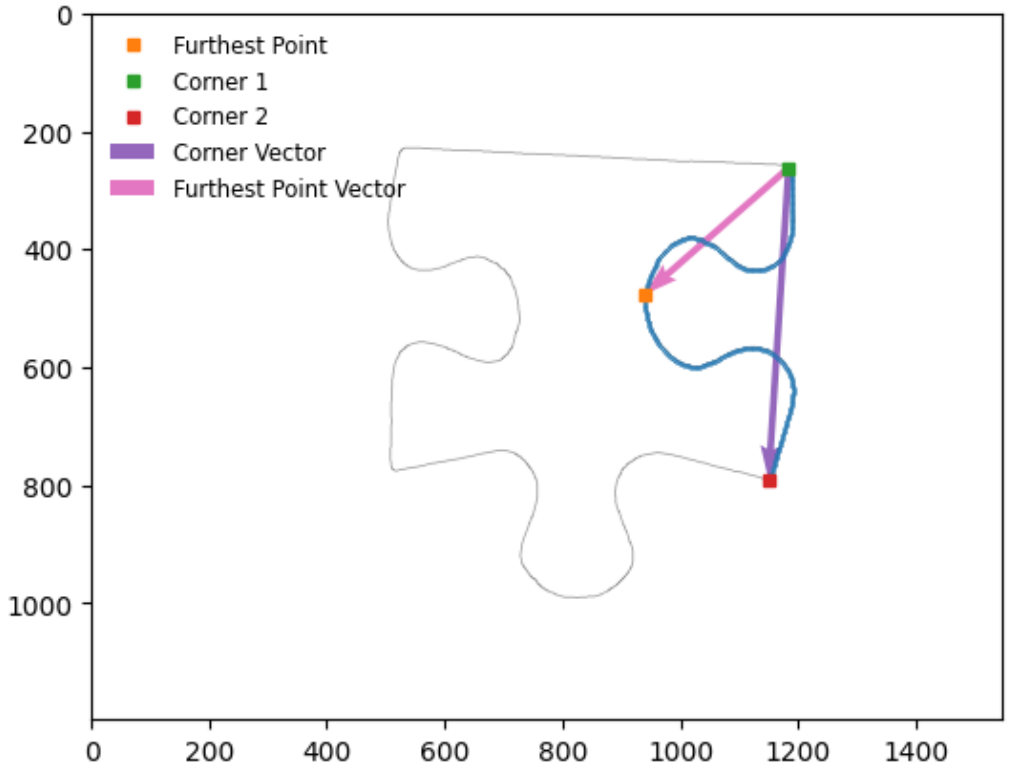
PUZZLE TEIL

- Kante wird an Ecken geteilt
- Ebenso wie Ecken, im Uhrzeigersinn definiert



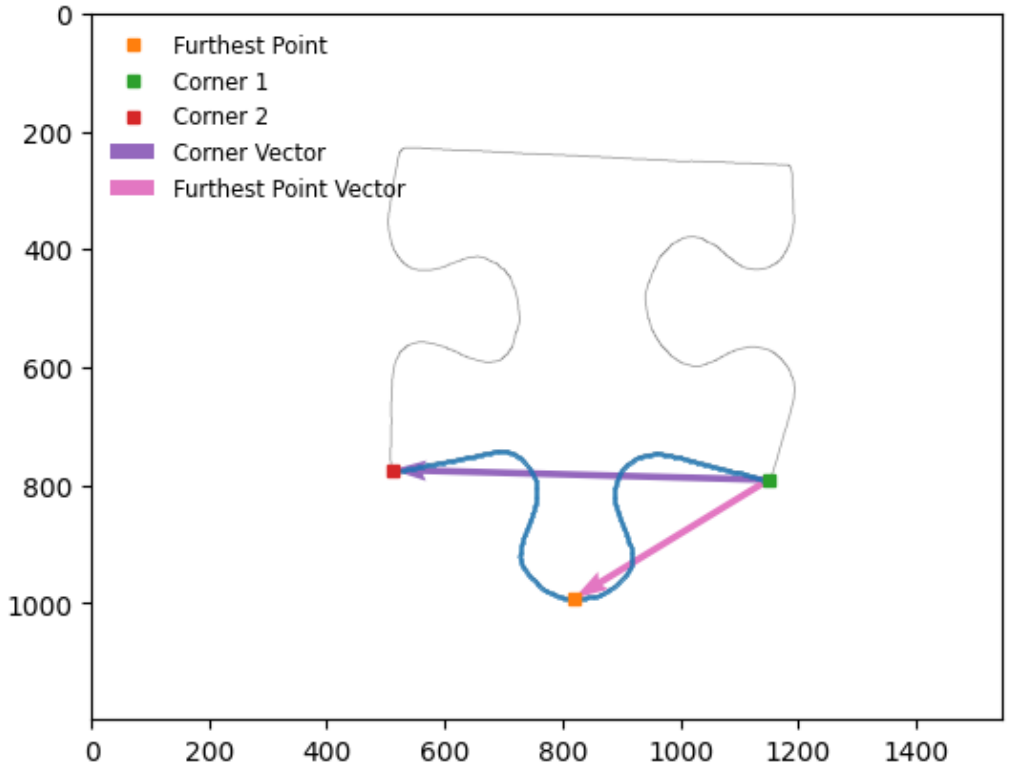
KANTEN TYP

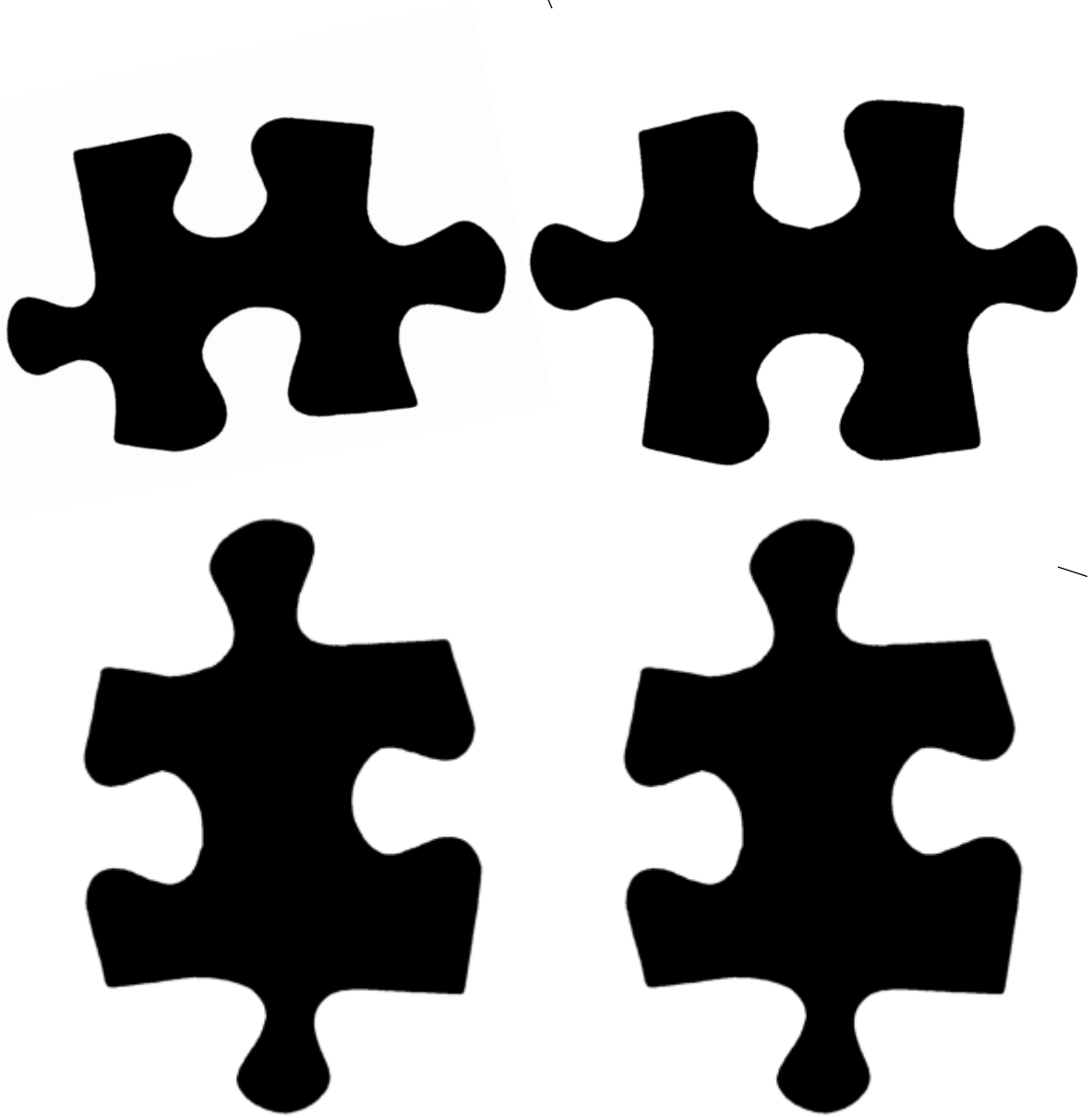
- Wir unterscheiden zwischen female, male und flat
- Kantentyp anhand von CCW Test



KANTEN TYP

- Wir unterscheiden zwischen female, male und flat
- Kantentyp anhand von CCW Test





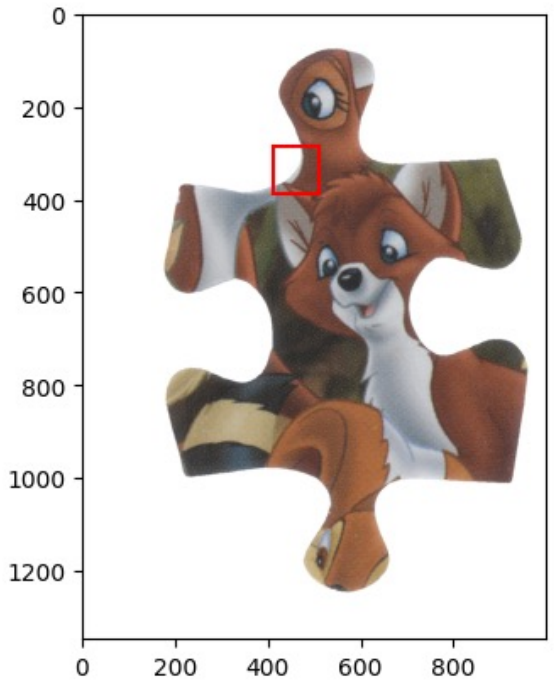
FARB MATCHING

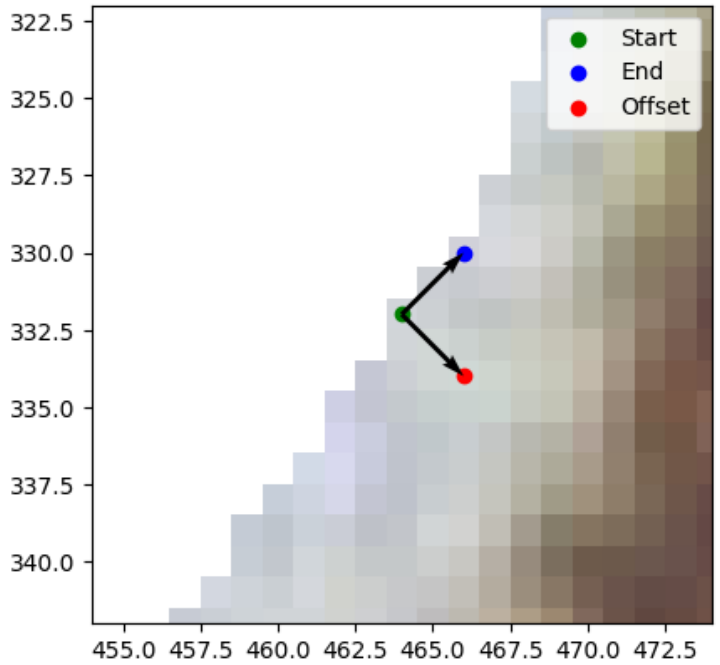


FARB MATCHING

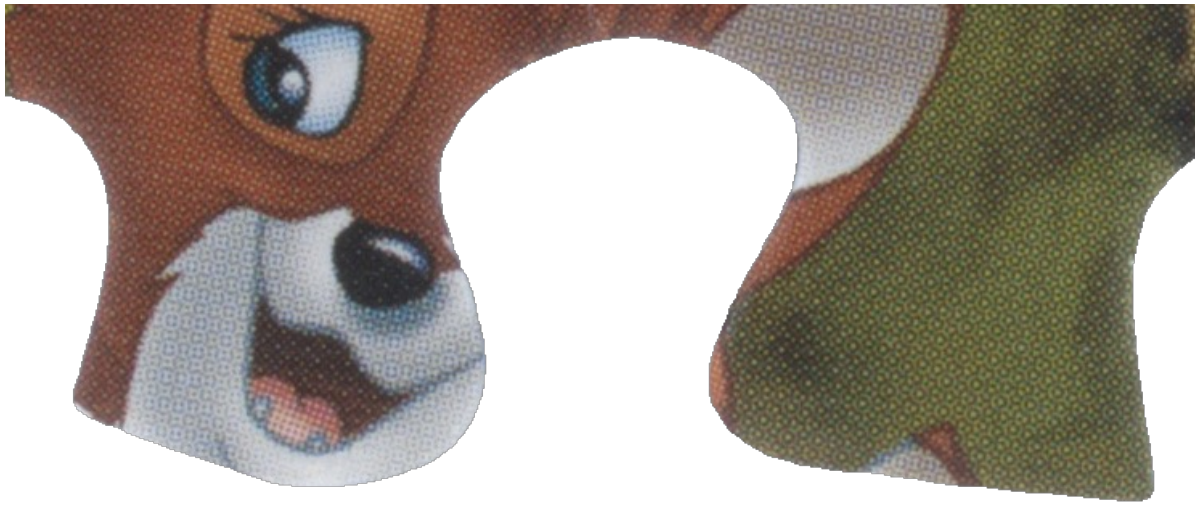
GEOMATCHER RGB

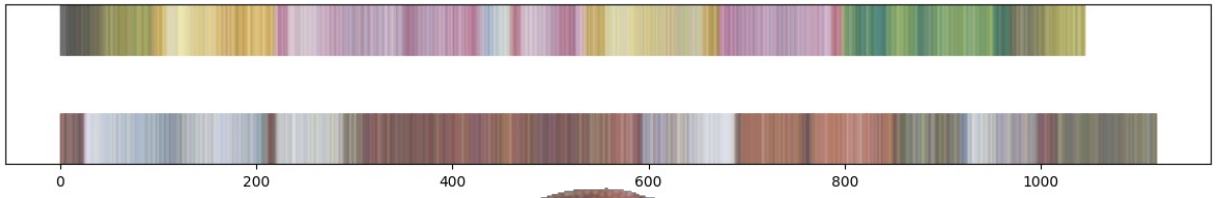
- Vorgehensweise wie beim Geomatcher aber statt Ortsinformationen (x,y) werden Farbwerte (r,g,b) extrahiert
- Summe der Unterschiede der Farbwerte ist das Gütekriterium zweier Kanten

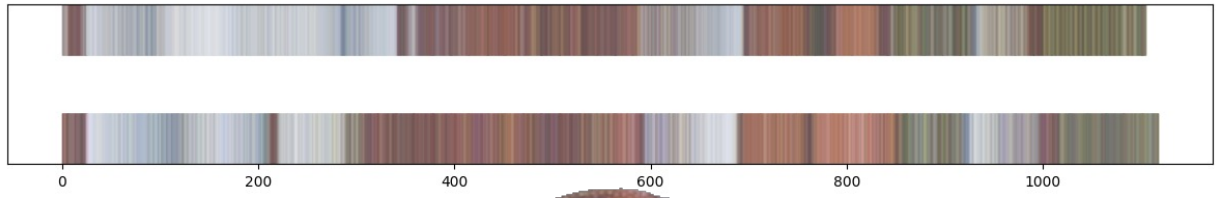




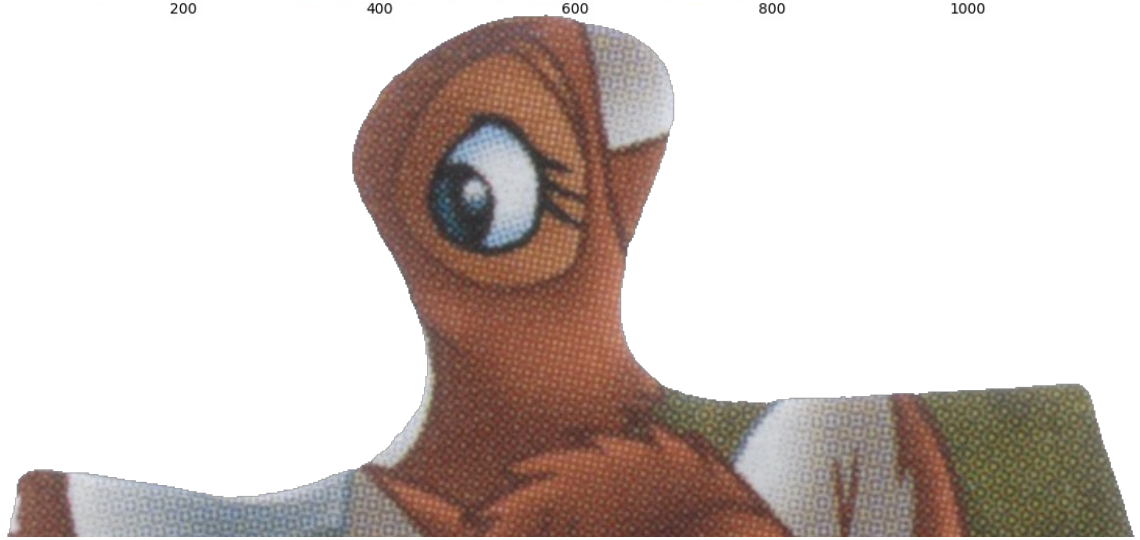


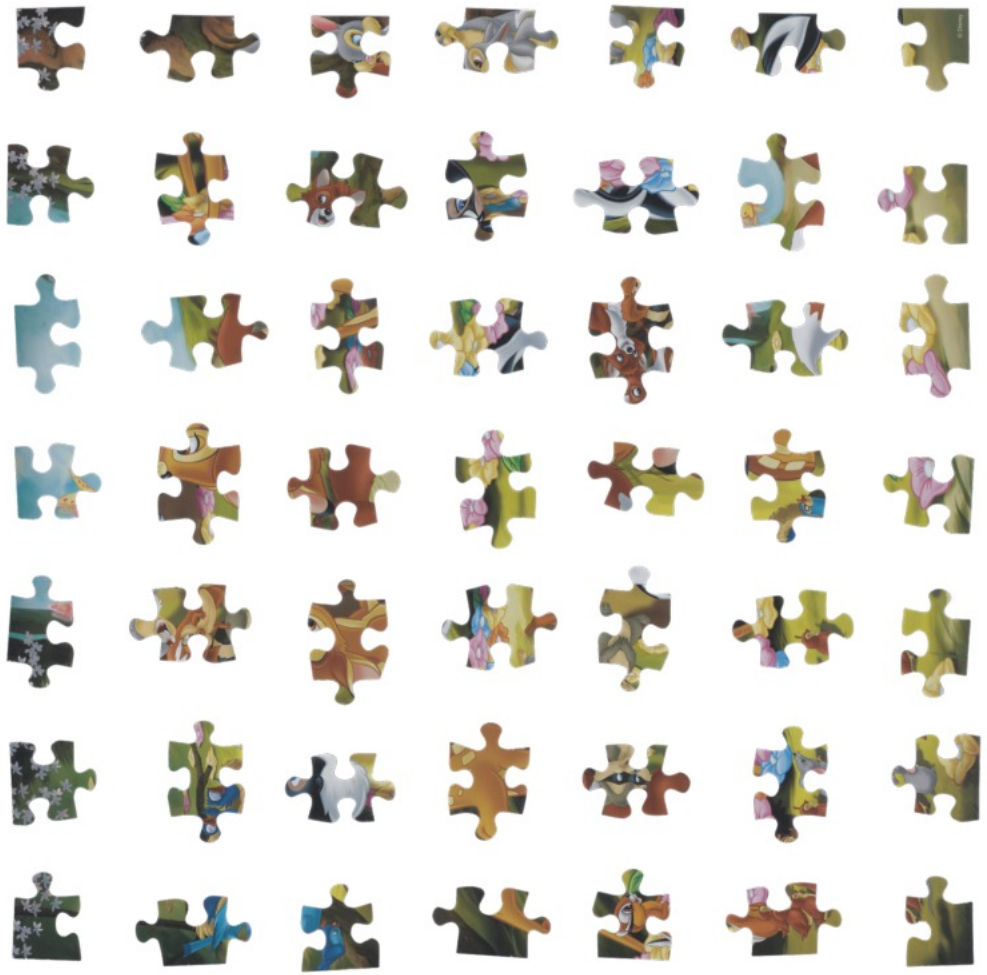



0
200
400
600
800
1000


0
200
400
600
800
1000







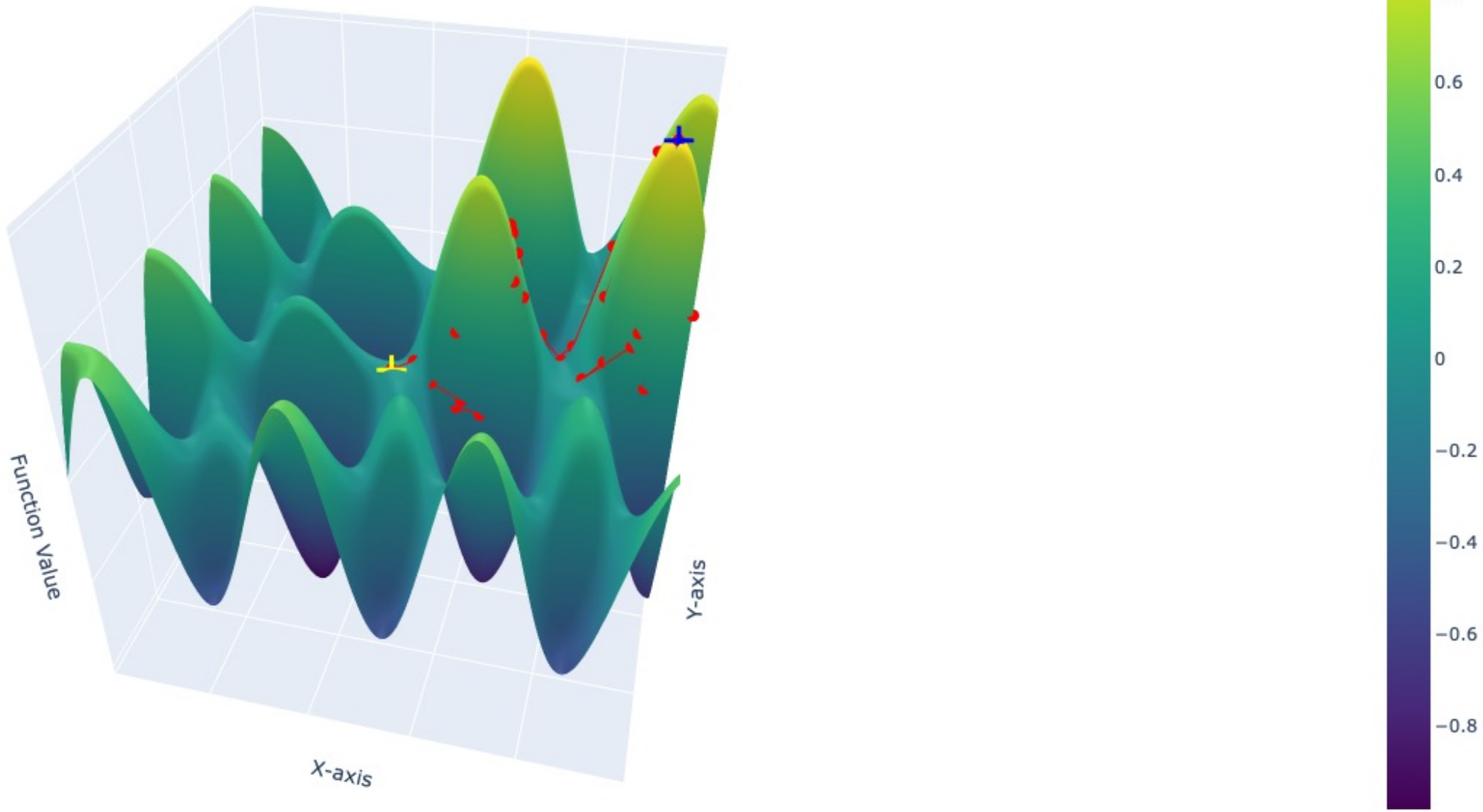
SIMULATED ANNEALING

PSEUDOCODE

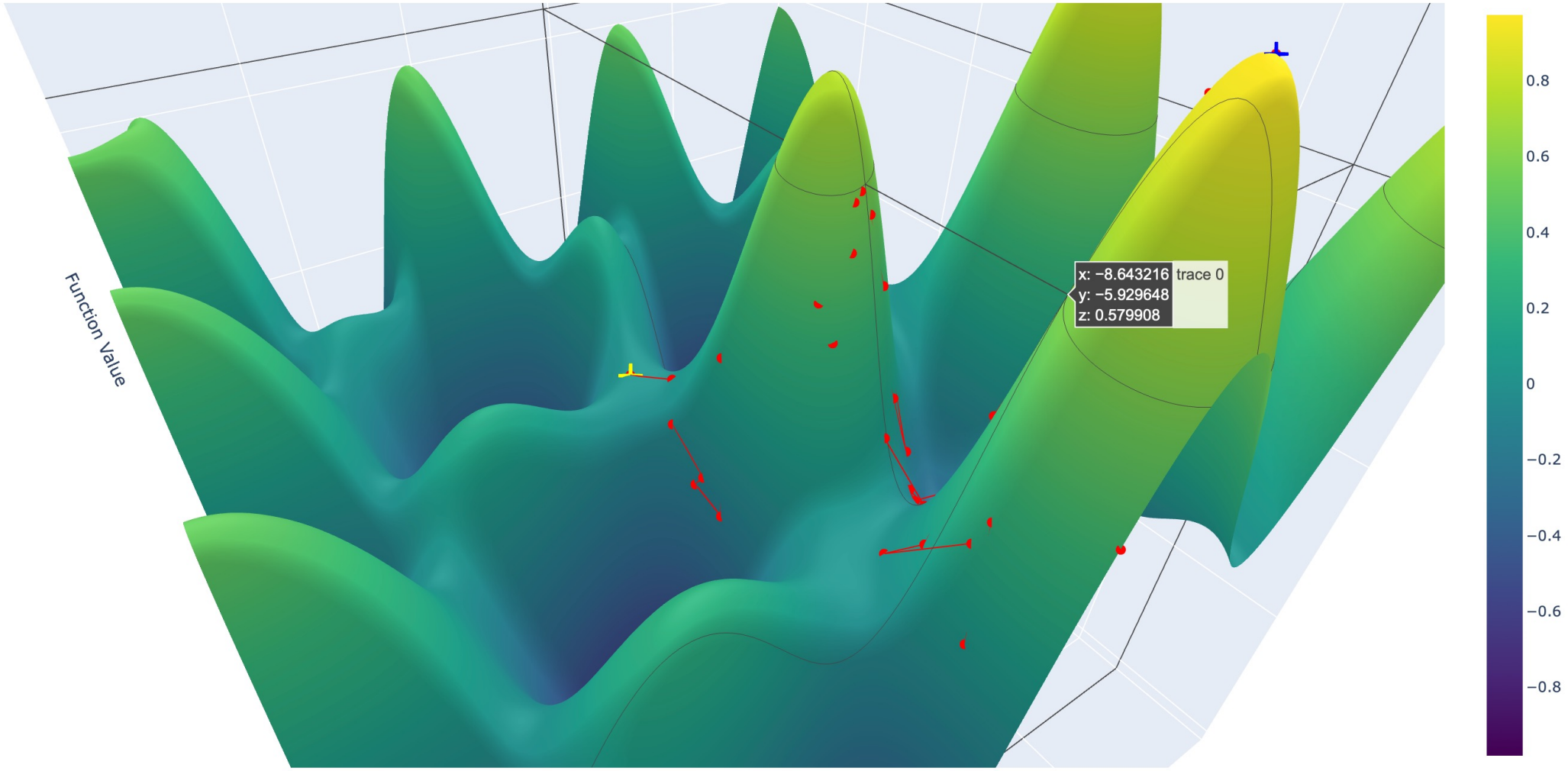
Algorithm 2 Simulated Annealing Algorithmus

Eingabe: Anfangszustand $\mathbf{s}_0$, maximale Iterationen $k_{\max}$, Anfangstemperatur T_0, Kühlrate α
Ausgabe: Finaler Zustand $\mathbf{s}$
$\mathbf{s} \leftarrow \mathbf{s}_0$
$T \leftarrow T_0$
for $k = 1$ bis $k_{\max}$ **do**
    $\mathbf{s}' \leftarrow \text{Bewegung}(\mathbf{s})$
    $\Delta E \leftarrow \text{Energie}(\mathbf{s}') - \text{Energie}(\mathbf{s})$
    if $\Delta E < 0$ **oder** $\text{zufall}(\Delta E, T)$ **then**
        $\mathbf{s} \leftarrow \mathbf{s}'$
    end if
    $T \leftarrow \text{AnpassungVonT}(T, \alpha, k)$
end for
return s

BEISPIEL

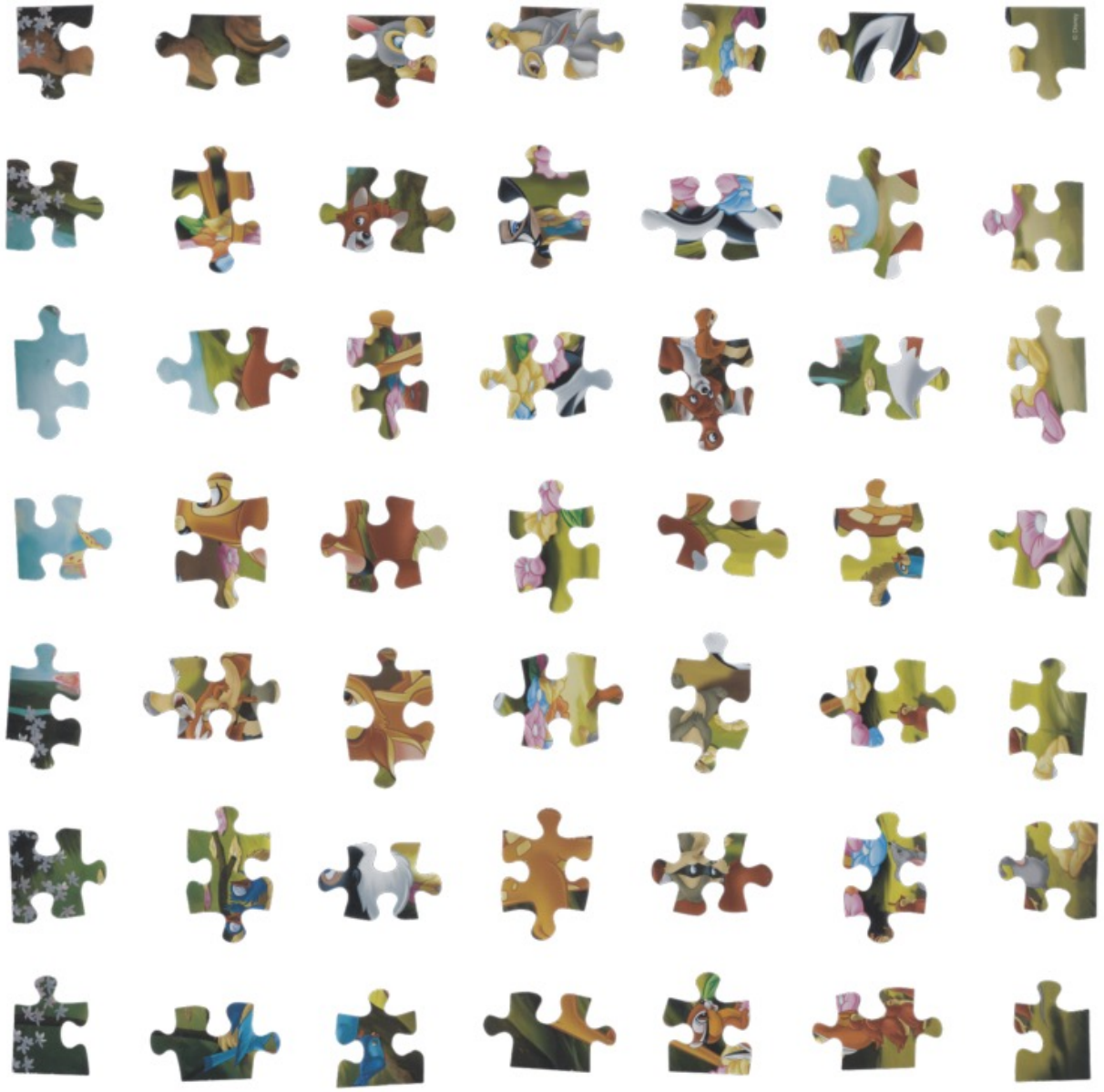


BEISPIEL


Function Value
x: −8.643216 trace 0
y: −5.929648
z: 0.579908
0.8
0.6
0.4
0.2
0
−0.2
−0.4
−0.6
−0.8

ERGEBNISSE

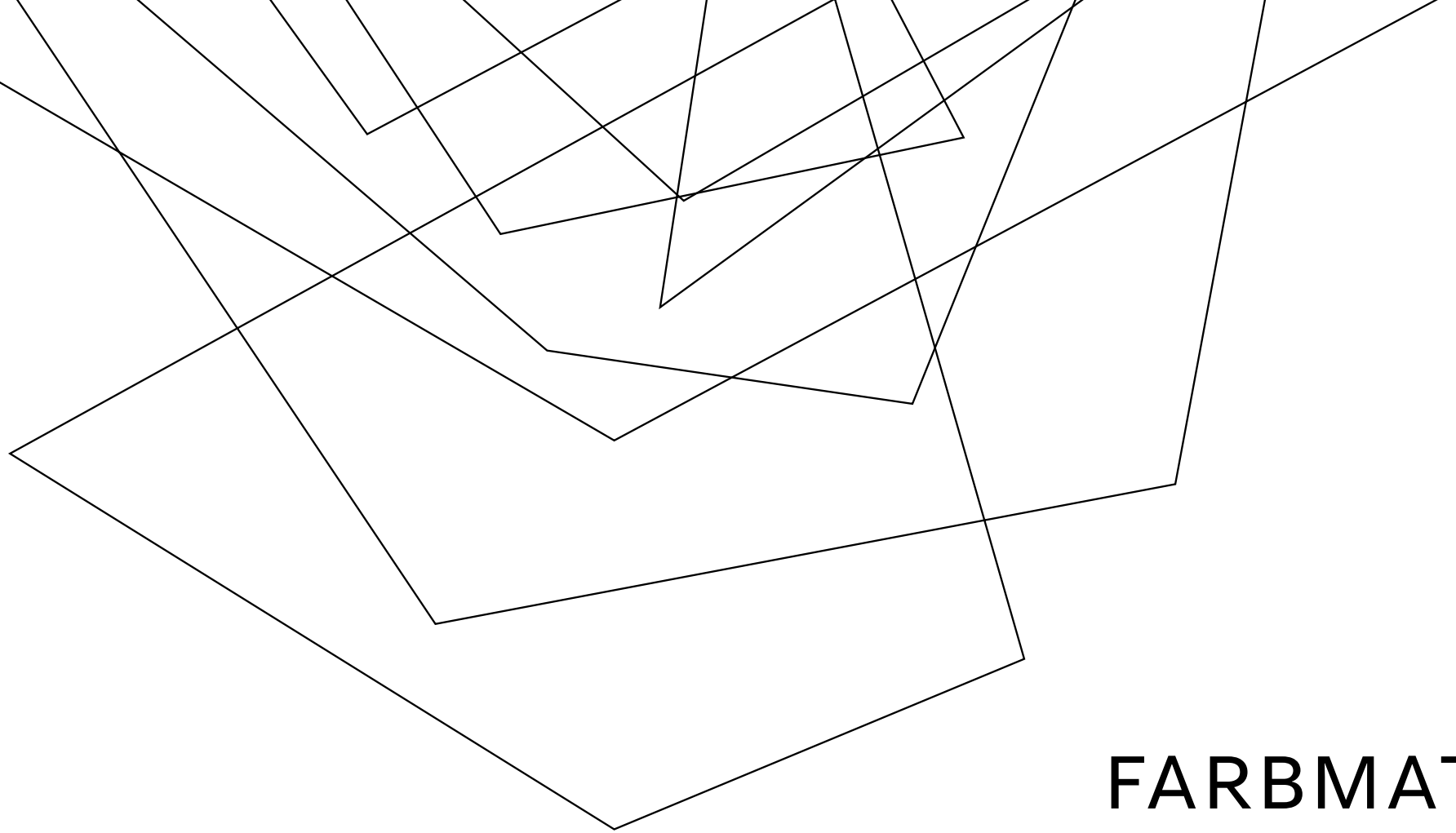
- Annealing nicht erfolgreich
- Zu viele Zustände
- Nicht effizient genug



VIELEN DANK FÜR IHRE AUFMERKSAMKEIT!

Simon Hampp

@simon-hampp (gitlab)



FARBMATCHING

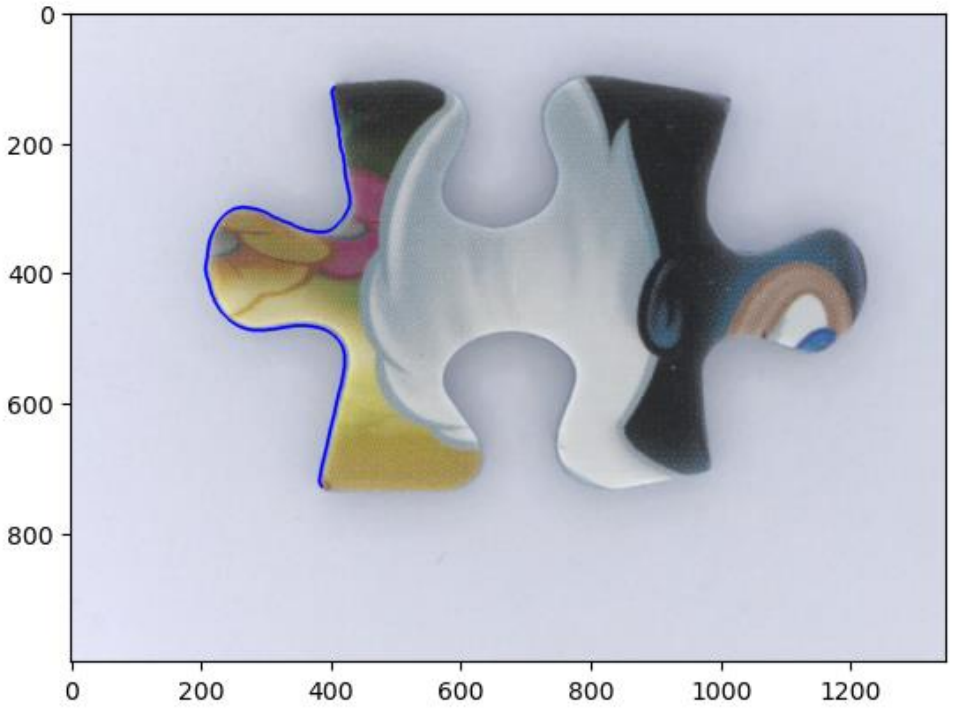
ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

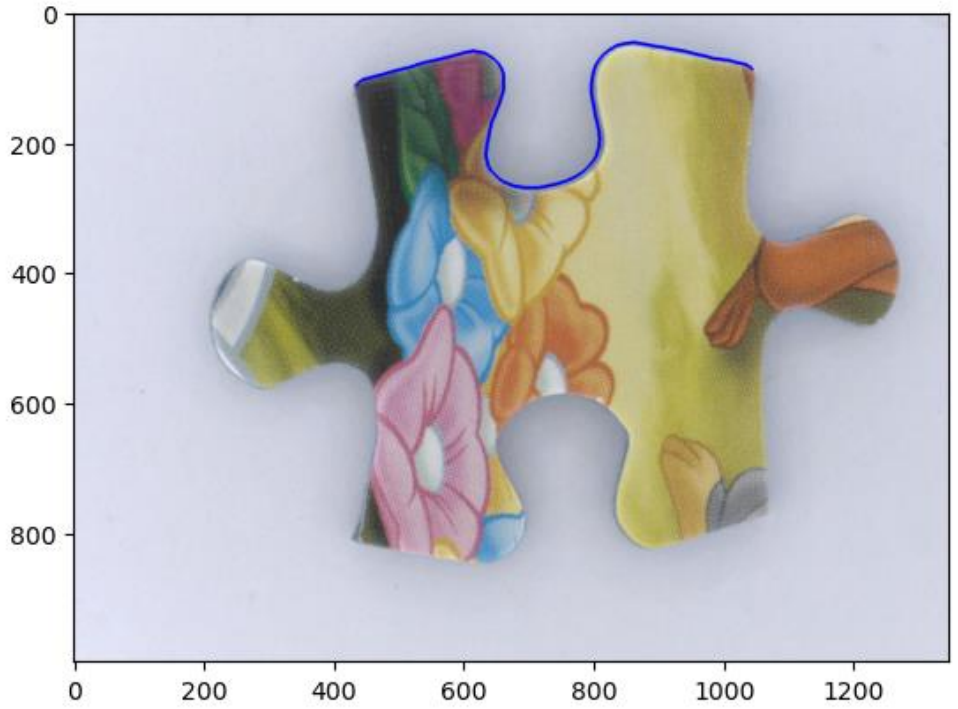
GLIEDERUNG

- Idee / Motivation
- Algorithmus
- Probleme und Lösungen

IDEE / MOTIVATION

IDEE / MOTIVATION





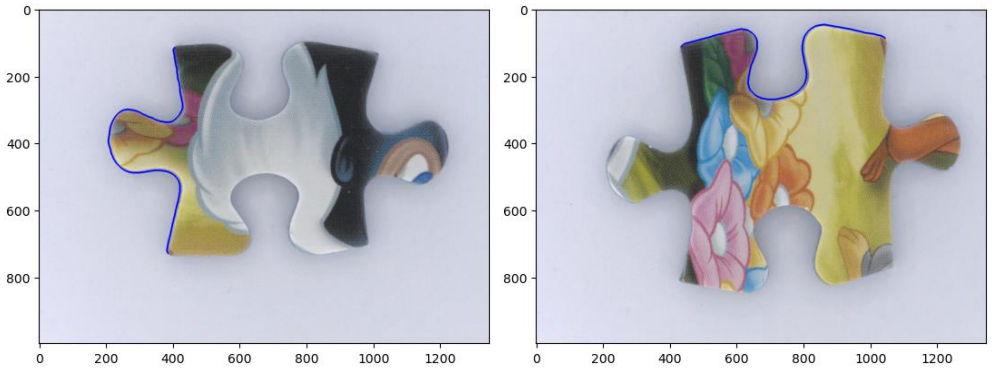
IDEE / MOTIVATION

- Matching der Puzzleteil-Kanten anhand der Bildinformationen (Farbwerte)
- Annahme: Farbwerte zweier aneinanderliegender Kanten sind ähnlich

ALGORITHMUS

ALGORITHMUS

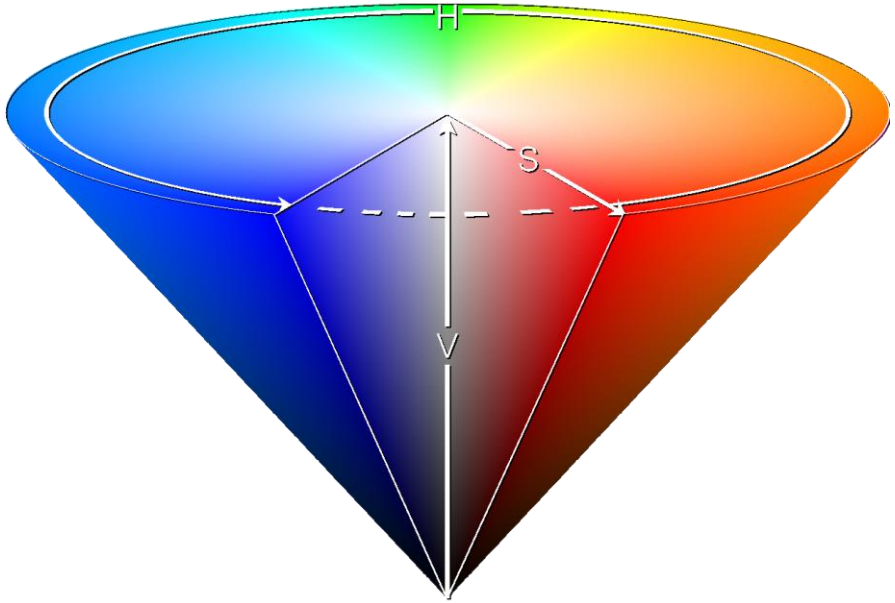
- Eingabe:
  - Zwei Puzzleteil-Kanten (Listen Pixel-Koordinaten)
  - PNG-Bilder der Puzzleteile
- Ausgabe:
  - Ähnlichkeit der Kanten zwischen [0, 1]



similarity: 0.65

ALGORITHMUS

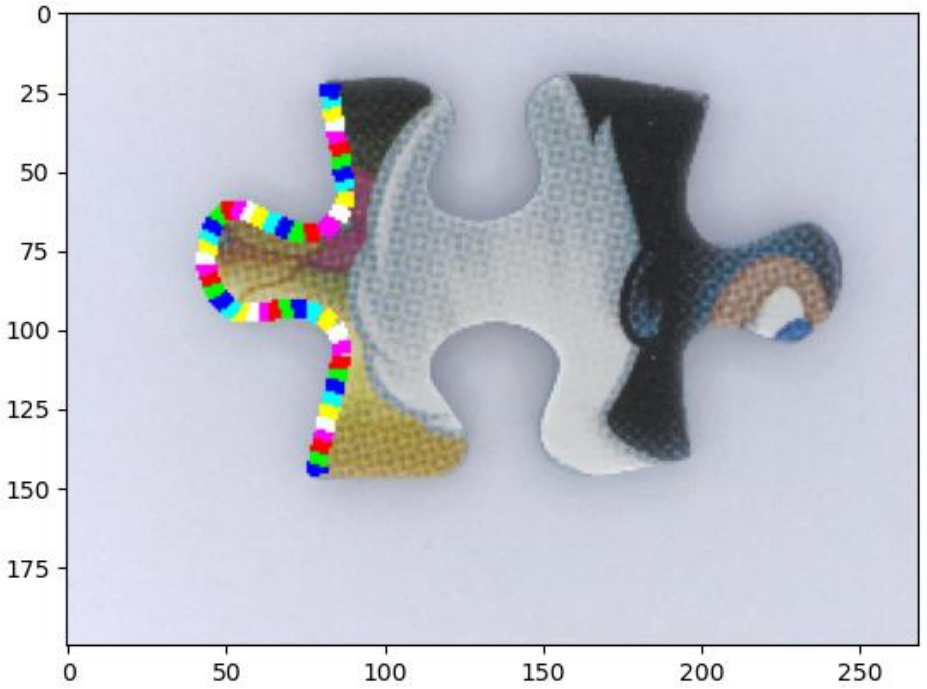
1. Bilder der Puzzleteile herunterskalieren (Faktor 0.2)
2. Umwandlung: **BGR** (OpenCV Standard) → **HSV** (Hue, Saturation, Value)



ALGORITHMUS

3. Aufteilung jeder Kante in 50 gleich große „Patches“

   Größe: ca. 5x5 Pixel



ALGORITHMUS

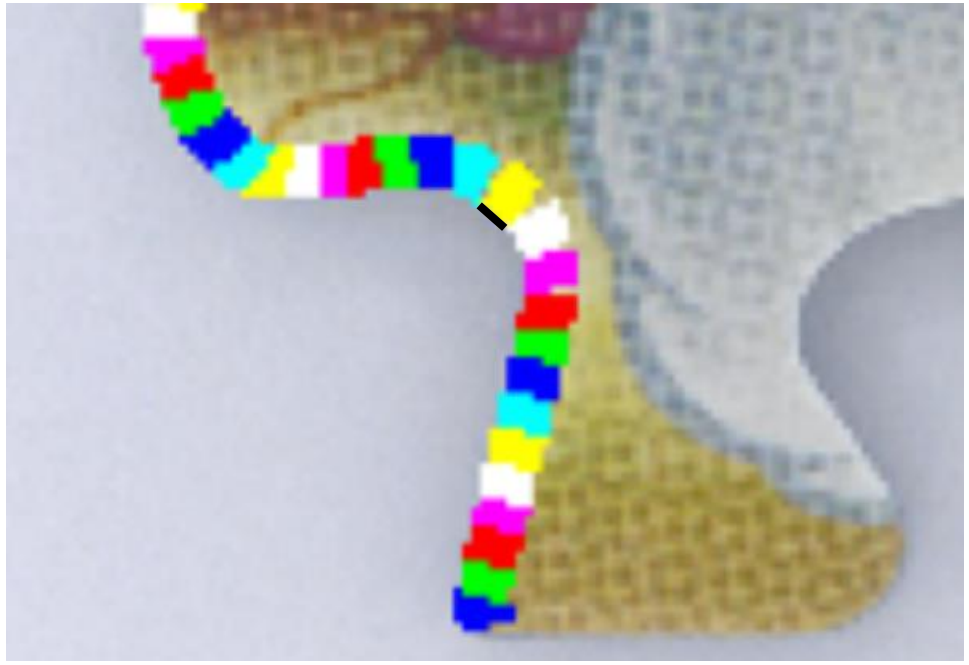
Berechnung der Patches



ALGORITHMUS

Berechnung der Patches

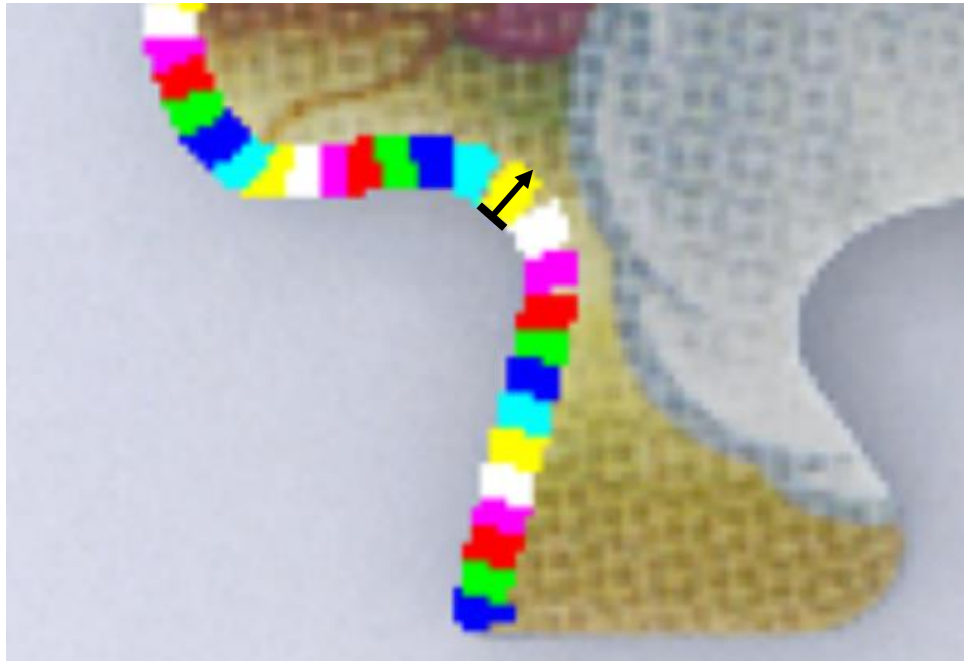
- Abstand zwischen Start- und Endkoordinate bestimmen



ALGORITHMUS

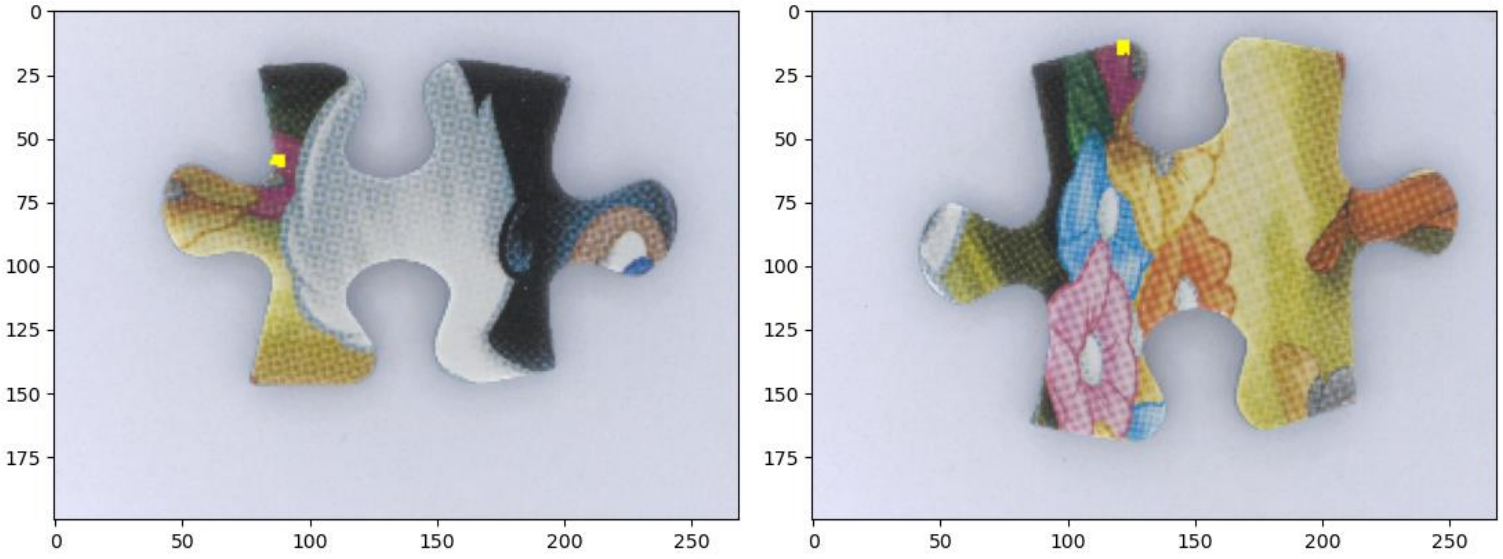
Berechnung der Patches

- Abstand zwischen Start- und Endkoordinate bestimmen
- Normale berechnen
- `cv.fillPoly()` um alle beinhalteten Pixel auszuwählen

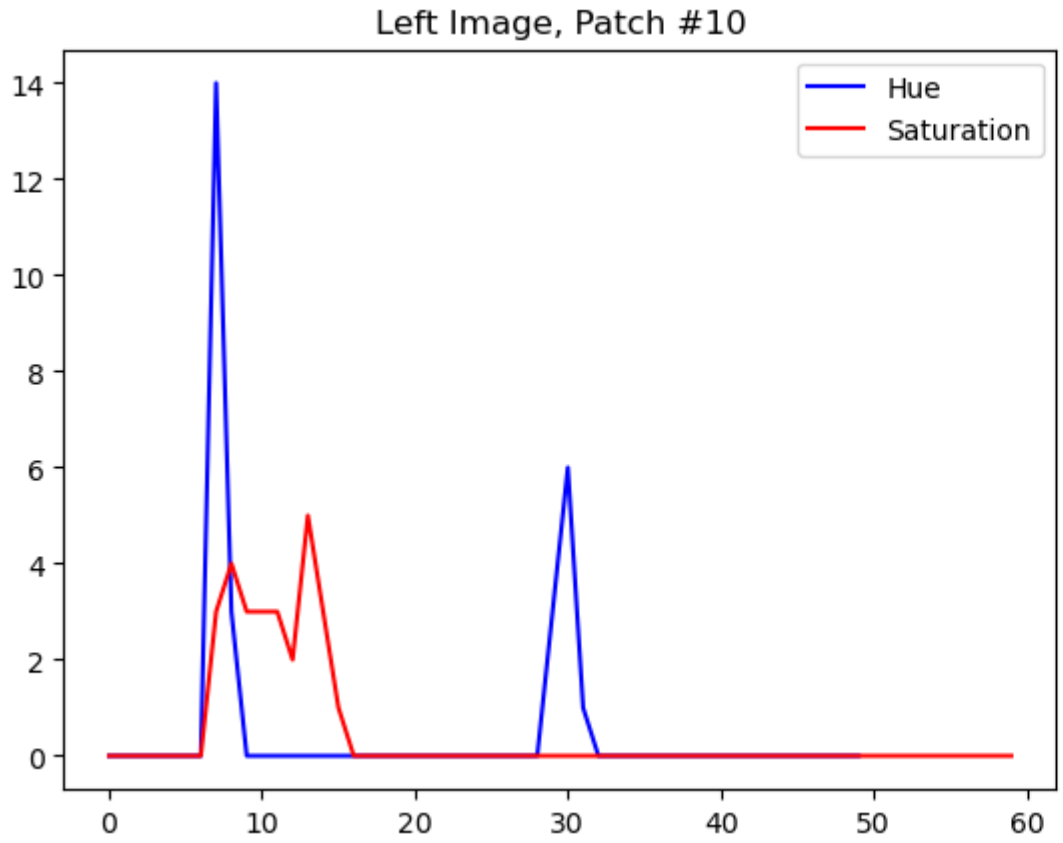


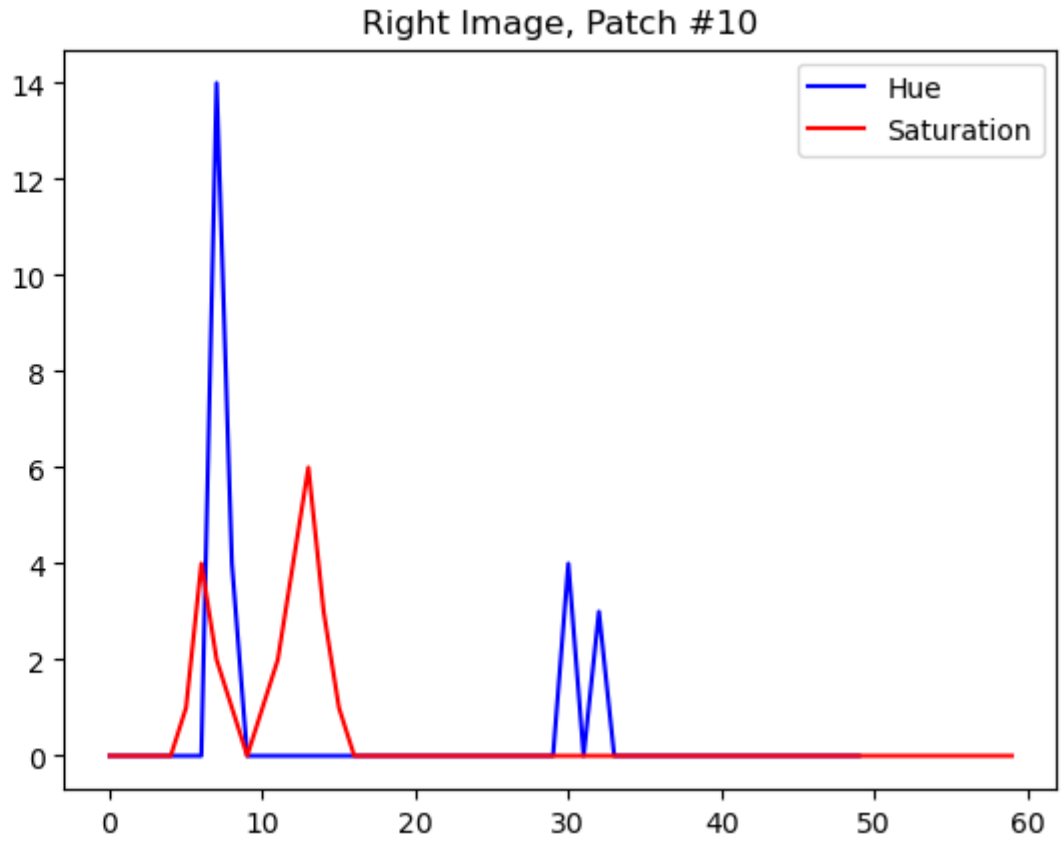
ALGORITHMUS

4. Bestimmung eines Histogramms über Hue und Saturation für jeden Patch



ALGORITHMUS



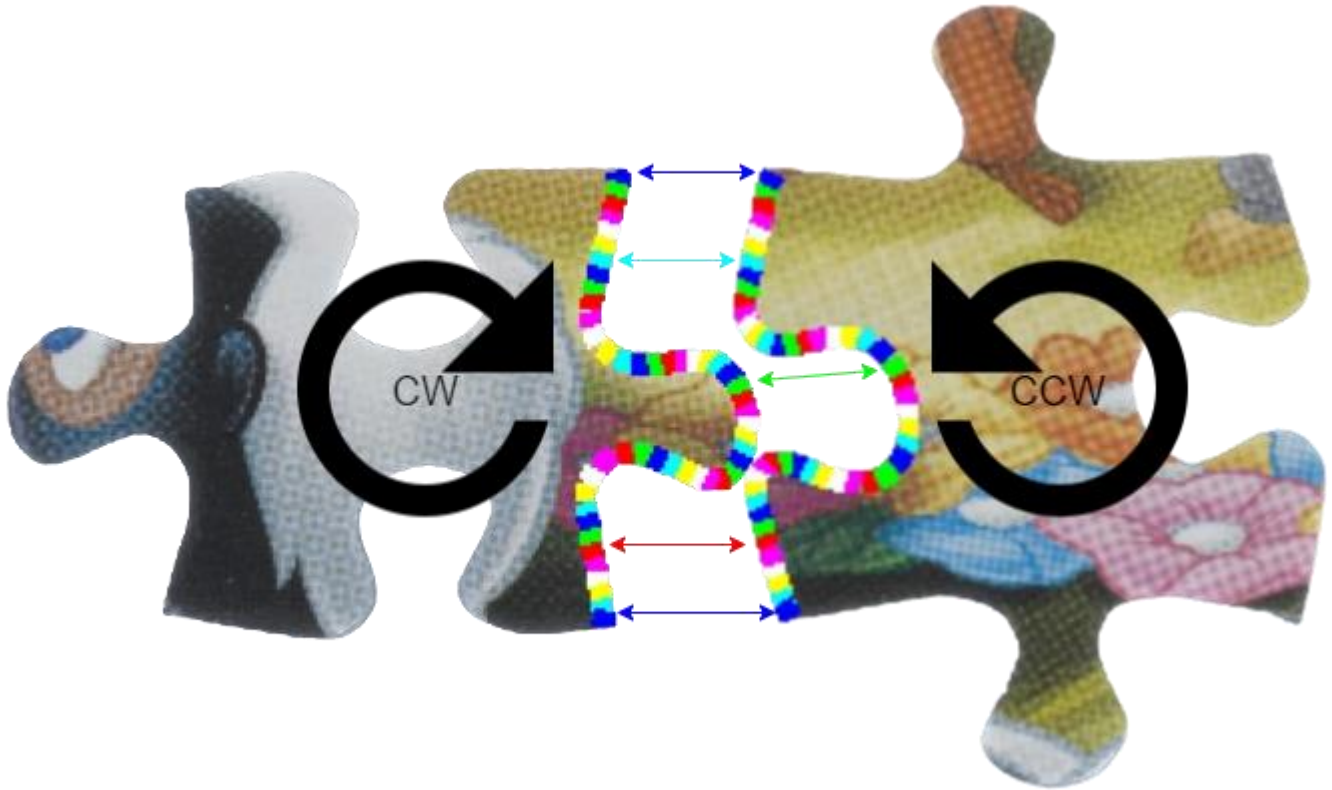


ALGORITHMUS

5. Histogramm-Vergleich der jeweils gegenüberliegenden Patches

   Funktion:
   `cv.compareHist()`

   jeweils eine Kante muss in entgegengesetzter Richtung iteriert warden (Counter-Clockwise)



ALGORITHMUS

6. Mittelwert über alle 50 Korrelations-Werte entlang der Kanten bestimmen
7. **Ergebnis:** Ähnlichkeitswert zwischen [0, 1] der beiden Kanten anhand der Farbwerte

Correlation

$$d(H_1, H_2) = \frac{\sum_I (H_1(I) - \bar{H}_1)(H_2(I) - \bar{H}_2)}{\sqrt{\sum_I (H_1(I) - \bar{H}_1)^2 \sum_I (H_2(I) - \bar{H}_2)^2}}$$

where

$$\bar{H}_k = \frac{1}{N} \sum_J H_k(J)$$

and N is a total number of histogram bins.

PROBLEME UND LÖSUNGEN

PROBLEME UND LÖSUNGEN

- Schlechte Performance bei der Berechnung der Patches und Histogramme

  → Skalierung auf 20% der Bildgröße

- Schlechte Performance beim Vergleich der Histogramme

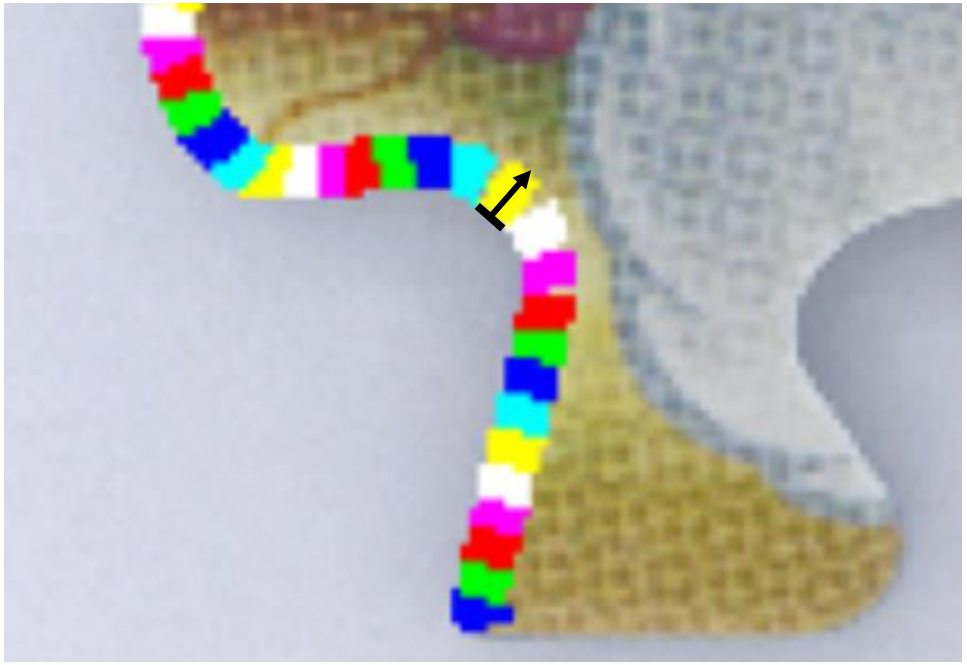
  → Reduzierung der Bins auf 50 (Hue), bzw. 60 (Saturation),
  anstelle von jeweils 255

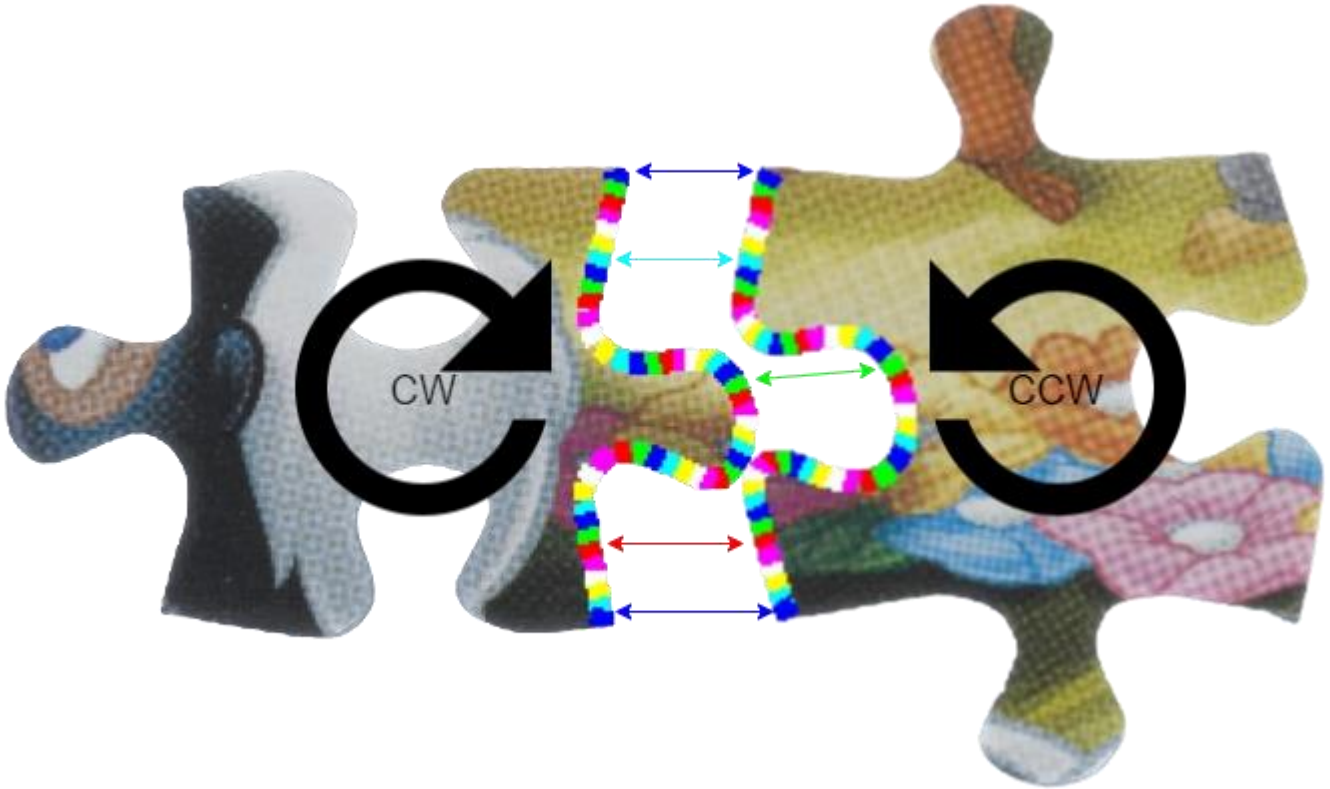
PROBLEME UND LÖSUNGEN

- Patch-Berechnung:
  Sicherstellen, dass die Normalen in die richtige Richtung zeigen
- Beim Vergleich der Patches entlang einer Kante nicht “am falschen Ende” anfangen

  → garantierte Clockwise (CW) Anordnung aller Kanten-Koordinaten

PROBLEME UND LÖSUNGEN





VIELEN DANK FÜR IHRE AUFMERKSAMKEIT!

Maximilian Weigle

@mweigle (gitlab)

Quellen:

- **HSV Cone**: en:File:HSV cone.png by (3ucky(3all; then transfered to Commons by Moongateclimber., CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=943857



XOR MATCHER

ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

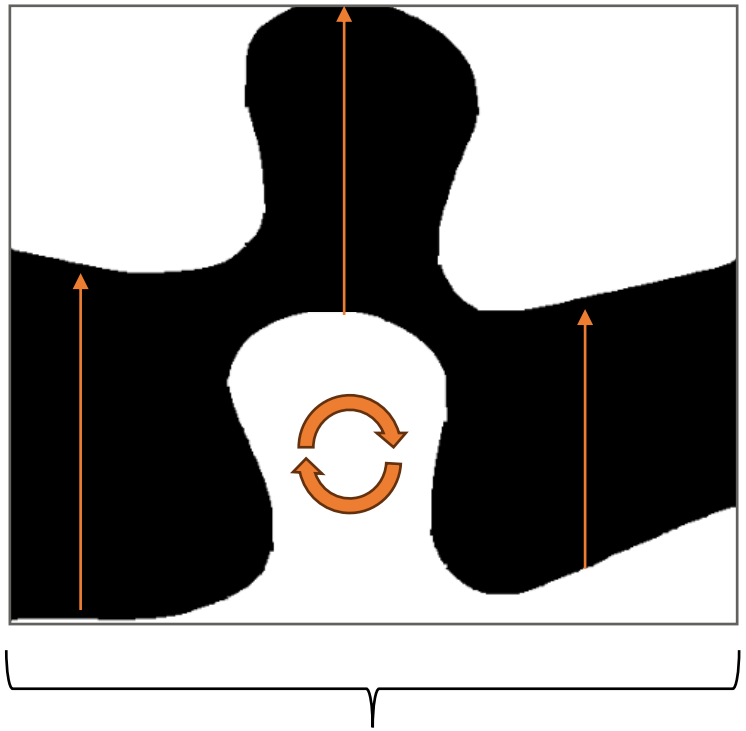
GLIEDERUNG

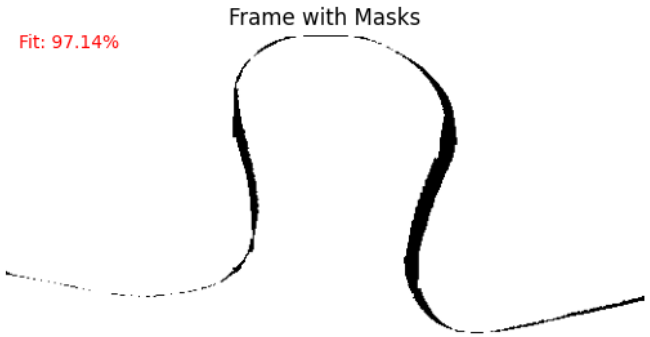
- Idee
- Data Preperation
- Optimierer
- Integration
- Ausblick

XOR MATCHER IDEE

IDEE







DATA PREPERATION

NORMALISIERUNG

Rotation + Verschiebung

- **Ausrichtung Erkennen (Varianz)**

$$\mathrm{Var}(x) < \mathrm{Var}(y) \implies \text{vertikale Kante}$$

$$\mathrm{Var}(x) > \mathrm{Var}(y) \implies \text{horizontale Kante}$$

- **Falls vertikale Kante**

$$B = S_x A^T$$

  - Achsenvertauschung
  - Spiegelung an der x-Achse

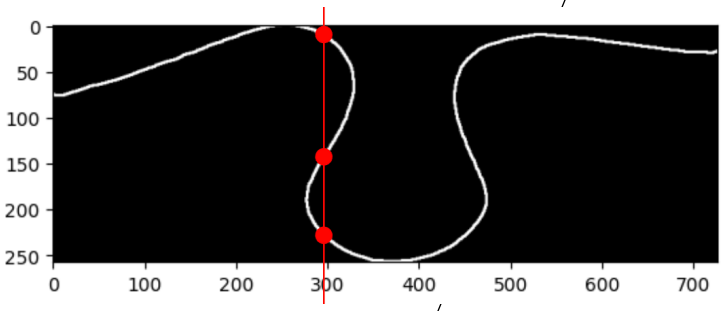
$$S_x = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

- **Verschiebung an die obere linke Bildkante**

BINÄRMASKENBILDUNG

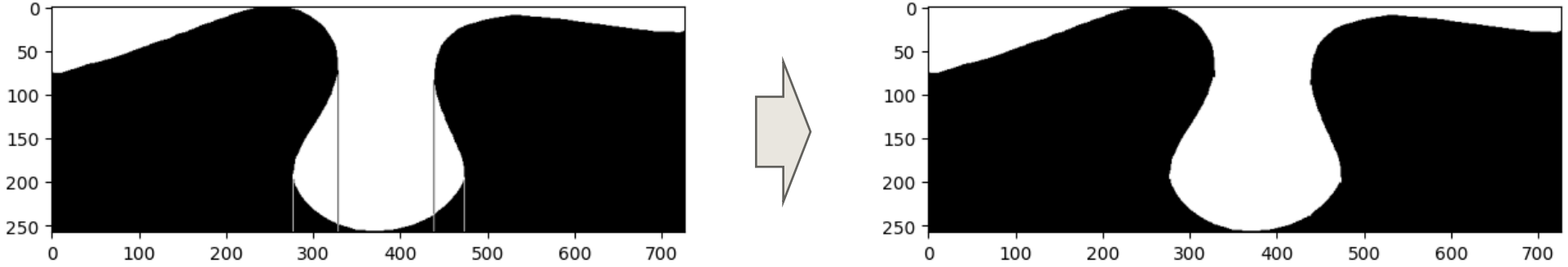
Algorithm 2 Maskenfüllungsalgorithmus

```
1:  Input: BildMaske, KantenMapping
2:  Output: GefüllteMaske
3:  Initialisiere GefüllteMaske mit den Abmessungen von BildMaske
4:  Setze alle Pixel von GefüllteMaske auf 0 (Schwarz)
5:  for Jede Spalte y von oben nach unten do
6:      farbe ← 1                                   ▷ Startfarbe Weiß
7:      for Jedes Pixel x in Spalte y do
8:          if x ist in KantenMapping für y then
9:              farbe ← 1 − farbe                   ▷ Wechselt zwischen 0 und 1
10:         end if
11:         Setze Pixel (x, y) in GefüllteMaske auf farbe
12:     end for
13: end for
14: return GefüllteMaske
```

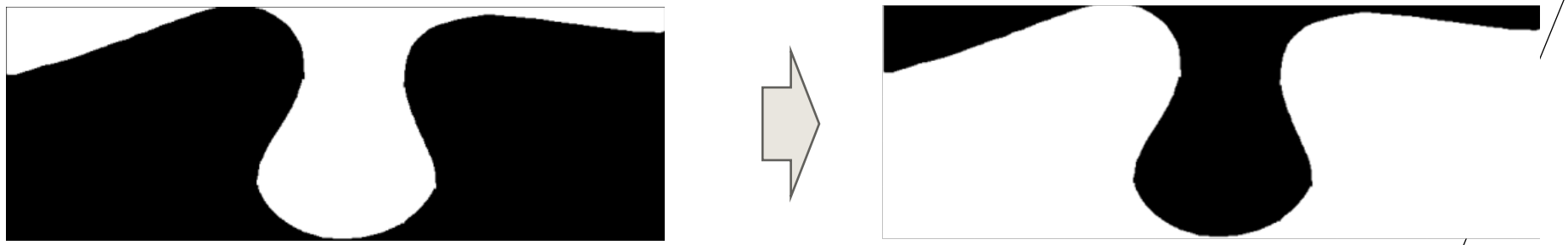


BINÄRMASKENBILDUNG

- Binärmaske säubern

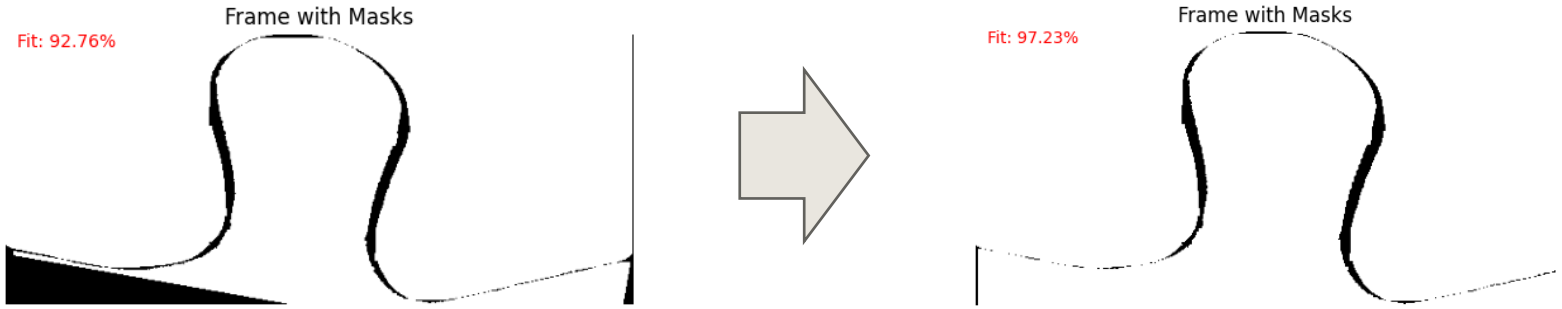


- Edge-Type anpassen
  - Ursprünglicher EdgeType: EdgeType.Female



BINÄRMASKENBILDUNG

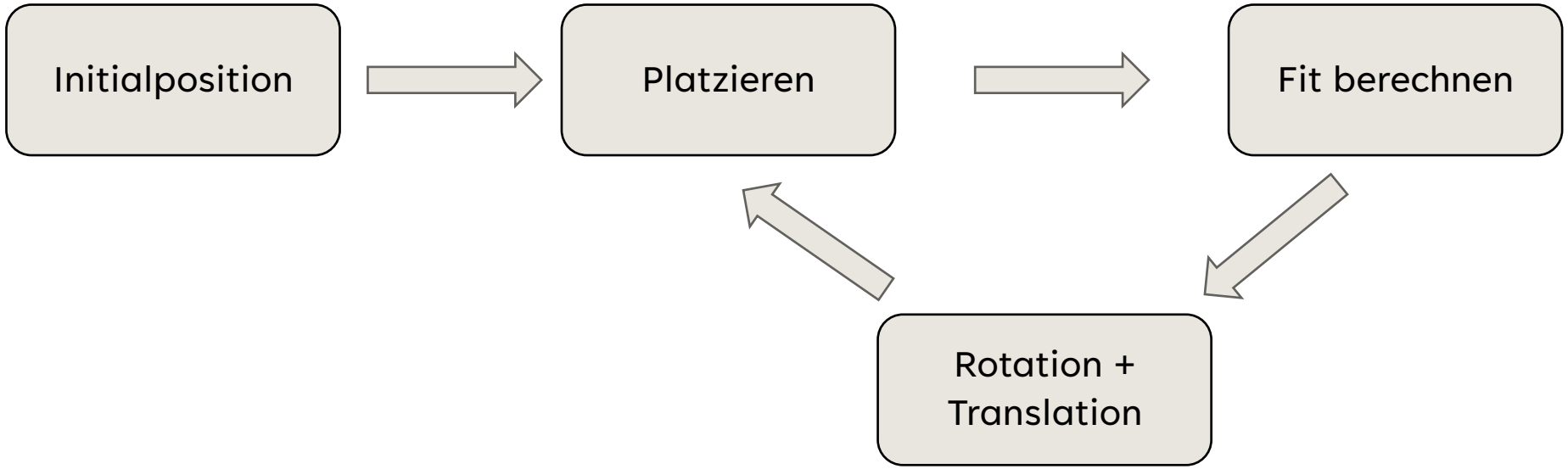
- Padding Hinzufügen



- Rotation von unterer Maske wird nicht bestraft

OPTIMIERUNG

OPTIMIERUNGS-ZYKLUS



INTEGRATION

INTEGRATION

- **Nutzt Matcher Interface**
- **Match Matrix wird gespeichert (npy – Format)**
- **Laufzeit bei 7x7 ca. 1,5 h**

AUSBLICK

AUSBLICK

- **Anpassung der Score-berechnung**
  - Reines XOR scheint Fit schlecht zu repräsentieren
  - wenige große Überlappungen > viele kleine Überlappungen
  - Überlappungen schlechter als Leerräume
  - Stärkere Bestrafung bei unterschiedlicherer Kantenlängen
- **Kombination mit Farb-Matcher**
- **Performance Optimierung**

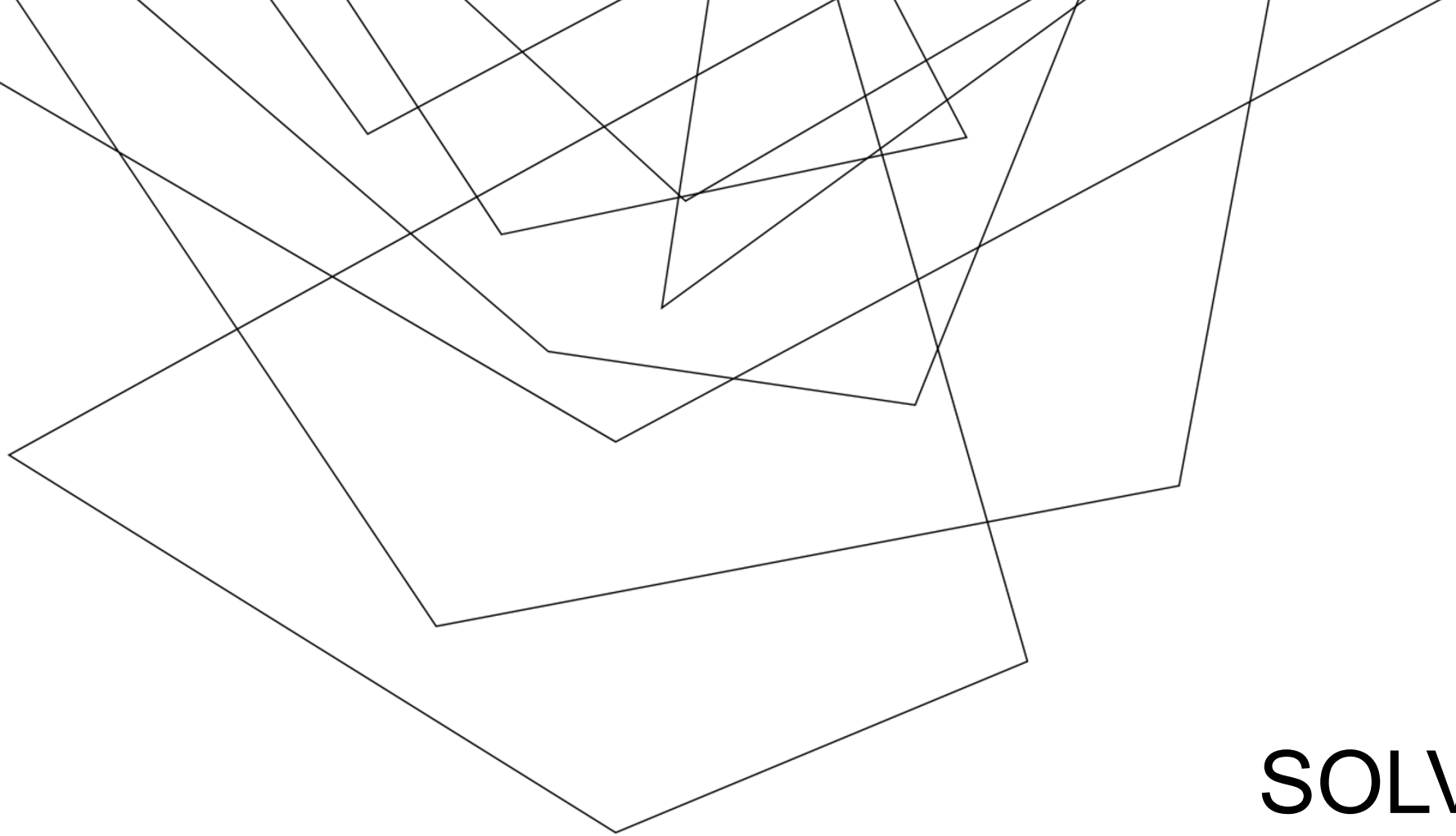
VIELEN DANK FÜR IHRE AUFMERKSAMKEIT!

Markus Ertl

@hm-mertl (gitlab)

Quellen:

- https://unsplash.com/de/fotos/stapel-von-puzzleteilen-3y1zF4hIPCg



SOLVER

ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

GLIEDERUNG

- Dimensionsbestimmung
- Mock-Puzzle
- Evaluierung



Dimensionsbestimmung

Dimensionsbestimmung: Warum?

- Bei perfektem Solver:

  beste Lösung = richtige Lösung ⇒ nicht nötig

- Aber:

  Weniger Möglichkeiten ⇒ schnellere Lösungsfindung

Dimensionsbestimmung: Wie?

- Lineares Gleichungssystem:

$$p = r \cdot c$$
$$k = 2(c + r) - 4$$

- Lösung:

$$c = \frac{2 + \frac{1}{2}k + \sqrt{(4 + 2k + \frac{1}{4}k^2 - 4p)}}{2}$$
$$r = \frac{p}{c}$$



Mock-Puzzle

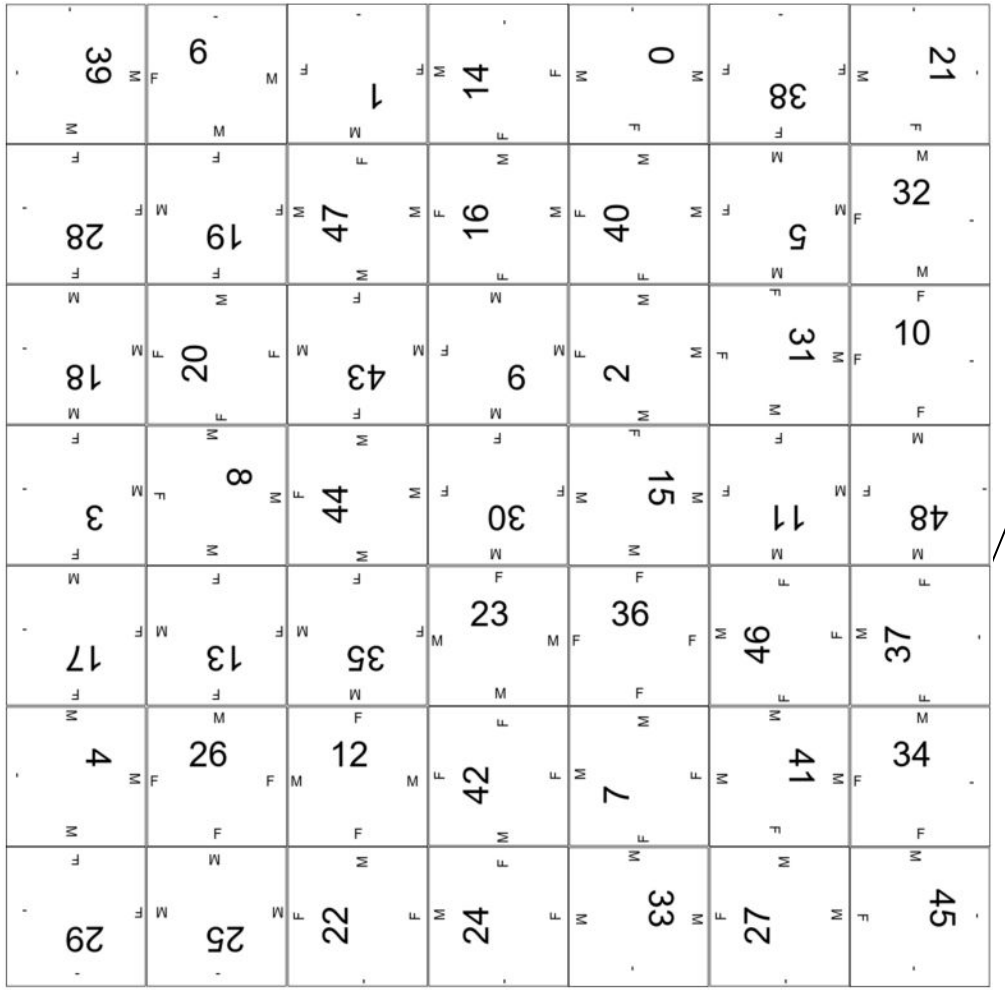
Mock-Puzzle: Warum?

- Testen / Evaluieren von Solvern
  - Reproduzierbar
  - Ohne Digitalisierung
  - Ohne Matching
  - Auf Puzzles beliebiger Größe

Mock-Puzzle: Wie?

- Generieren von Puzzleteilen
  - Zufällige Position
  - Zufällige Rotation
  - Sodass Lösung existiert

- Erstellen der Matchmatrix



Mock-Puzzle: Verrauschen der Matches

- Tatsächliches Ergebnis: $m_t \in \{0, 1\}$

- “Spiegeln” in [0, 1]:

$$m'_{i,j} = \begin{cases} \mathcal{N}(m_t, k_{noise}) & \texttt{if } 0 \leq \mathcal{N}(m_t, k_{noise}) \leq 1 \\ |\mathcal{N}(m_t, k_{noise})| & \texttt{if } \mathcal{N}(m_t, k_{noise}) < 0 \\ 1 - \mathcal{N}(m_t, k_{noise}) + 1 & \texttt{if } \mathcal{N}(m_t, k_{noise}) > 1 \end{cases}$$

- Begrenzen auf [0, 1]:

$$m_{i,j} = \begin{cases} m'_{i,j} & \texttt{if } 0 \leq m'_{i,j} \leq 1 \\ 0 & \texttt{if } m'_{i,j} < 0 \\ 1 & \texttt{if } m'_{i,j} > 1 \end{cases}$$

Evaluierung

Evaluierung: Warum?

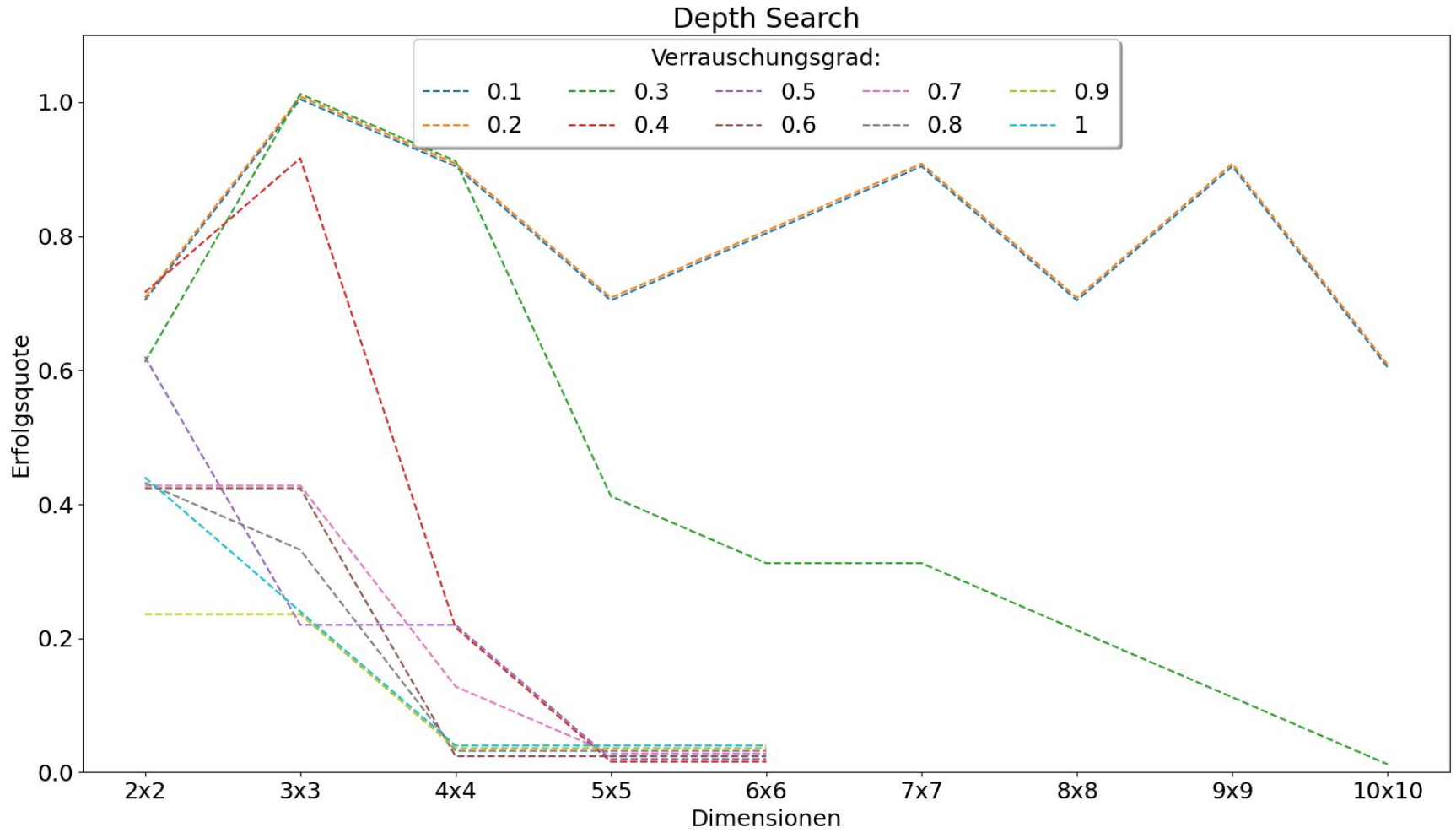
- Erfolgsrate
  - bei verschiedenen Puzzlegrößen
  - bei verschiedenen Matching Ergebnisse

- Laufzeiten
  - der einzelnen Algorithmen
  - bei verschiedenen Puzzlegrößen

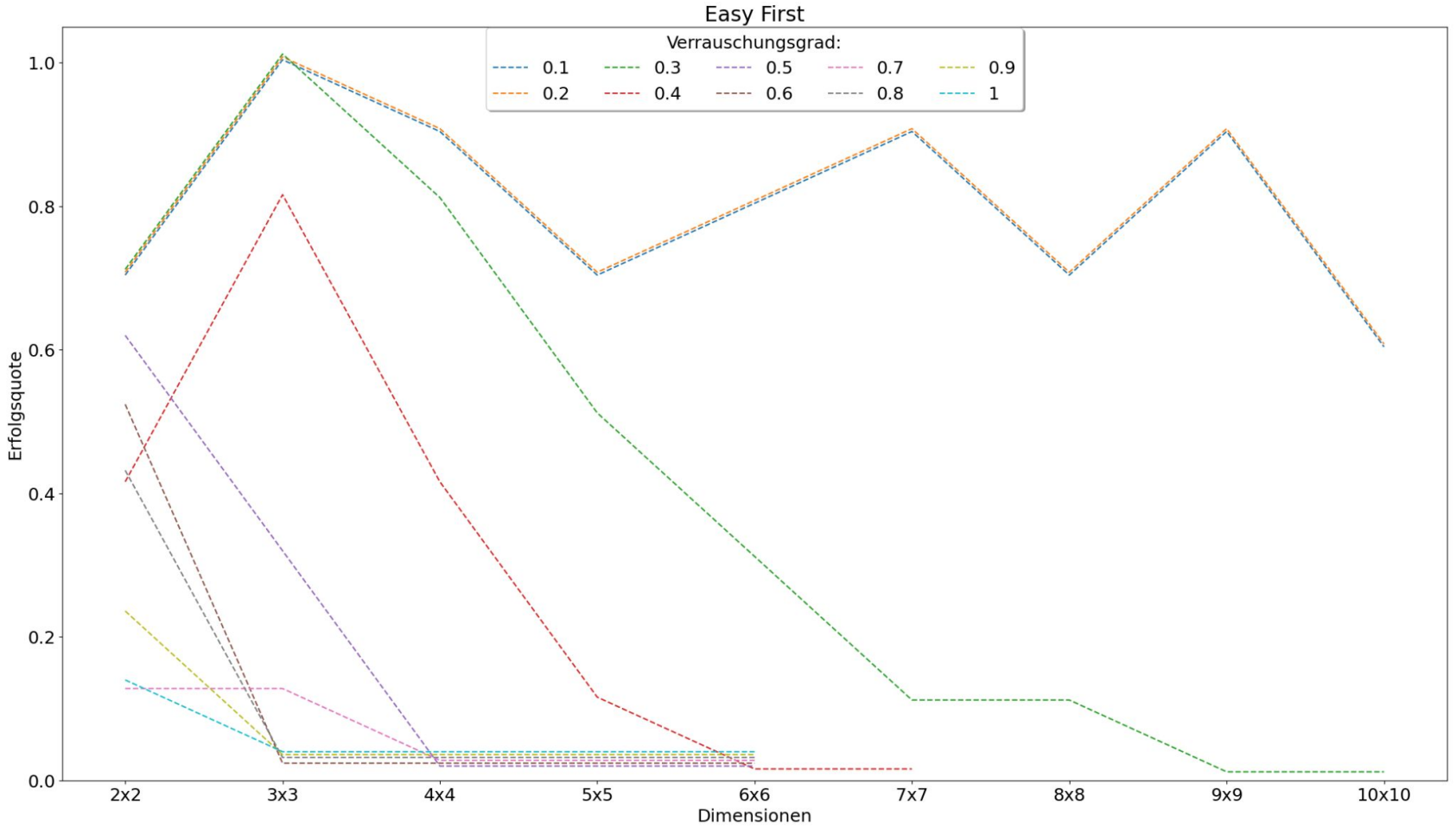
Evaluierung: Wie?

- Verschiedene Puzzlegrößen
- Verschiedene “Verrauschungsgrade”
- 10x pro Puzzlegröße und Verrauschungsgrad
- Für (fast) jeden Solver

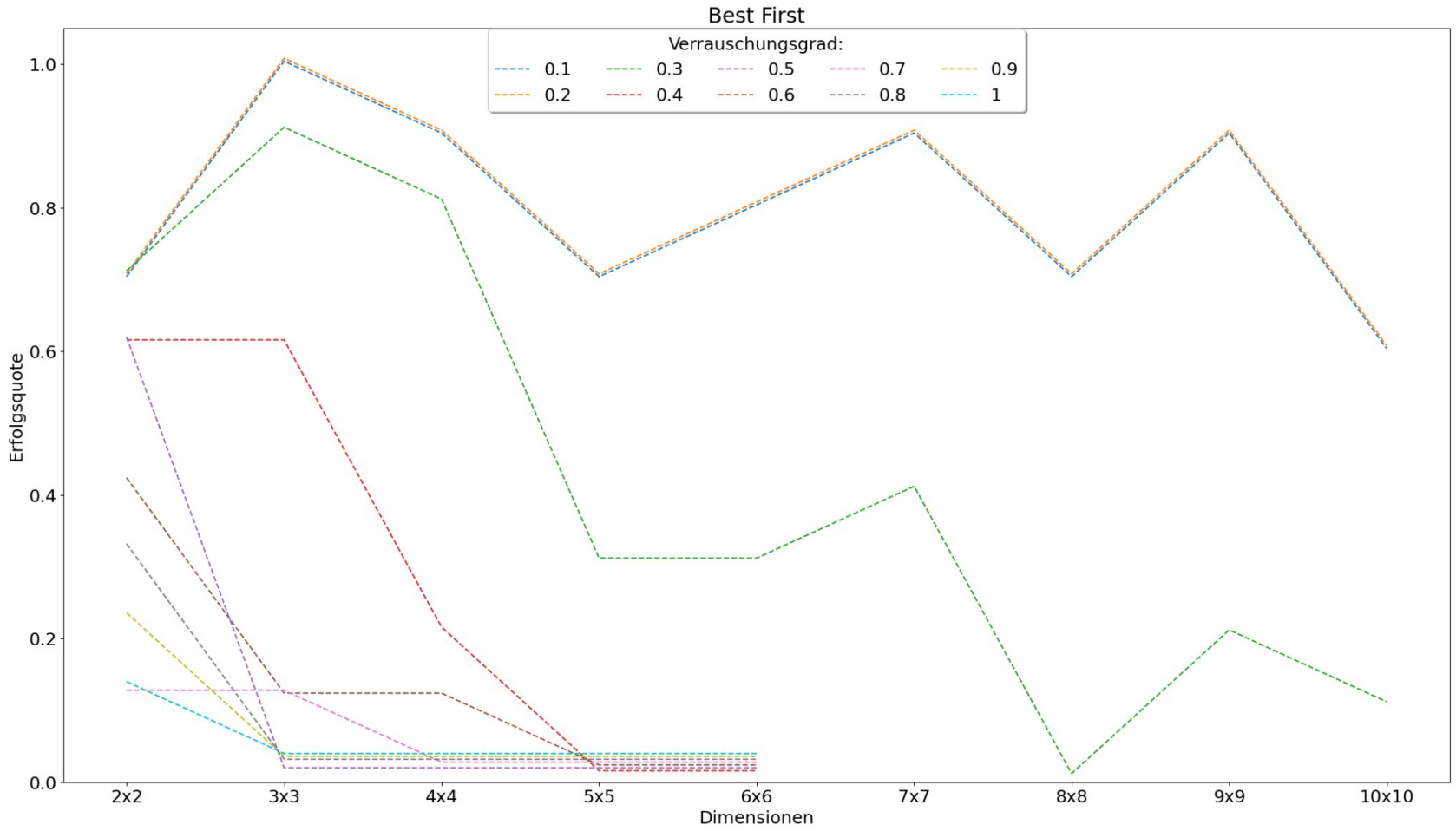
Evaluierung: Erfolgsquote



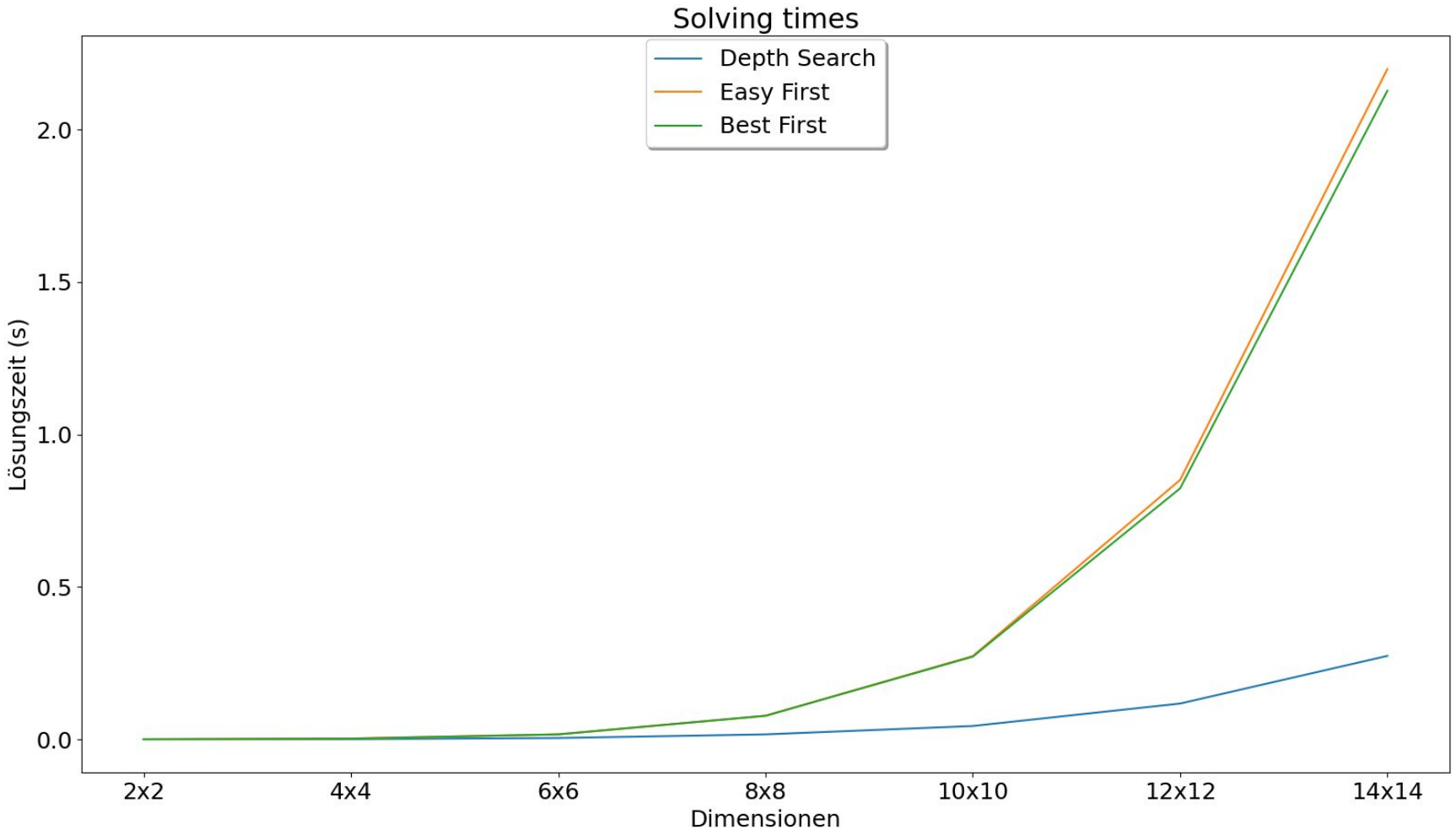
Evaluierung: Erfolgsquote



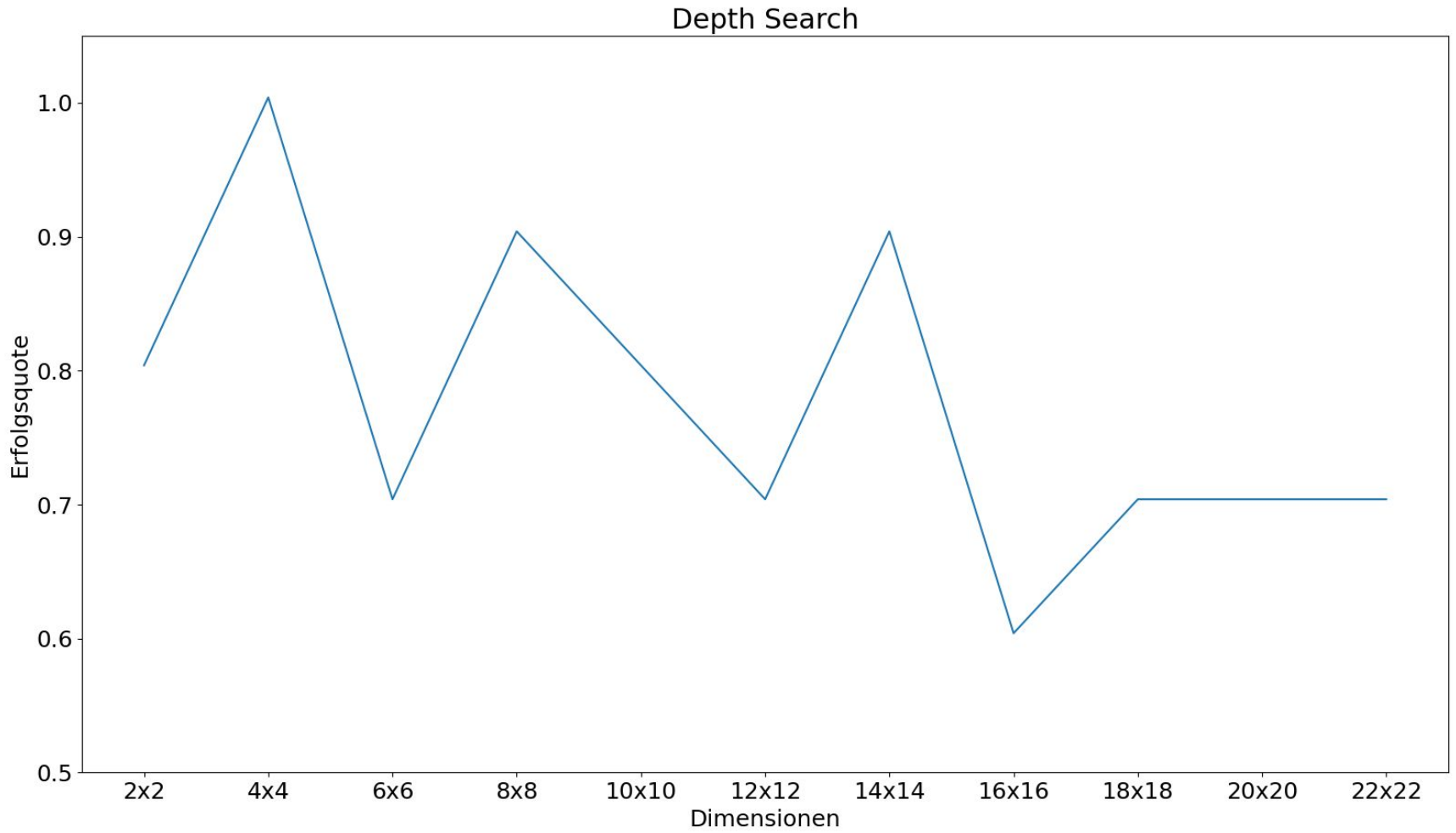
Evaluierung: Erfolgsquote



Evaluierung: Laufzeit



Evaluierung: Größere Puzzles



VIELEN DANK FÜR IHRE AUFMERKSAMKEIT!

Luis Schubert

luis.schubert@hm.edu



SOLVER / POST-PROCESSING

ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

GLIEDERUNG

- NICHTLINEARER LAYOUT-OPTIMIERER
- HERAUSFORDERUNGEN UND LÖSUNGEN
- KOMBINATION LINEARER UND NICHTLINEARER OPTIMIERER

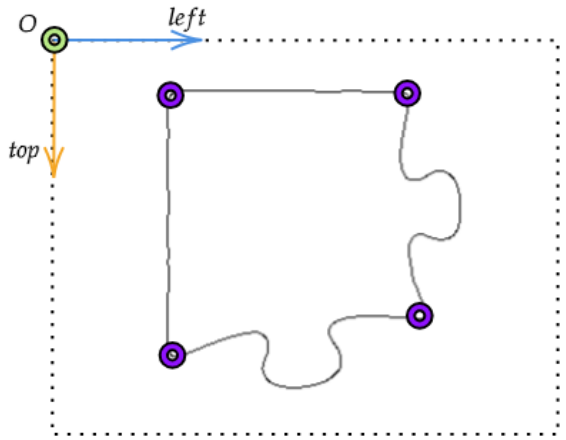
NICHTLINEARER LAYOUT-OPTIMIERER

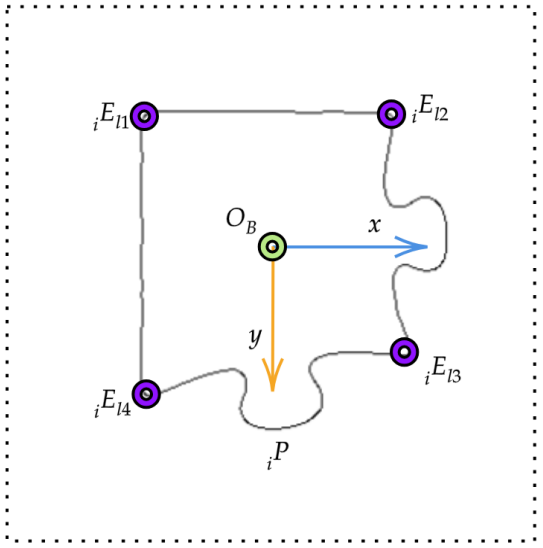
MOTIVATION

- Ziel: Minimierung der Gesamtabstände zwischen benachbarten Ecken von Puzzleteilen
- Berücksichtigt exakte Rotation jedes Puzzleteils ohne Rotationsschätzungen
- Führt zu nichtlinearen Operationen (Rotation von Vektoren, euklidische Abstände)
- Strebt ein globales Optimum an

VERFAHRENSÜBERSICHT

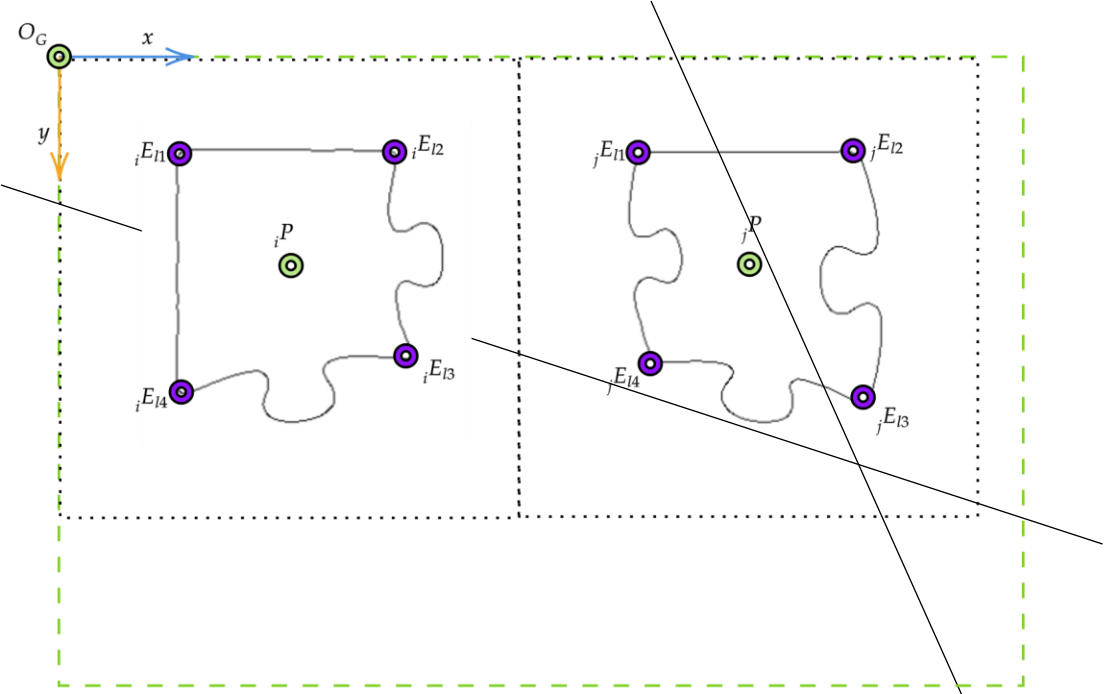
- Gegeben: Gelöstes Puzzle mit N Zeilen und M Spalten
- Eckpositionen jedes Teils relativ zum Bildkoordinatensystem
- Transformation der Teile in globales Koordinatensystem
- Definition der Fehlerfunktion basierend auf euklidischen Abständen zwischen benachbarten Ecken
- Minimierung der Fehlerfunktion mit scipy.minimize

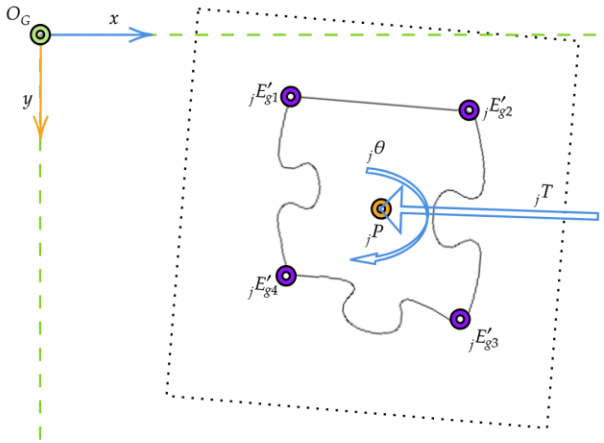




KOORDINATENTRANSFORMATION

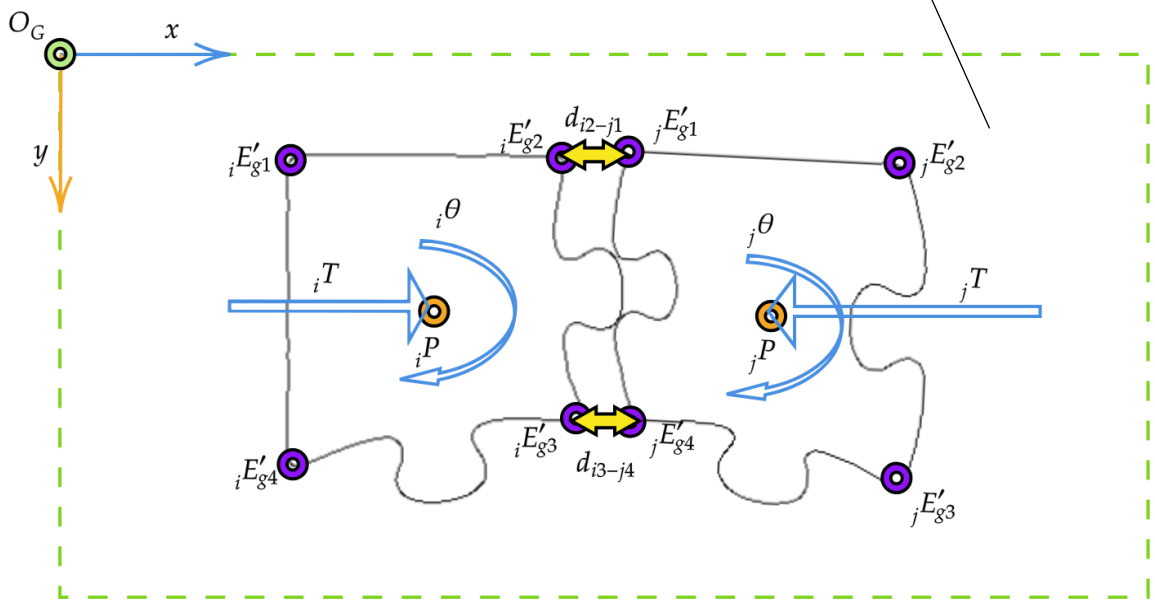
- Auffüllen der Bilder zu Quadraten
- Verschiebung des Ursprungs von der oberen linken Ecke zum Bildmittelpunkt
- Transformation der Eckpositionen in neues Koordinatensystem
- Speicherung der Eckkoordinaten in 2N x 2M Matrix





GLOBALE POSITIONIERUNG

- Positionierung der Teile im globalen Koordinatensystem
- Speicherung globaler Teilpositionen in N x M Matrix
- Berechnung der Eckpositionen im globalen System:
  $E'g1 = R_\theta i \cdot Eg1 + Pi + Ti$



FEHLERFUNKTION

- Summe der euklidischen Abstände zwischen benachbarten Ecken: Ferr(Eg) = Σ ||Egi - Egj||^2
- Eg: Positionsmatrix aller Ecken
- Egi, Egj: Positionen benachbarter Ecken

OPTIMIERUNGSMETHODE (L-BFGS-B)

- Geeignet für großskalige, nichtlineare Probleme
- Speichereffizient
- Box-Beschränkungen
- Skalierbar mit Anzahl der Variablen

MINIMIERUNGSPROZESS

- Verwendung von scipy.minimize mit L-BFGS-B Methode
- Erstellung eines Vektors der Unbekannten: X = [Ti_x, Ti_y, θi]
- Bereitstellung einer Anfangsschätzung und Definition von Grenzen

HERAUSFORDERUNGEN UND LÖSUNGEN



HERAUSFORDERUNGEN UND LÖSUNGEN

- Problem lokaler Minima
- Lösungen:

  Hinzufügen von zufälligem Rauschen und erneute Optimierung

  Kombination linearer und nichtlinearer Optimierung (in nächsten Folien diskutiert)



ERGEBNISSE

- Vor der Optimierung: Ferr = 1,23e+5
- Nach der Optimierung: Ferr = 6,62e+2
- Durchschnittliche Laufzeit: Über 90 Sekunden pro Durchlauf
- Visualisierung des Puzzle-Layouts vor und nach der Optimierung

KOMBINATION LINEARER UND NICHTLINEARER OPTIMIERER

KOMBINATION LINEARER UND NICHTLINEARER OPTIMIERER

- Motivation: Verbindung der Geschwindigkeit linearer Optimierung mit der Genauigkeit nichtlinearer Optimierung
- Verfahren: Verwendung der Ergebnisse des linearen Optimierers als Startwerte für den nichtlinearen Optimierer



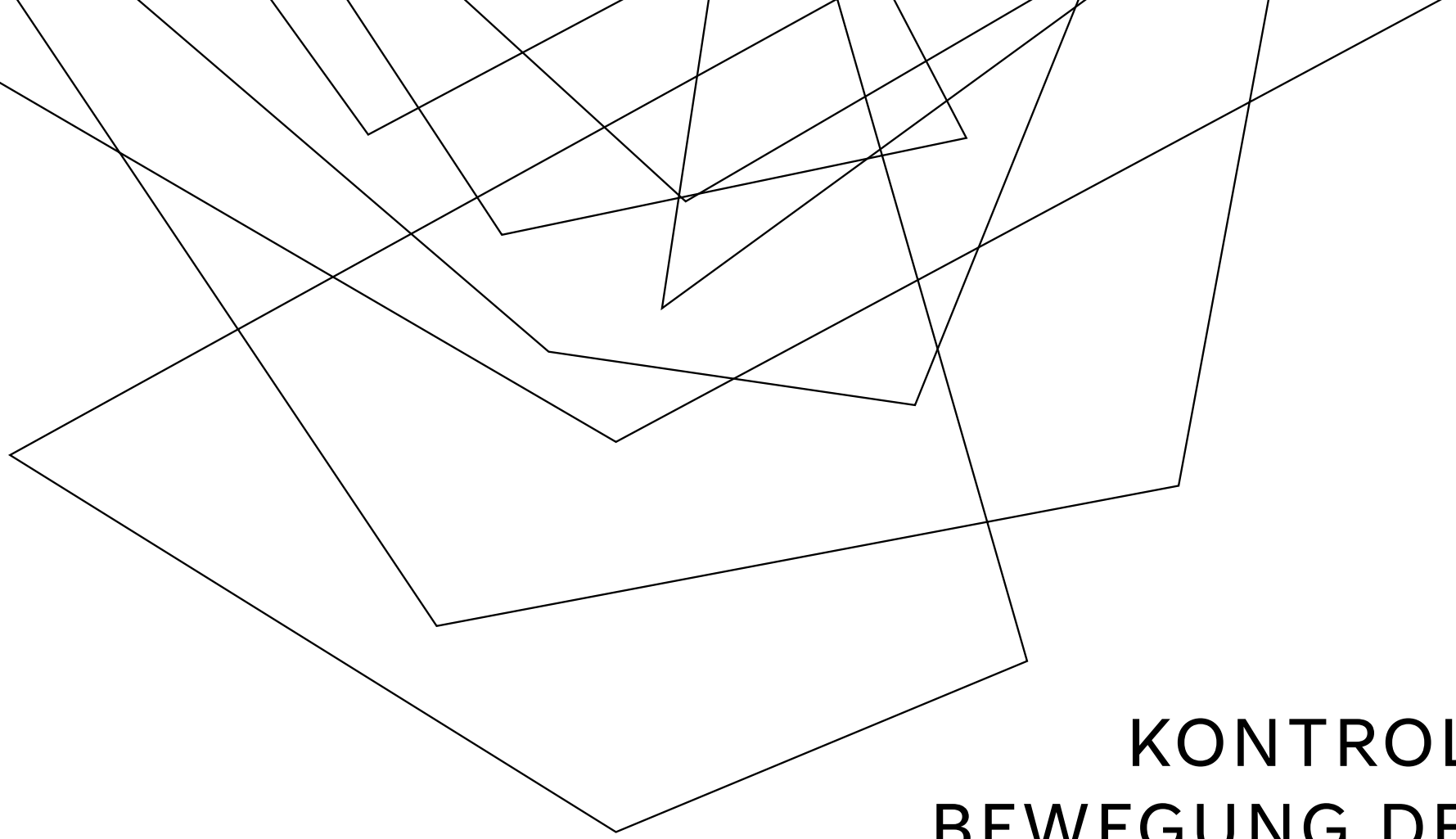
ERGEBNISSE DES KOMBINIERTEN ANSATZES

- Ergebnis des linearen Optimierers: Ferr = 6,81e+2
- Nach nichtlinearer Verfeinerung: Ferr = 6,61e+2
- Durchschnittliche Laufzeit: Unter 15 Sekunden
- Visualisierung des Puzzle-Layouts nach linearer und nichtlinearer Optimierung

VIELEN DANK FÜR IHRE AUFMERKSAMKEIT!

Mohammadjavad Zamani Pozveh

Zamani-p@hm.edu



KONTROLLE DER BEWEGUNG DES ROBOTERS

ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

GLIEDERUNG

- Kurze Übersicht
- DH-Parameter

ÜBERSICHT DER CONTROL - NODE

- Implementiert KUKA-Roboter und übernimmt Steuerung
  - Beschrieben durch modifizierte DH-Parameter
- Verwendet Robotics-toolbox

	Programm	Version
	Python	latest
	ROS2	Humble:latest
	Robotics Toolbox by Peter Corke	1.1.1

ÜBERSICHT DER CONTROL - NODE

- Grob: vier Klassen
  - Main
    - Implementiert Ros-Funktionalitäten
  - KukaKr6
    - Implementiert Roboter Bewegung
  - Position
    - Wrappt x,y,z Koordinaten
  - Watchdog



MODIFIZIERTE DH-PARAMETER

- a: Gliedlänge, Abstand der Glieder anhand der x_{i-1} Achse
- α: Winkel zwischen z_i und z_{i-1} über x_i
- d: Abstand der Glieder anhand der z_i Achse
- Θ: Winkel zwischen x_{i-1} und x_i über z_i

a	α	d	Θ
0	π	-0,675	0
0,3	$\pi/2$	0	0
0,65	0	0	$-\pi/2$
0,155	$\pi/2$	-0,6	0
0	$-\pi/2$	0	0
0	$-\pi/2$	0,1	0



ORGANISATION UND INVERSE KINEMATIK

ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

GLIEDERUNG

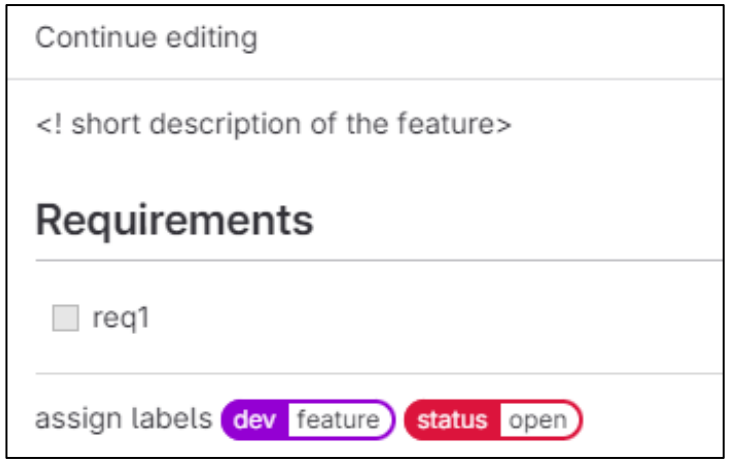
- Organisation
  - Workflow
  - Wiki
- Inverse Analytische Kinematik

CONTROL/ ORGANISATION

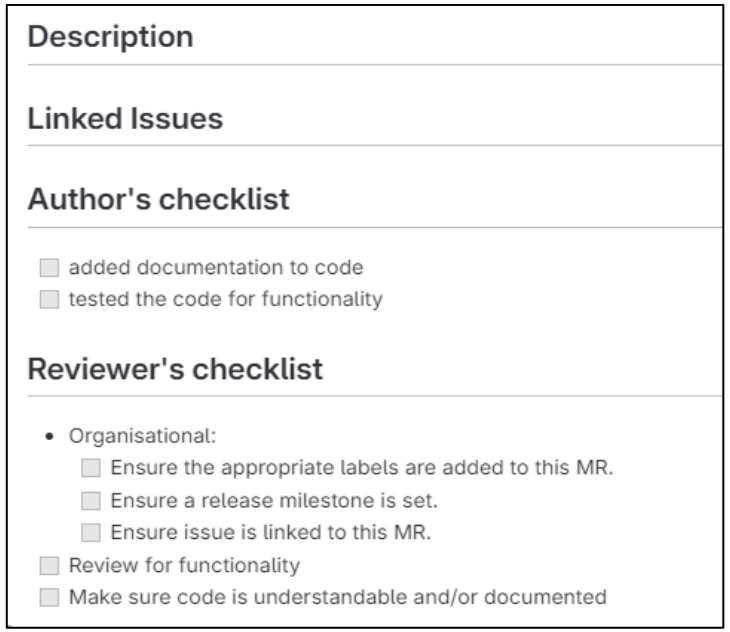
ORGANISATION

Templates und Gitlabstruktur

- Verschiedene Templates für bessere Übersicht
  - Issues (Feature, Bug, Refactor)
  - Merge Requests (Feature)
- Insgesamt: 34 MR, 46 Issues, 5 MS
- Issue Boards pro MS



Issue Template



MR Template

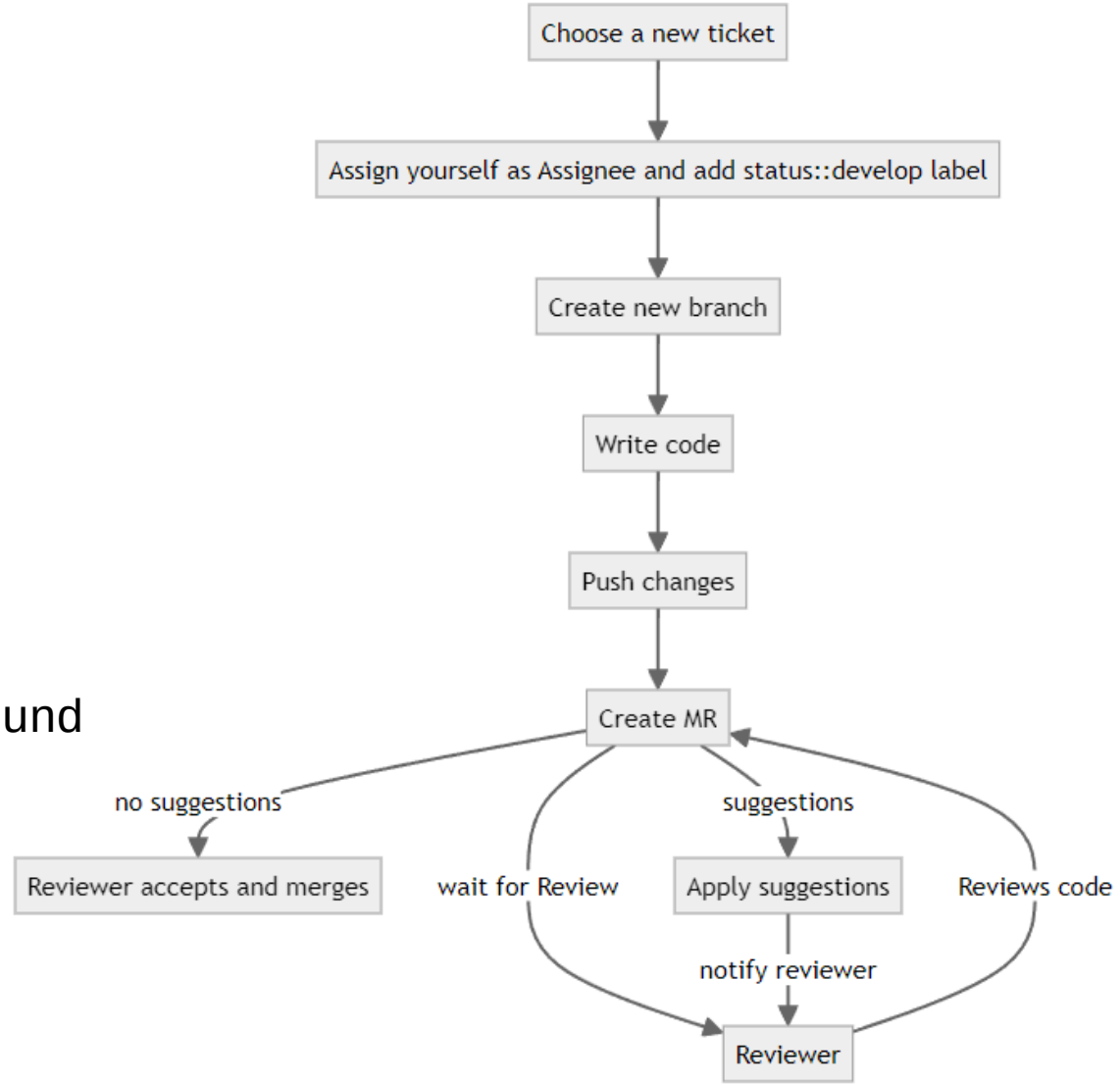
MEILENSTEINE

1	2	3	4	5
04.04.24-18.04.24	**18.04.24-02.05.24**	**02.05.24-16.05.24**	**16.05.24-06.06.24**	**06.06.24-20.06.24**
Erstellen des Roboters	Basis-Funktionalität und Inverse Kinematik	Tests und Interpolation	Verbessern der Roboterbewegung	Optimierung und Performance

ORGANISATION

Workflow in Gitlab

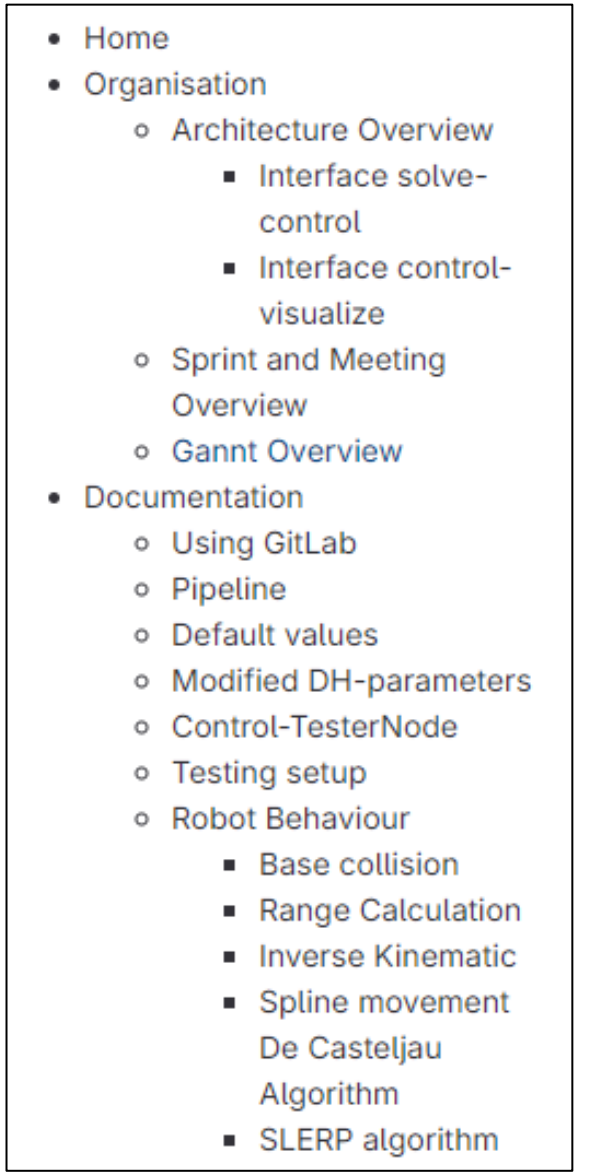
- Jeweils 2-wöchige Meilensteine
- Einteilung von Aufgaben (jeden Donnerstag)
- Jede Aufgabe über MS gemerget und Review



GITLAB WIKI

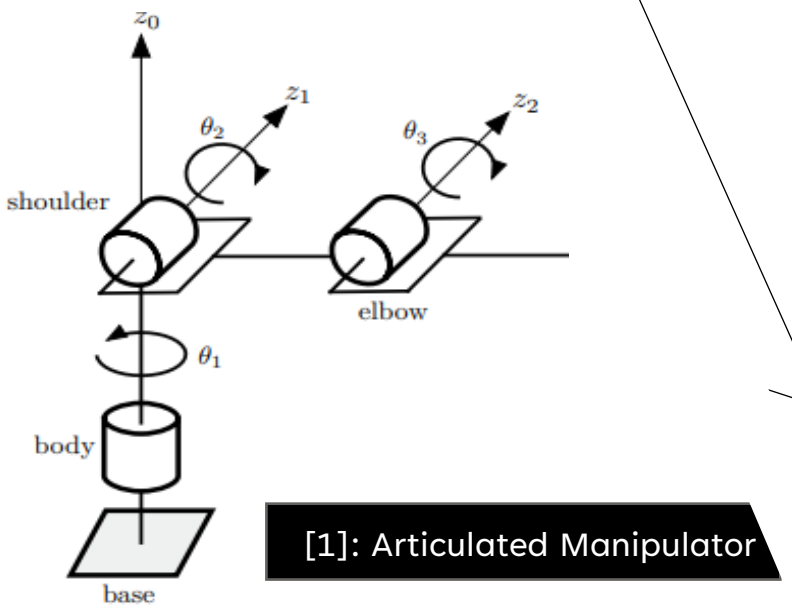
Übersicht

- Wiki gibt eine Komplettübersicht über den Node
- Beinhaltet:
  - Verwendete Programme/Tools
  - Organisation
  - Dokumentation
    für implementierte Funktionen

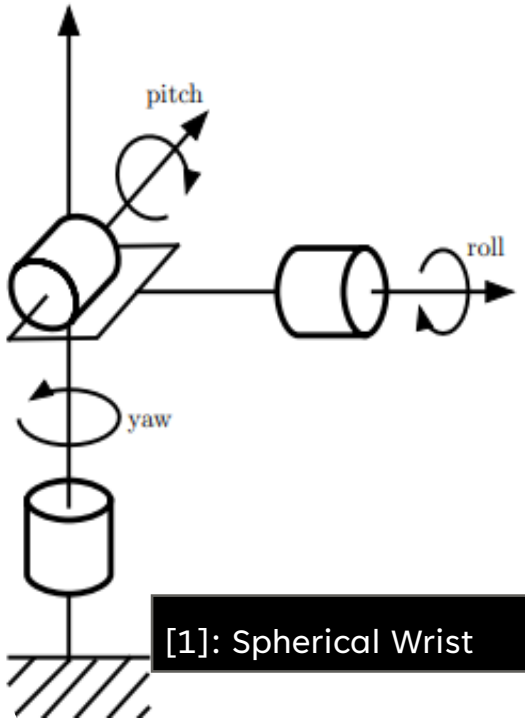




CONTROL/ INVERSE KINEMATIK



[1]: Articulated Manipulator



[1]: Spherical Wrist

ANALYTISCHE INVERSE KINEMATIK

Ziel

- Berechnung
  der Gelenkwinkel anhand geometrischer Eigenschaften des Roboters

Voraussetzung

- Aufteilbarkeit des Roboters
  - Inverses Positionierungsproblem
  - Inverses Orientierungsproblem
- Möglichst kurze Gliedabstände im Orientierungsproblem

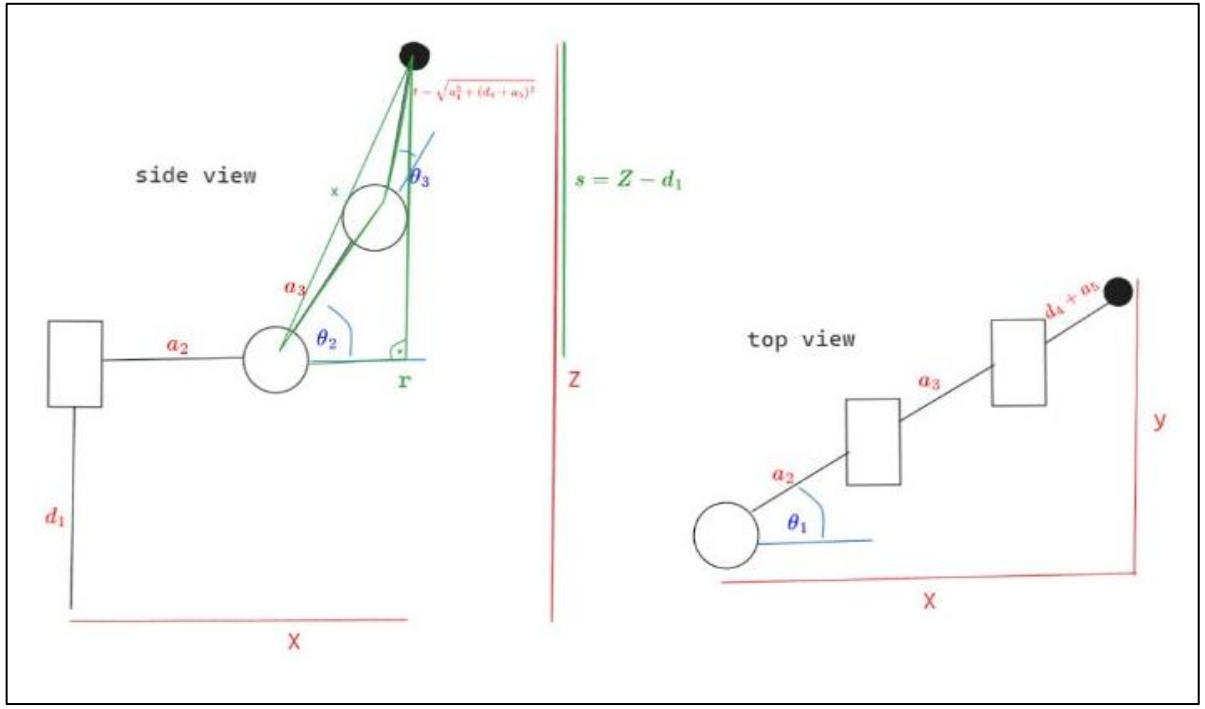
ANALYTISCHE INVERSE KINEMATIK / POSITIONIERUNG

- Betrachtet Roboter
  schematisch von oben und von der Seite
- Berechnen der ersten drei Winkel
  - Bestimmen Position des Endeffektors

$$\theta_1 = \tan^{-1}\left(\frac{Y}{X}\right)$$

$$\theta_2 = \cos^{-1}\left(\frac{s^2 - r^2 x^2}{2rx}\right) - \cos^{-1}\left(\frac{t^2 - a_3^2 x^2}{2a_3 x}\right)$$

$$\theta_3 = 180 - \cos^{-1}\left(\frac{x^2 - t^2 a_3^2}{2ta_3}\right)$$



ANALYTISCHE INVERSE KINEMATIK / ORIENTIERUNG

$$R_6^3 \begin{bmatrix} cos(q4)*cos(q5)*cos(q6) - sin(q4)*sin(q6) & -cos(q6)*sin(q4) - cos(q4)*cos(q5)*sin(q6) & -cos(q4)*sin(q5) \\ cos(q4)*sin(q6) + cos(q5)*cos(q6)*sin(q4) & cos(q4)*cos(q6) - cos(q5)*sin(q4)*sin(q6) & -sin(q4)*sin(q5) \\ cos(q6)*sin(q5) & -sin(q5)*sin(q6) & cos(q5) \end{bmatrix}$$

- Betrachtet Rotation der letzten drei Glieder
  - Aus Produkt der Homogenen Transformationsmatrix

$$\theta_4 = \cos^{-1}\left(-\frac{rot(1,2)}{\sin(\theta_5)}\right)$$

$$\theta_5 = \cos^{-1}(rot(2,2))$$

$$\theta_6 = \cos^{-1}\left(\frac{rot(2,0)}{\sin(\theta_5)}\right)$$

VIELEN DANK FÜR IHRE AUFMERKSAMKEIT!

Pascal Severin

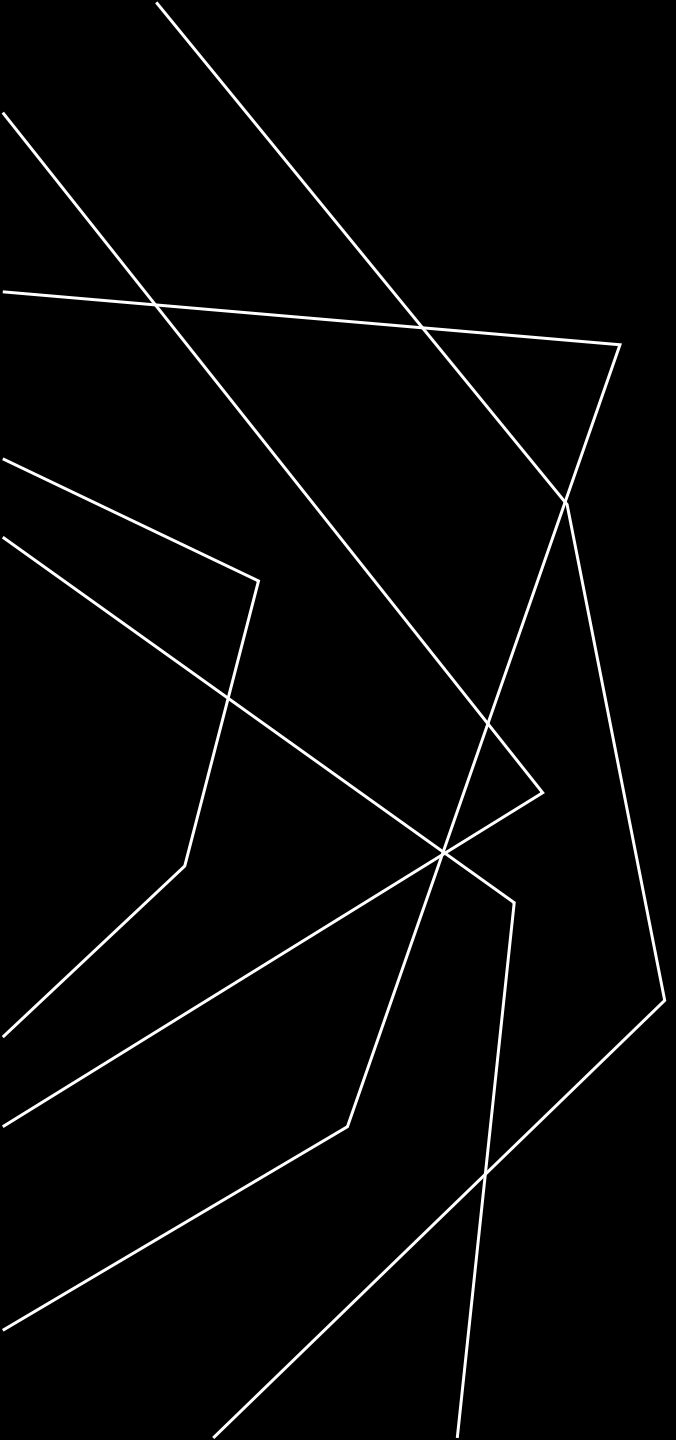
@pascal_severin (gitlab)

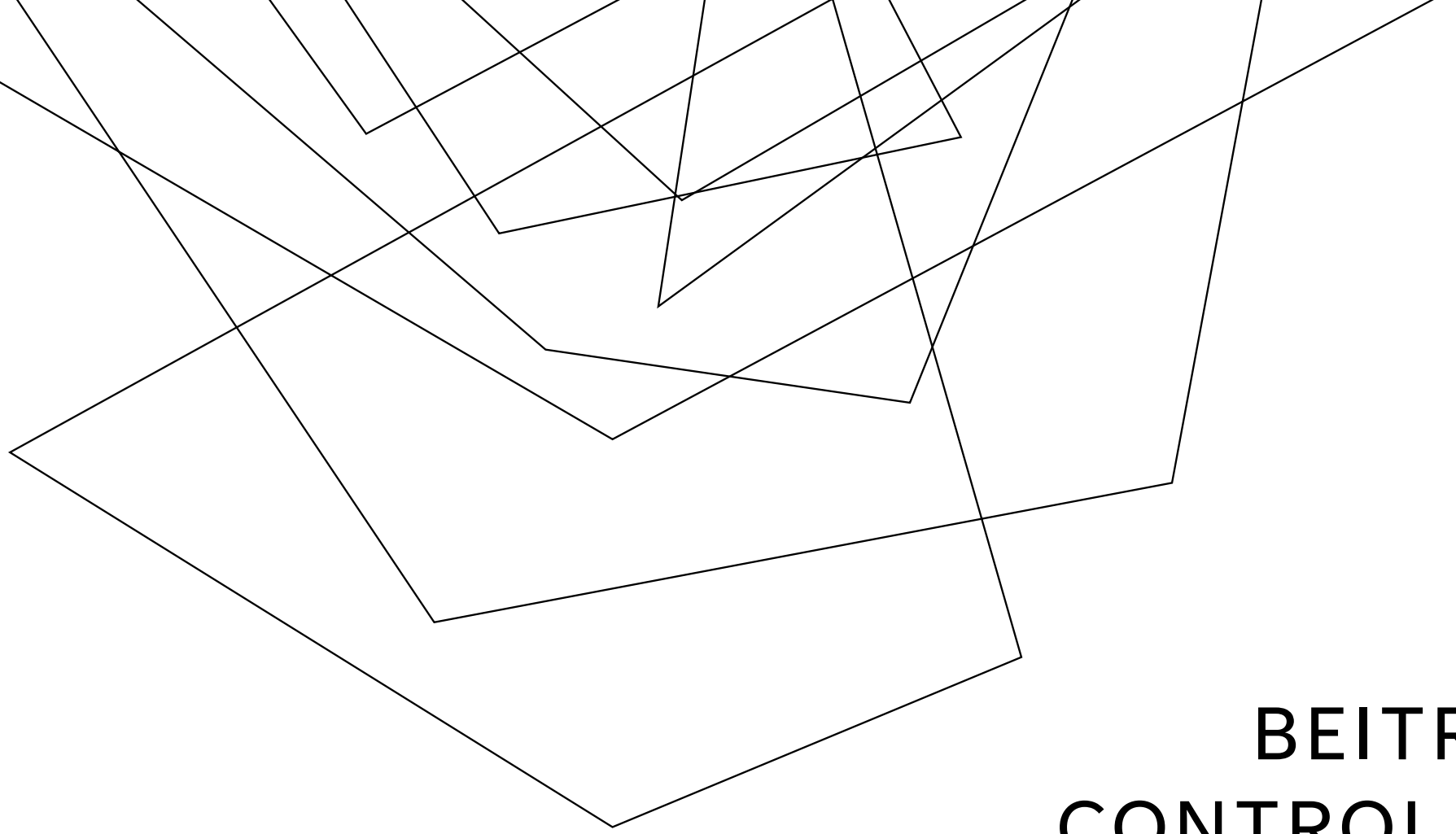
Bildquellen:

- Python-Logo: https://www.cleanpng.com/png-python-computer-icons-programming-language-executa-7247110/
- ROS-Logo: https://docs.ros.org/en/humble/_static/humble-small.png
- Peter Corke Logo: https://raw.githubusercontent.com/petercorke/robotics-toolbox-python/master/docs/figs/RobToolBox_RoundLogoB.png

Textquellen:

- [1] Arteaga, M.A., Gutiérrez-Giles, A., Pliego-Jiménez, J. (2022). A General Overview of Robot Manipulators. In: Local Stability and Ultimate Boundedness in the Control of Robot Manipulators. Lecture Notes in Electrical Engineering, vol 798. Springer, Cham. https://doi.org/10.1007/978-3-030-85980-0_1





BEITRAG CONTROL/SOLVER

ROBOTIK PROJEKTSTUDIUM
PROF. DR. FISCHER - SS24

GLIEDERUNG

- Beitrag Team Control
  - Inverse Kinematik
  - ROS2 Integration
- Beitrag Team Solver
  - Layout Optimierung
  - ROS2 Integration

GLIEDERUNG

- Beitrag Team Control
  - Inverse Kinematik
  - ROS2 Integration
- Beitrag Team Solver
  - Layout Optimierung
  - ROS2 Integration

BEITRAG TEAM CONTROL

INVERSE KINEMATIK

INVERSE KINEMATIK

Implementierung der Newton-Raphson Methode

1. Newton-Raphson Methode bereits bekannt
2. Implementierung dieser in der Roboter-Klasse
3. Benötigt Startpose Tep und Startkonfiguration q0
4. Gibt zurück:
   a. Fehlerwert
   b. Gelenkkonfiguration
   c. Status (Fehler oder Erfolg)

```
def ikine_NR(self, Tep: SE3,
             q0: NDArray,
             eps: float = 1e-3,
             i_lim: int = 100,
             We: NDArray = np.ones((6,))
             ) -> Tuple[float, NDArray, bool]:
```

BEITRAG TEAM CONTROL

ROS2 INTEGRATION

ROS2 INTEGRATION

Aufgaben

1. Empfangen der Spawn- und Zielpose von Solver-Node
2. Berechnen der Trajektorien, um Teil-Zielpose zu erreichen
3. Publishen der Gelenkkonfigurationen an Visualizer-Node
4. Publishen des Puzzle-Status (Abgelegt oder Angeheftet) an Solver-Node

ROS2 INTEGRATION

Schnittstellen

1. Topic: „/solver/nextpiece“

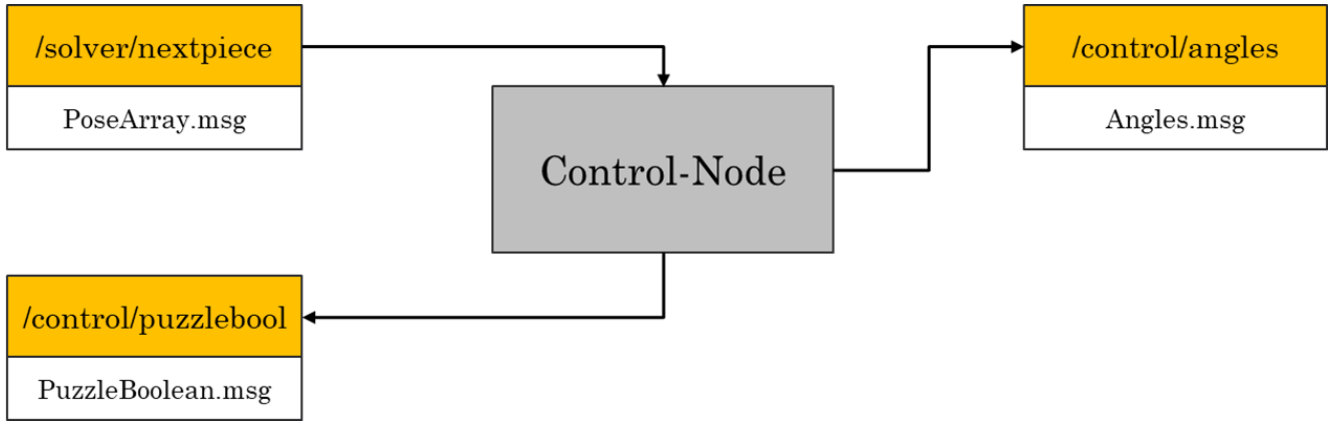
   ⇒ Spawn- und Zielpose

2. Topic: „/control/puzzlebool“

   ⇒ Status des aktuellen Puzzleteils

3. Topic: „/control/angles“

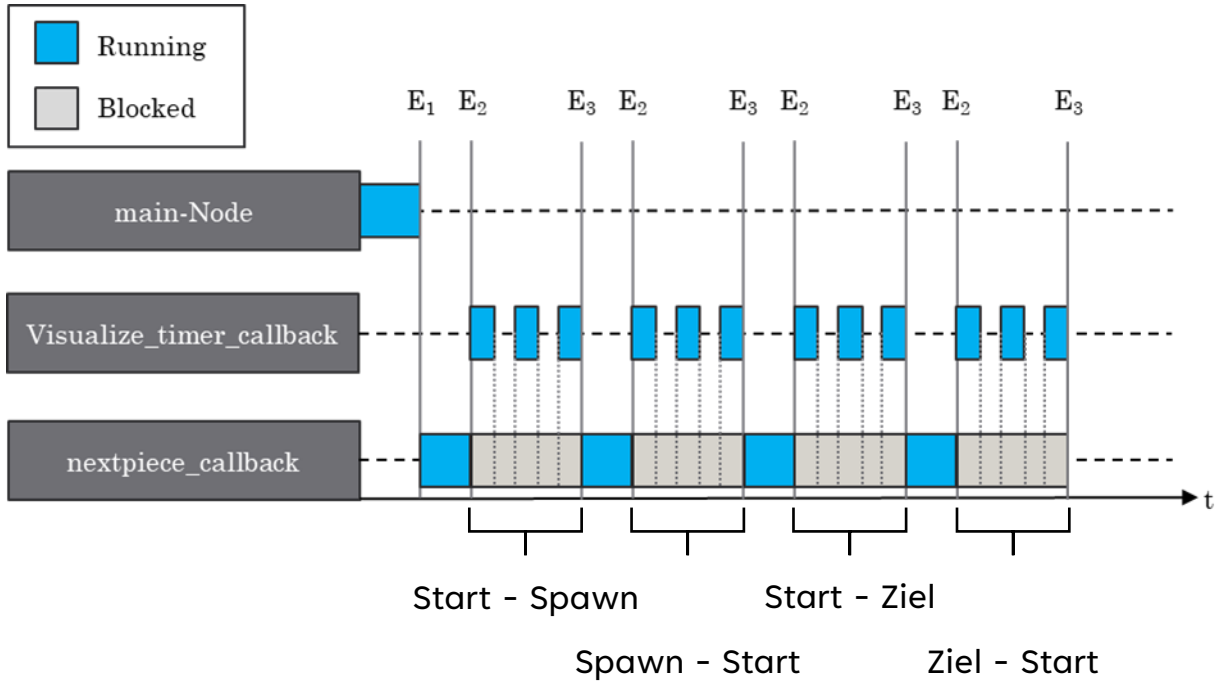
   ⇒ Gelenkkonfiguration und Greifinformation



ROS2 INTEGRATION

Ablauf

1. E_1: Nextpiece-Nachricht empfangen
2. E_2: Teiltrajektorie berechnet
3. E_3: Trajektorie versendet



GLIEDERUNG

- Beitrag Team Control
  - Inverse Kinematik
  - ROS2 Integration
- Beitrag Team Solver
  - Layout Optimierung
  - ROS2 Integration

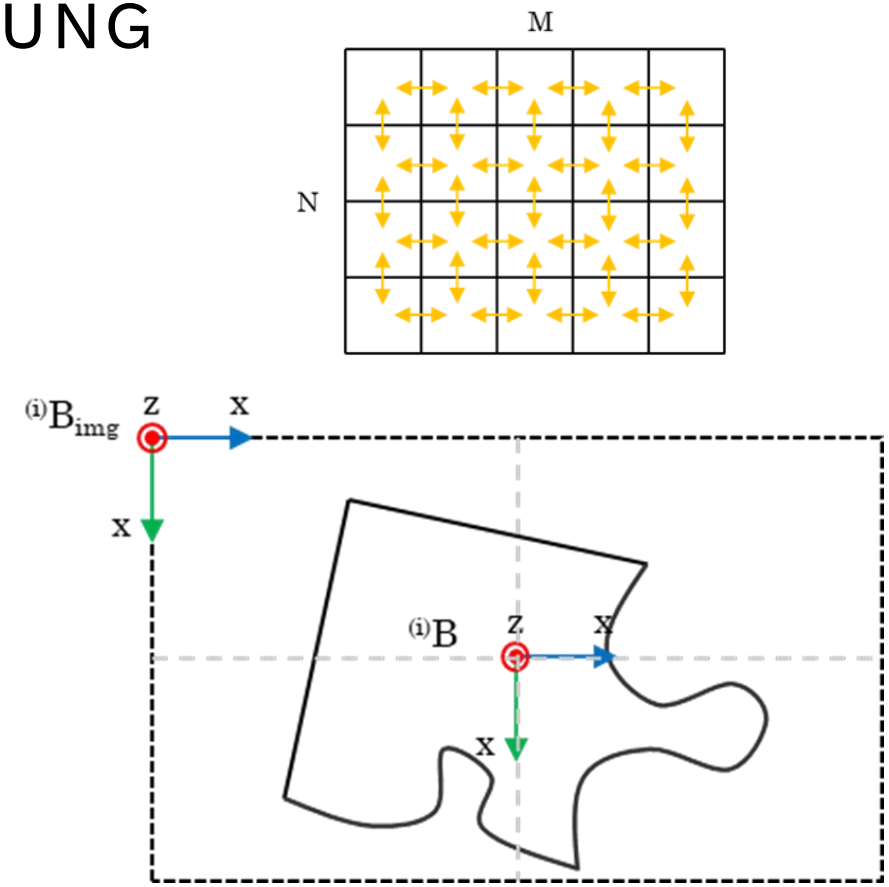
BEITRAG TEAM SOLVER

LAYOUT OPTIMIERUNG

(LINEARE) LAYOUT OPTIMIERUNG

Gegeben

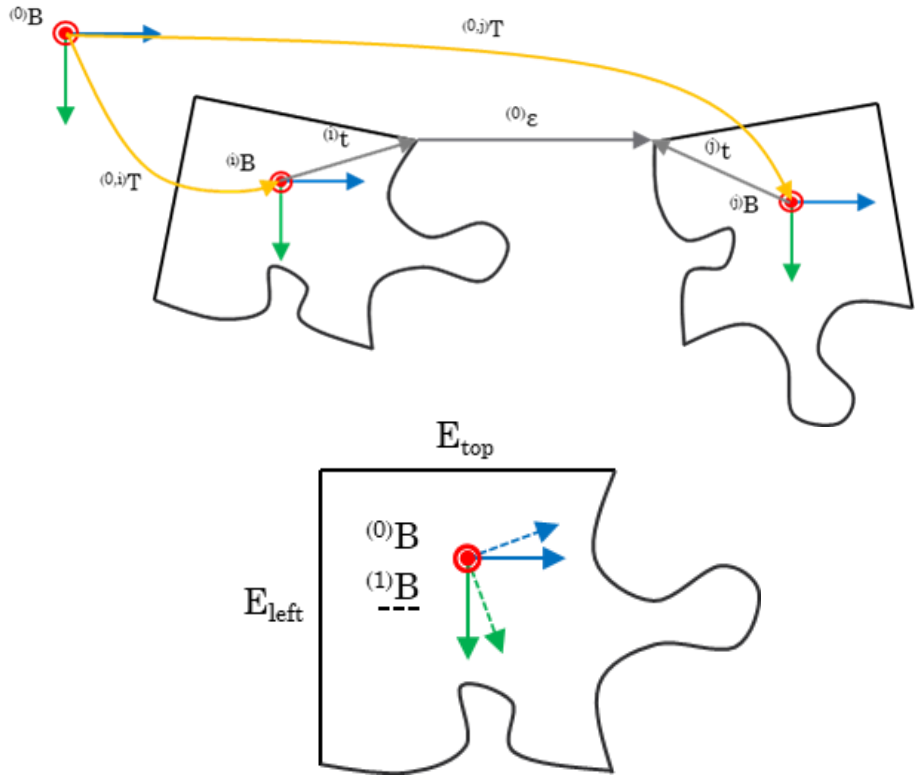
1. Gegeben das gelöste Puzzle (Kanten/ Ecken-Korrespondenzen)
2. Koordinatensysteme der Puzzleteile befinden sich in der Mitte des Bildes (reine Translation)



(LINEARE) LAYOUT OPTIMIERUNG

Ziel

1. Finden der Posen der Puzzleteile ${}^{(0,i)}T$ im Referenzsystem ${}^{(0)}B$, so dass:
   a. Die quadratische Summe der euklidischen Distanz der Fehlervektoren zwischen korrespondierendem Ecken minimal wird
   b. Referenzsystem in Ursprung der linken, oberen Puzzleteil-Basis ${}^{(1)}B$
   c. Kanten des linken, oberen Teils entlang der x- bzw. y-Achse von ${}^{(0)}B$



(LINEARE) LAYOUT OPTIMIERUNG

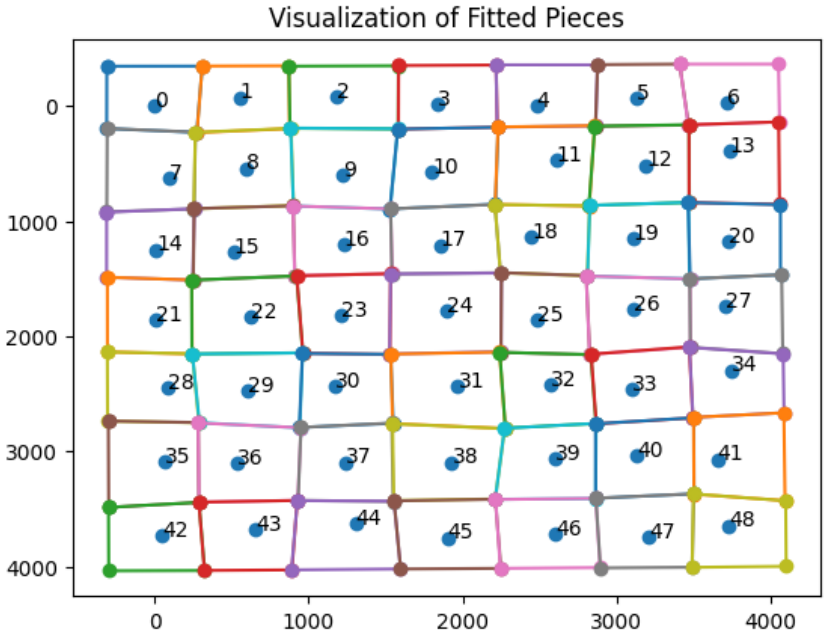
Formulierung des Problems

1. Least Squares Problem
2. Berücksichtigung von b) und c) durch zusätzliche Fehlerterme
3. Linearisierung der Rotation nach dem Winkel erlaubt effiziente Lösung des Problems über SVD
4. Mehrere Iterationen nötig, da Linearisierungsfehler

$$\arg\min_x \frac{1}{2} \sum^{|K|} \|\epsilon(x)\|_2^2$$

$$x = \begin{bmatrix} \phi_i & x_i & y_i & \dots & \phi_{|P|} & x_{|P|} & y_{|P|} \end{bmatrix}^T$$

(LINEARE) LAYOUT OPTIMIERUNG



Ergebnisse

1. Konvergenz nach 30 ms (3 Iterationen)

2. Optisch werden sehr gute Ergebnisse erreicht

3. Einheit der Posen Translation ist Pixel!



BEITRAG TEAM SOLVER

ROS2 INTEGRATION

ROS2 INTEGRATION

Aufgaben

1. Empfangen der segmentierten Bilder von Digitalize-Node
2. Lösen des Puzzles
3. Publishen der Spawnpose des Puzzles an Visualizer-Node
4. Publishen der Spawn- und Zielpose des Puzzles an Control-Node

ROS2 INTEGRATION

Schnittstellen

1. Action Client: „/digitalize/segement“

   ⇒ Bilder (+ ID)

2. Topic „/contol/puzzlebool“

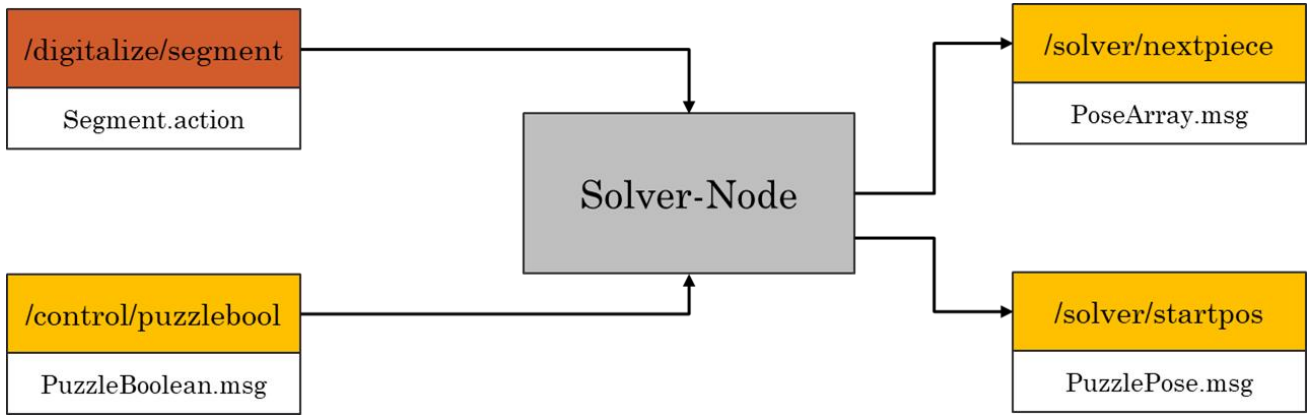
   ⇒ Status des aktuellen Puzzleteils

3. Topic „/solver/nextpiece“

   ⇒ Spawn- und Zielpose (Translation in m)

4. Topic „/solver/startpos“

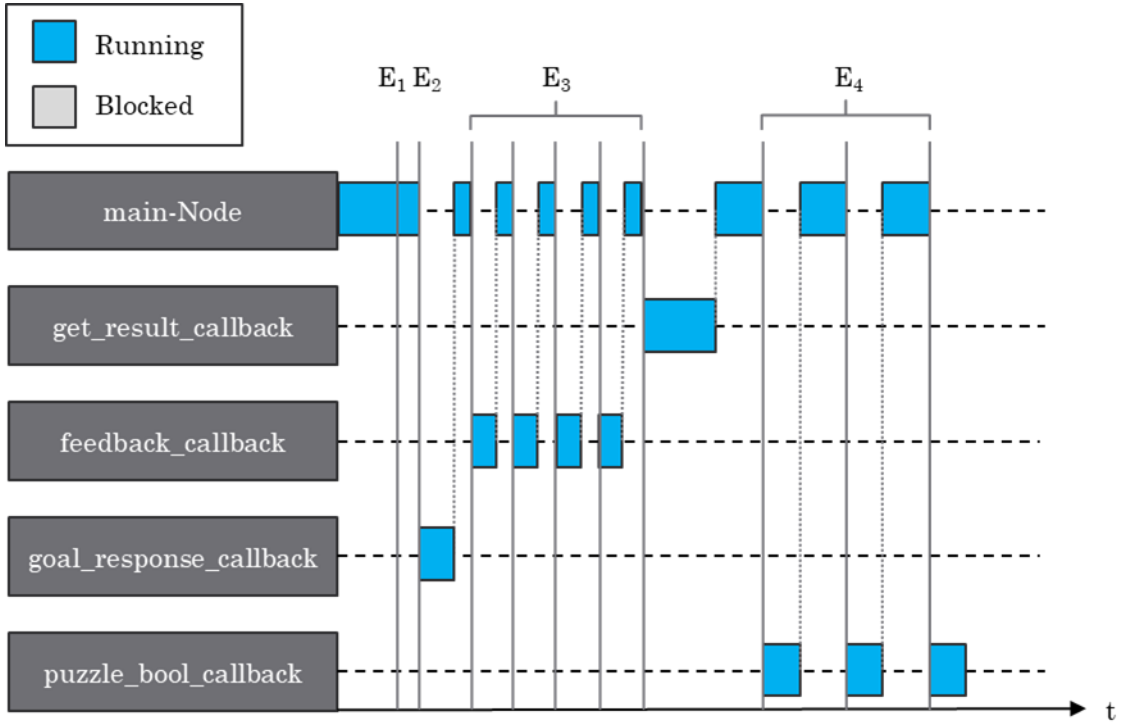
   => Spawnpose (Translation in mm)



ROS2 INTEGRATION

Ablauf

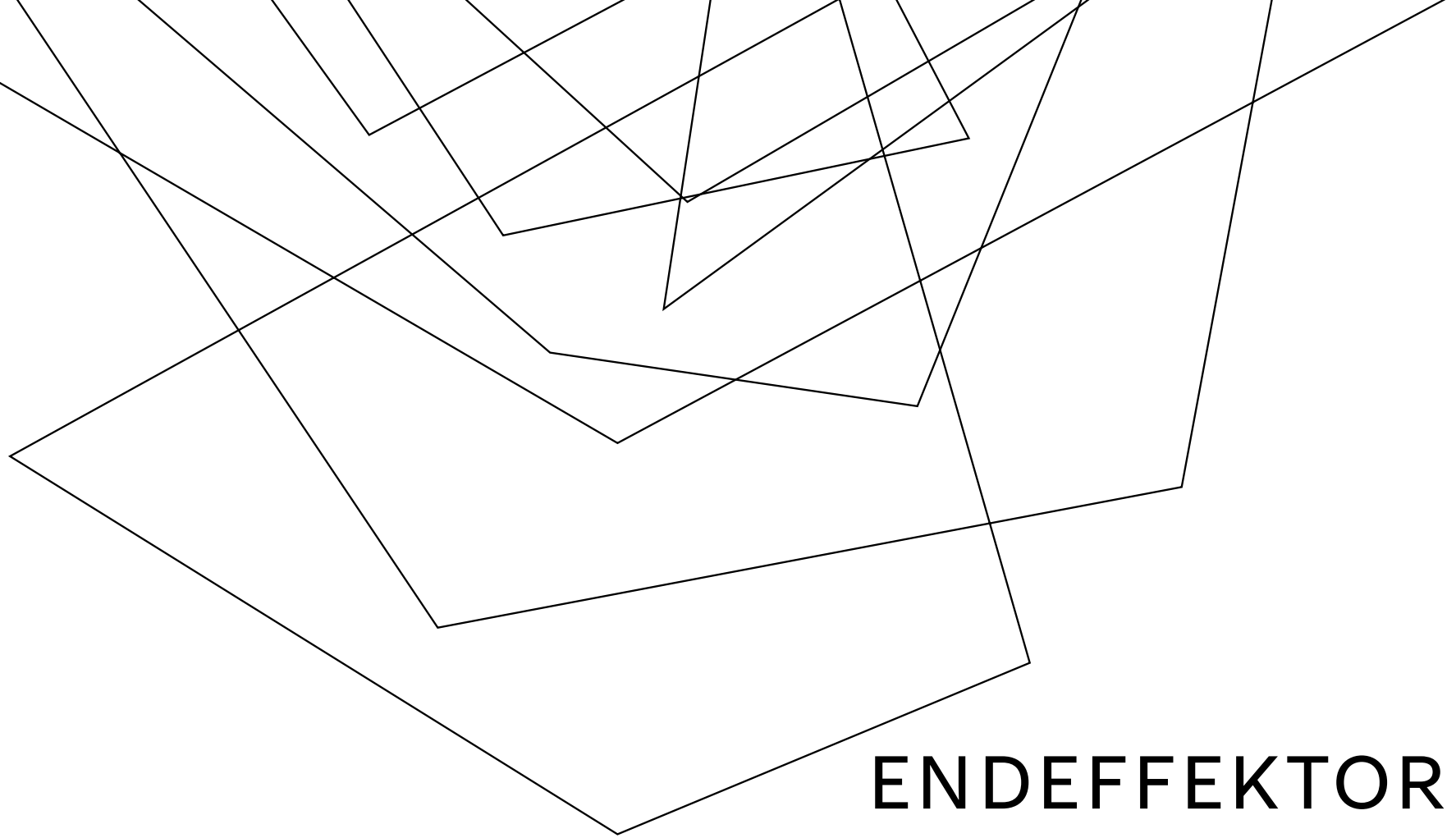
1. E_1: Action-Client Goal-Anfrage
2. E_2: Antwort des Action-Servers
3. E_3: Feedback-Nachricht (Bild) kommt an
4. E_4: Puzzle Teil wurde abgelegt (PuzzleBoolean data toggelt auf False)



VIELEN DANK FÜR IHRE AUFMERKSAMKEIT!

Sami Hamza

sami.hamza0@hm.edu



ENDEFFEKTOR BEWEGUNG UND ROTATION

ROBOTIK PROJEKTSTUDIUM

PROF. DR. FISCHER - SS24

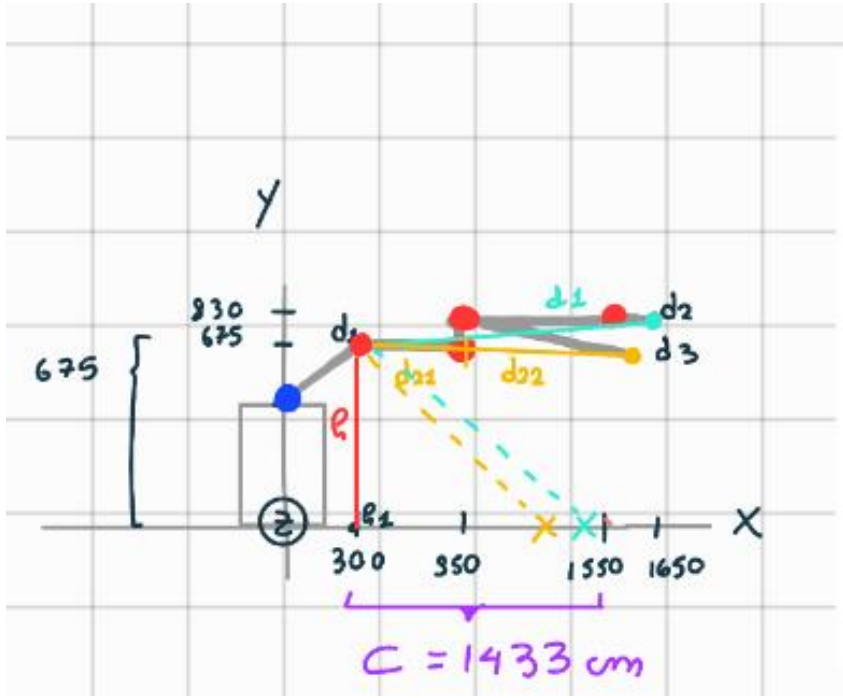
GLIEDERUNG

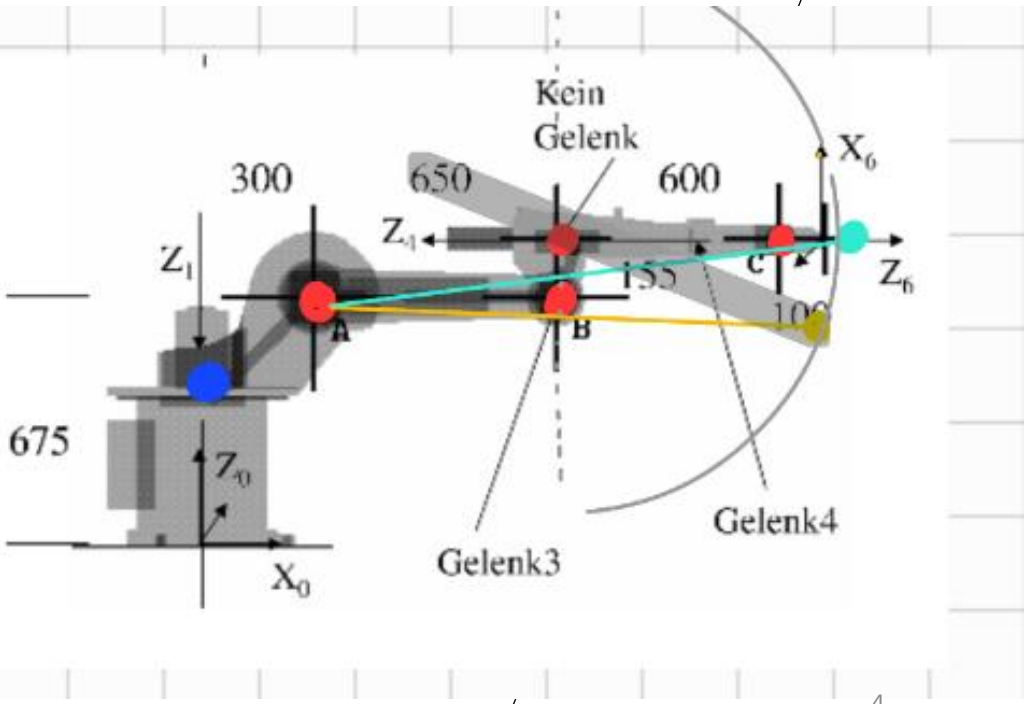
- Berechnung der Robotergreifweite
- Implementierung des SLERP Algorithmus
- Implementierung des Bewegung-wrappers

BERECHNUNG DER ROBOTERGREIFWEITE

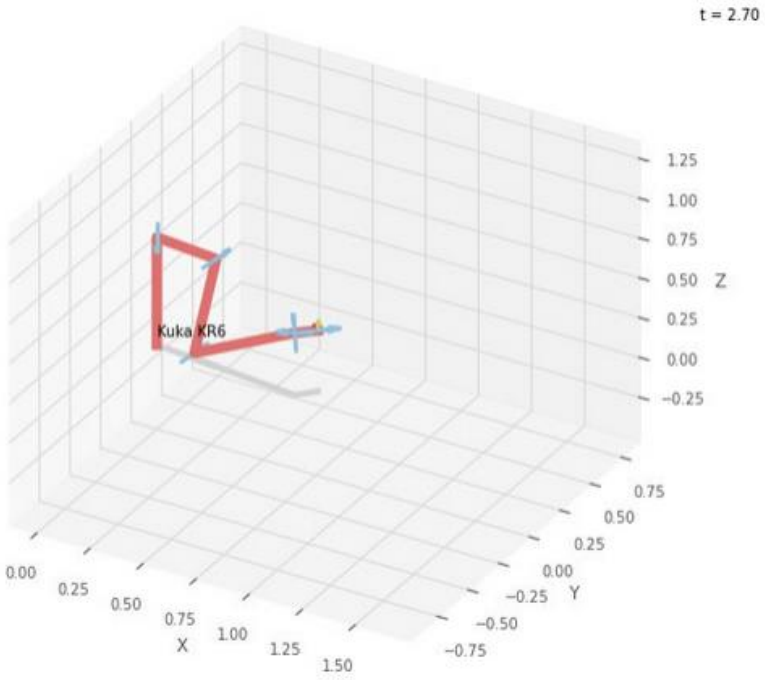
BERECHNUNG DER GREIFWEITE

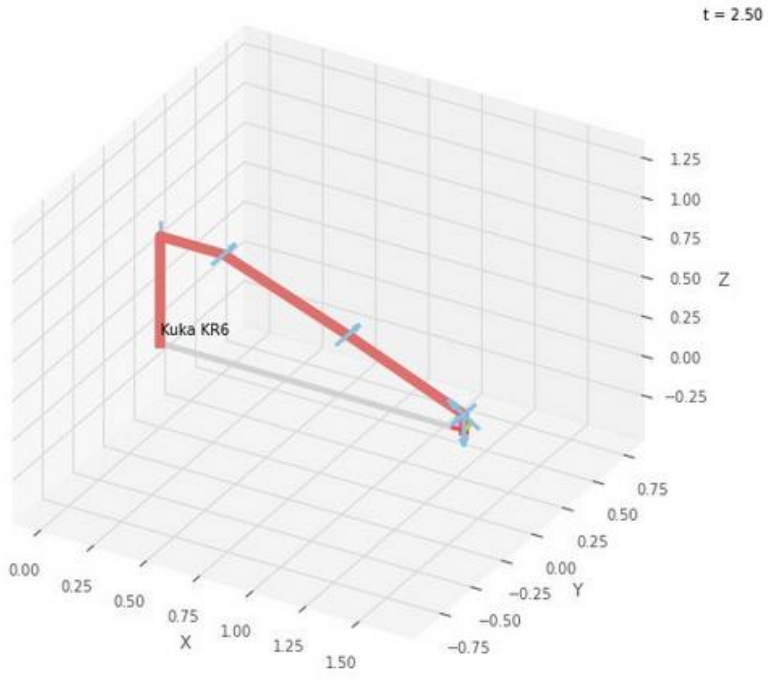
Ziel: Maximale und minimale Greifweite





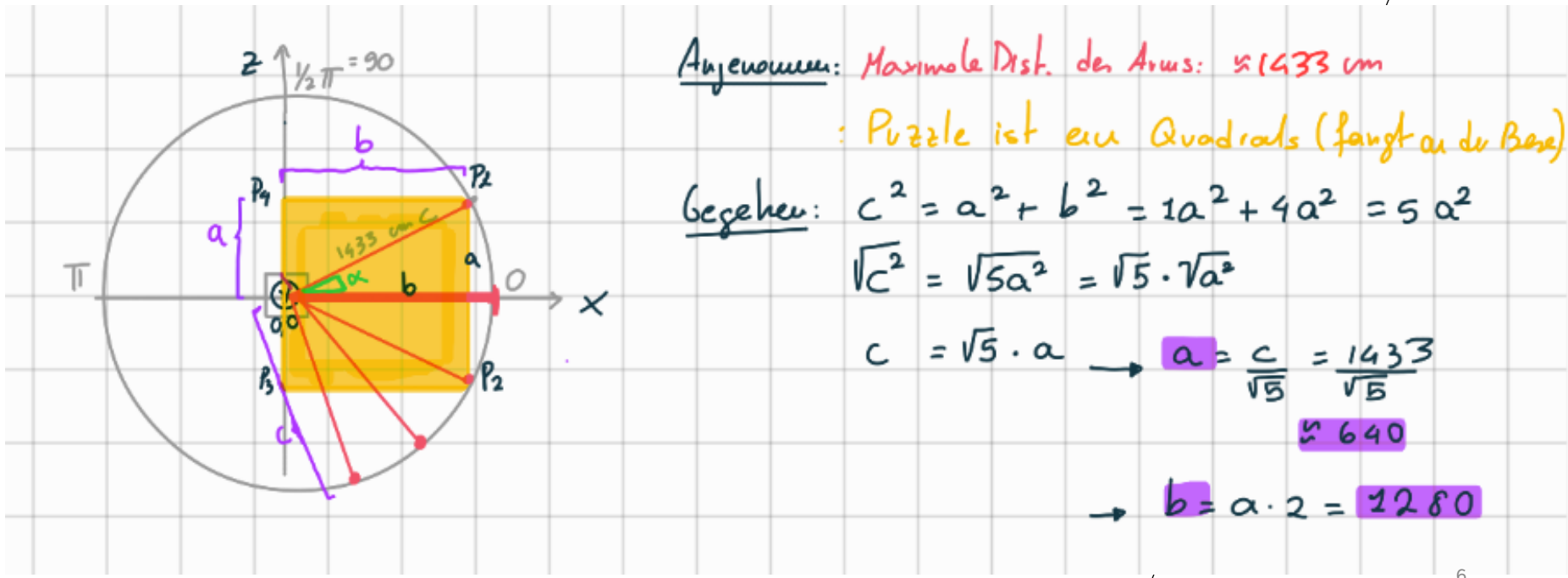
BERECHNUNG DER GREIFWEITE





BERECHNUNG DER GREIFWEITE

Ergebnis : 1433 cm Radius



Angenommen: Maximale Dist. der Arms: ≈ 1433 cm

: Puzzle ist ein Quadrats (fangt an der Base)

Gegeben: $c^2 = a^2 + b^2 = 1a^2 + 4a^2 = 5a^2$

$\sqrt{c^2} = \sqrt{5a^2} = \sqrt{5} \cdot \sqrt{a^2}$

$c = \sqrt{5} \cdot a \rightarrow a = \frac{c}{\sqrt{5}} = \frac{1433}{\sqrt{5}}$

≈ 640

$\rightarrow b = a \cdot 2 = 1280$

INTERPOLATION DER ENDEFFEKTOR-ROTATION MIT DEM SLERP-ALGORITHMUS

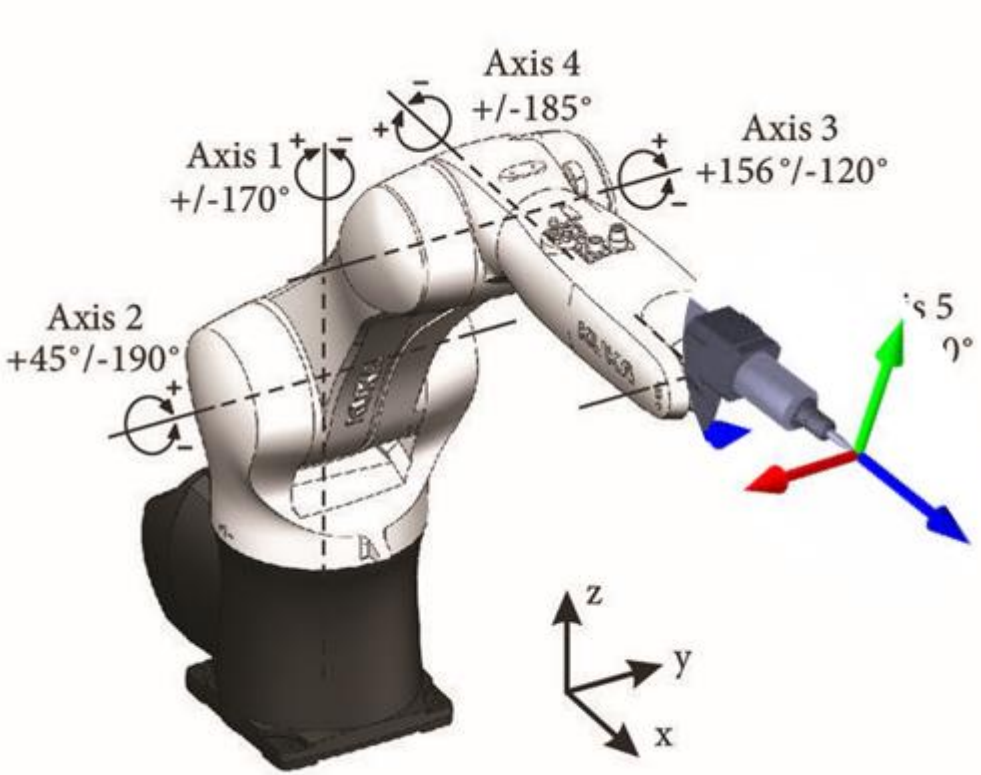
ROTATION

Typologien:

1. Objekt Rotation
2. Frame Rotation

Darstellungsmöglichkeiten:

1. Quaternionen
2. Rotationmatrix
3. Euler Winkeln
4. Axis-Angle
5. Rotation Vektor

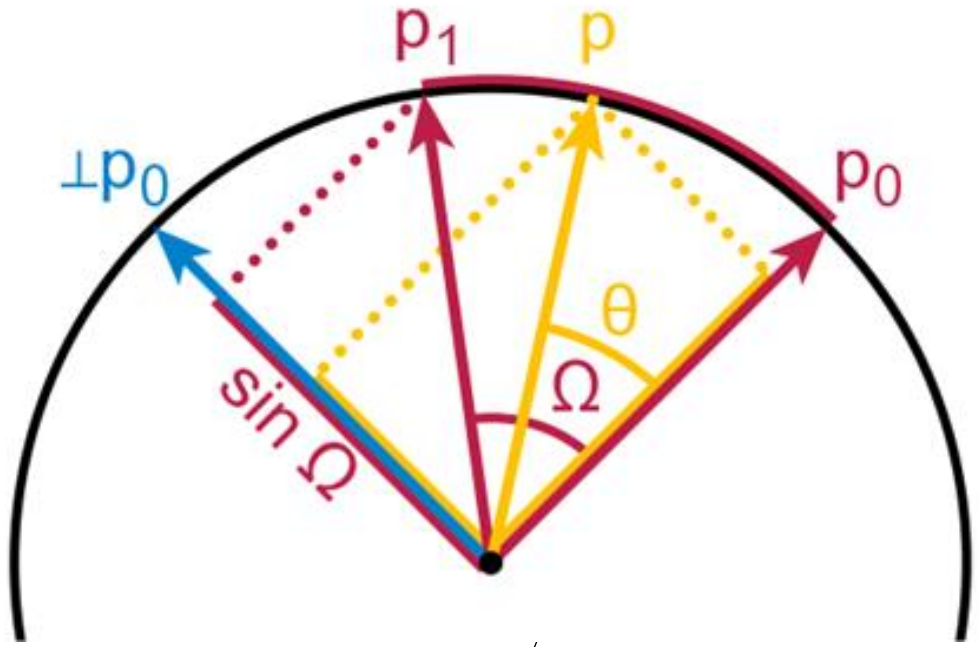


SLERP - ALGORITHMUS

Ziel:

Interpolation zwischen zwei Punkten p0 und p1 auf dem Kreis

$$\operatorname{slerp}(p_0, p_1; t) = \frac{\sin\left[(1-t)\Omega\right]}{\sin\Omega} p_0 + \frac{\sin[t\Omega]}{\sin\Omega} p_1 .$$



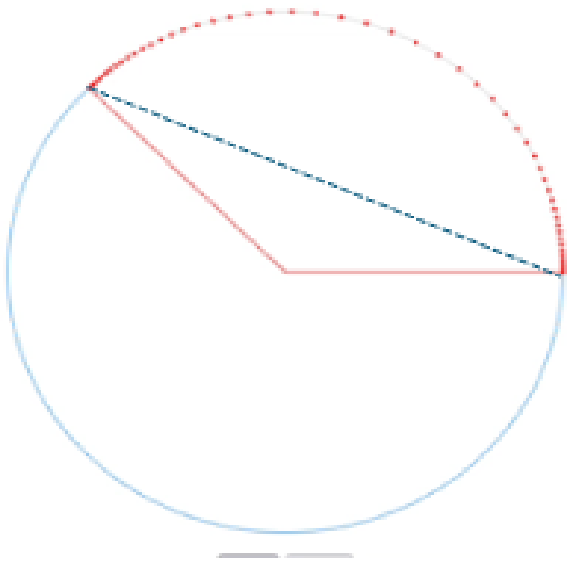
SLERP – ALGORITHMUS

LERP:

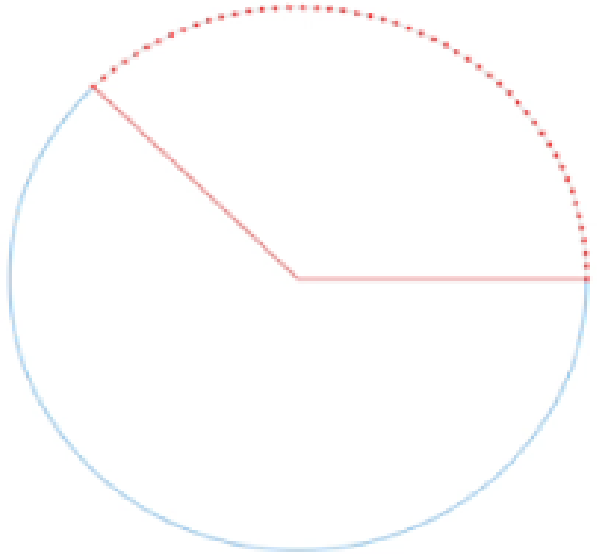
- Einfacher und schneller
- erzeugt keine einheitliche Winkelgeschwindigkeit

SLERP:

- schwieriger
- erzeugt eine einheitliche Winkelgeschwindigkeit



LERP

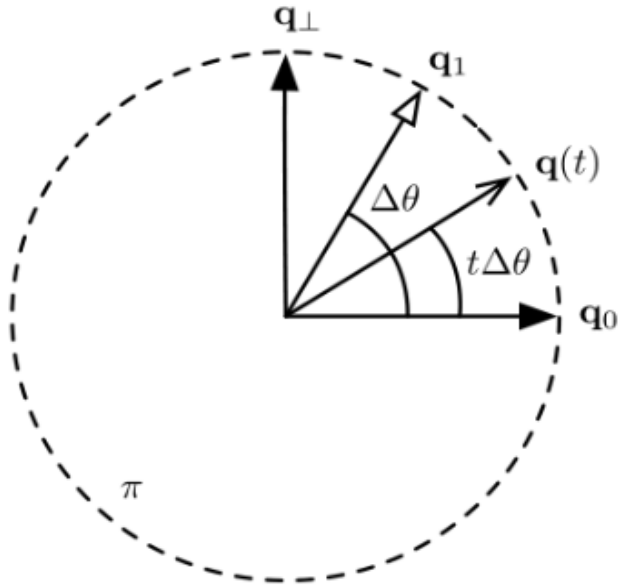


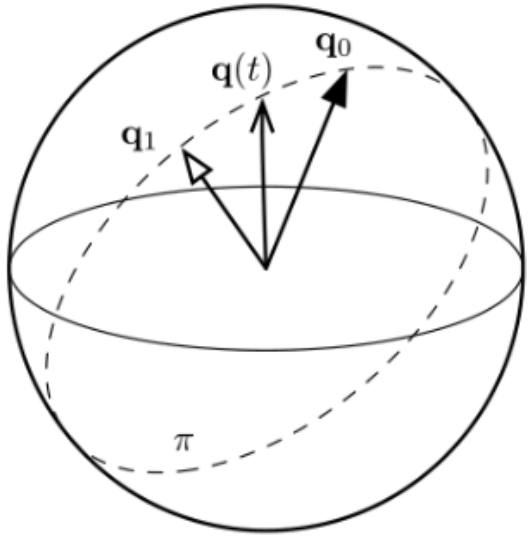
SLERP

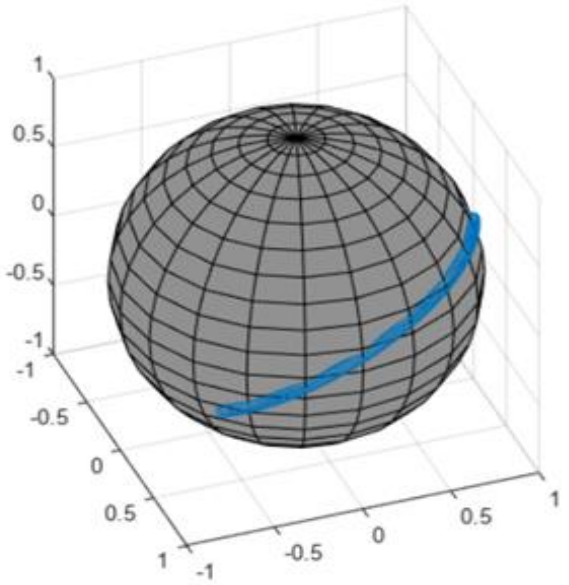
SLERP – QUATERNIONEN

Ziel:

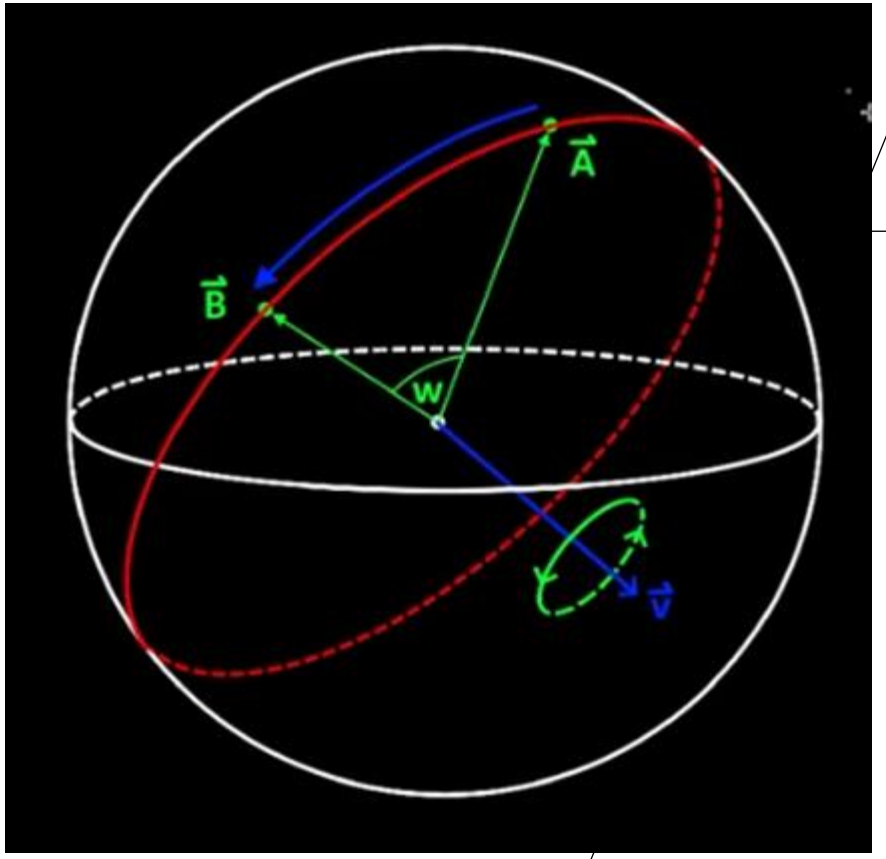
Interpolation zwischen zwei Rotationen







SLERP – QUATERNIONEN



Definition:

- q = a + bi + cj + dk -> Vektor & Winkel

Vorteile

- Kompakte Darstellung
- Schnelle Berechnungen
- Umwandlung in Euler-Winkeln und Rotationsmatrizen
- Kein Gimble lock
- Rotationsinterpolation

BEWEGUNG - WRAPPER

BEWEGUNG – WRAPPER

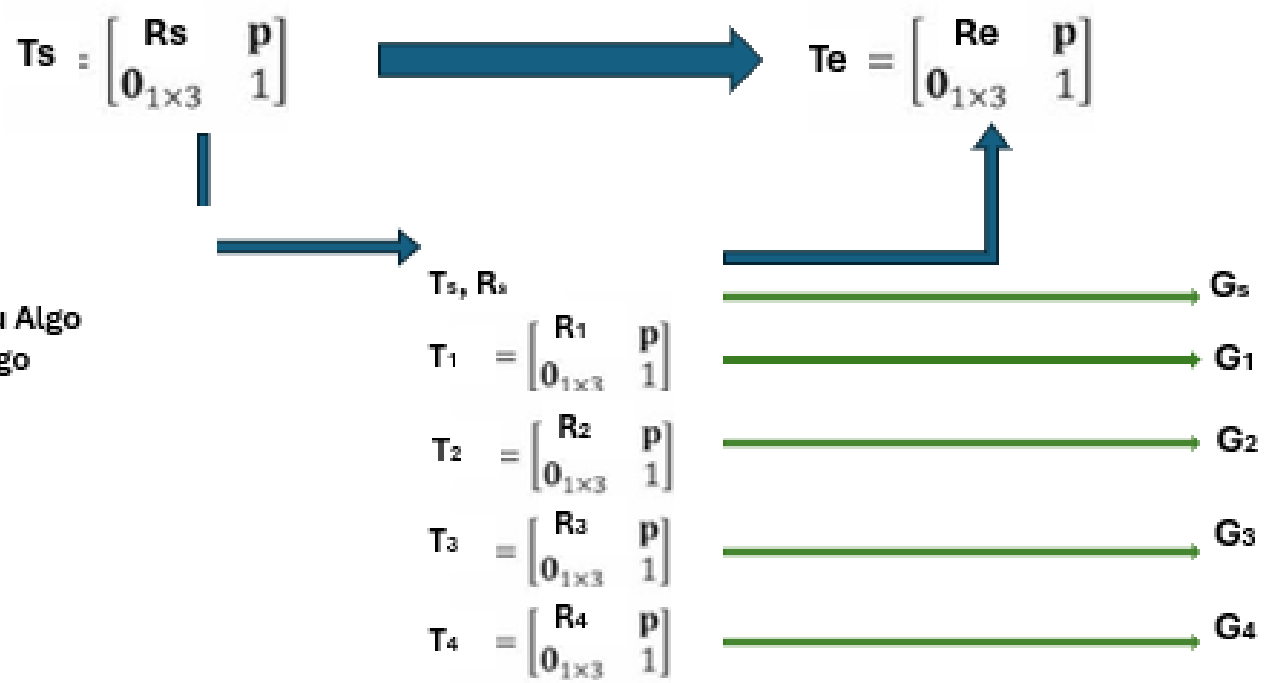
Solver Team:

Start (Ts) und End (Te) Transformationsmatrix

1. Interpolieren
   → Ti
   Pi: De Casteljau Algo
   → Ri: SLERP Algo
2. Inverse Kinematik
   → Gi

Visualization Team:

Interpolierte (Gi) Gelenkkonfigurationen





KONTROLLE DER BEWEGUNG DES ROBOTERS

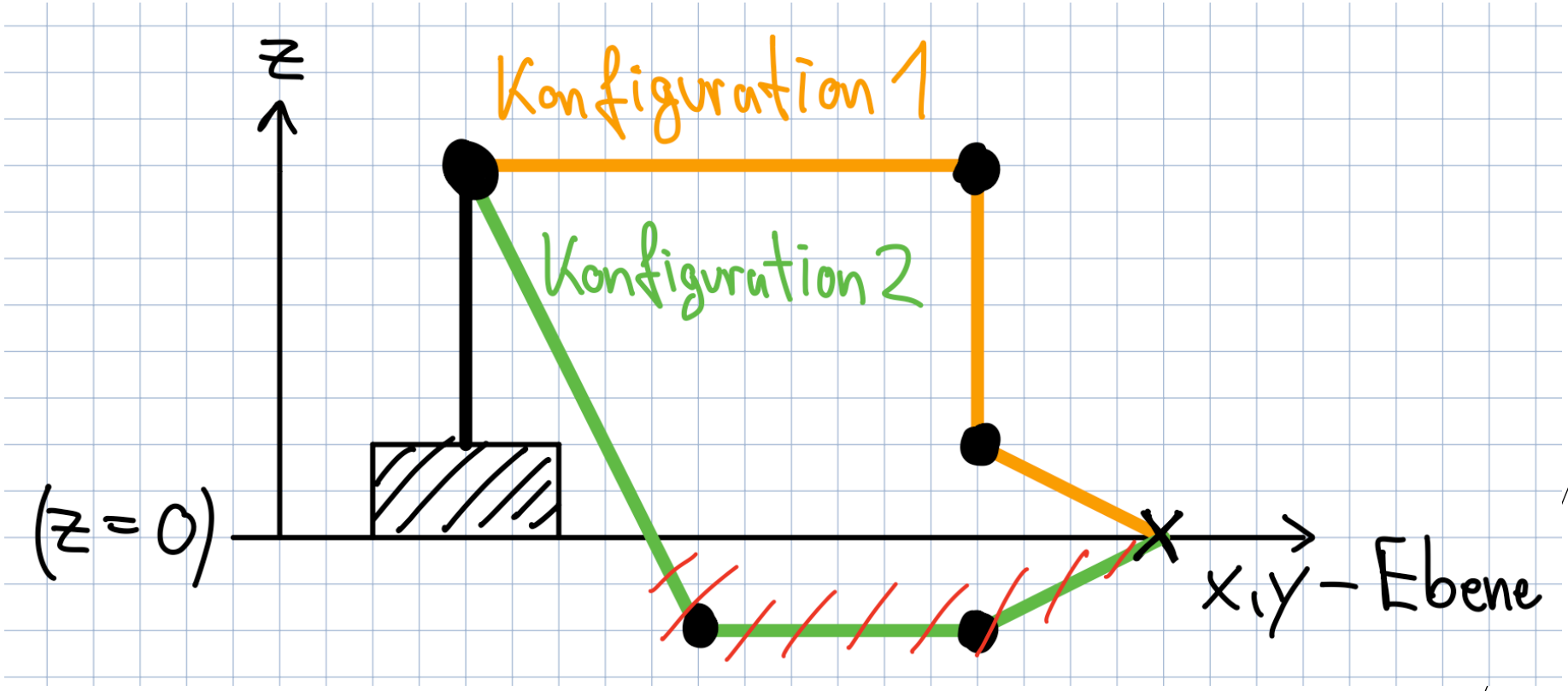
ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

KOLLISIONSPRÜFUNG GEGENÜBER DER BASIS

NUTZEN & MOTIVATION

- Roboter bewegt sich stehts nahe am Boden
- Lösungen der inversen Kinematik für Bewegung im $z \leq 0$ Raum potenziell möglich
- Kollisionsprüfung gegenüber der Basis ($z = 0$)
- Verhindern von Bewegungen im $z \leq 0$ Raum, da in Realität unmöglich

MÖGLICHE GELENK KONFIGURATIONEN



KOLLISIONSPRÜFUNG

- Aufbau des KUKA-KR6 Roboters ähnelt einem menschlichen Arm

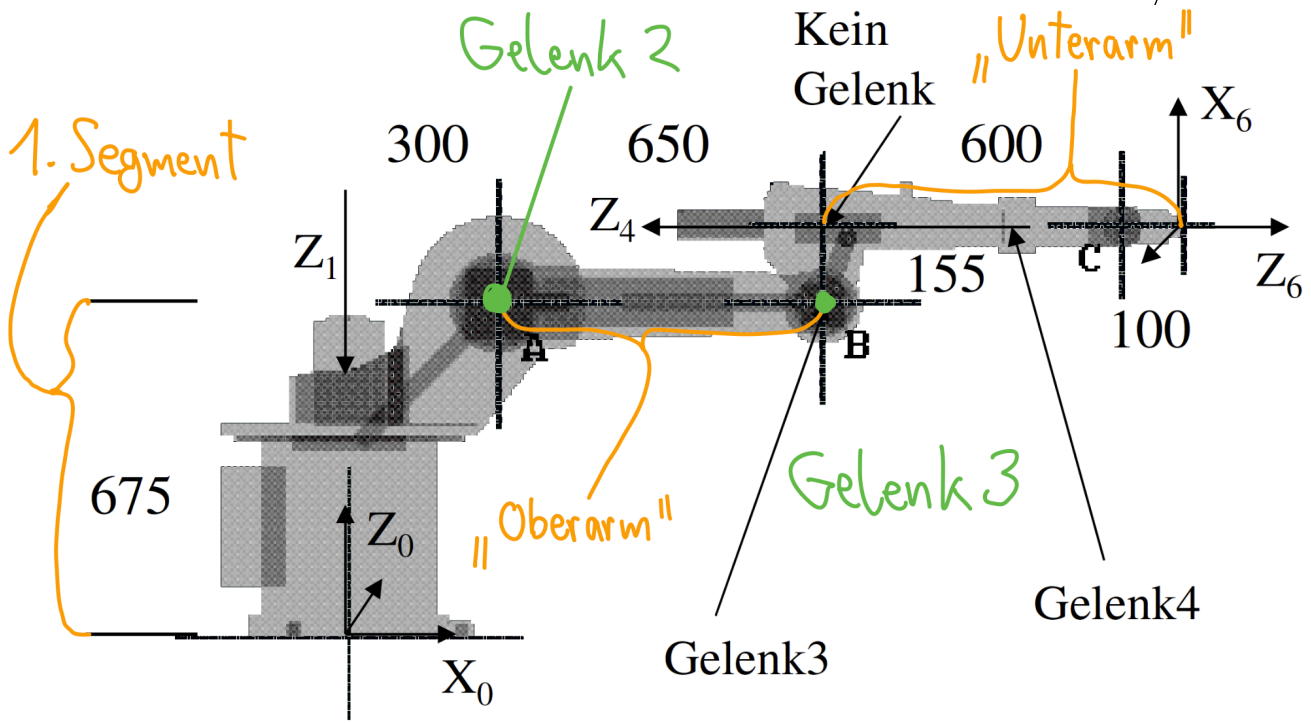
  => Potenzielle Kollision gegenüber der Basis geschieht immer durch mindestens ein Gelenk

- Kollisionsprüfung auf $z \geq 0$ Bedingung für alle Gelenke und jede Bewegung des Roboters
  - Berechnen der Vorwärtskinematik für den Endpunkt eines Bewegungsschritts
  - z-Positionen aller Gelenke anhand der Vorwärtskinematik auf $z \geq 0$ Bedingung prüfen

NOTWENDIGKEIT DER KOLLISIONSPRÜFUNG

- 1. Segment länger als 2. Segment „Oberarm“ => Gelenk 3 kann nicht mit Boden kollidieren
- Möglich Lösungen der inversen Kinematik für Bewegung im $z \leq 0$ Raum ausgeschlossen

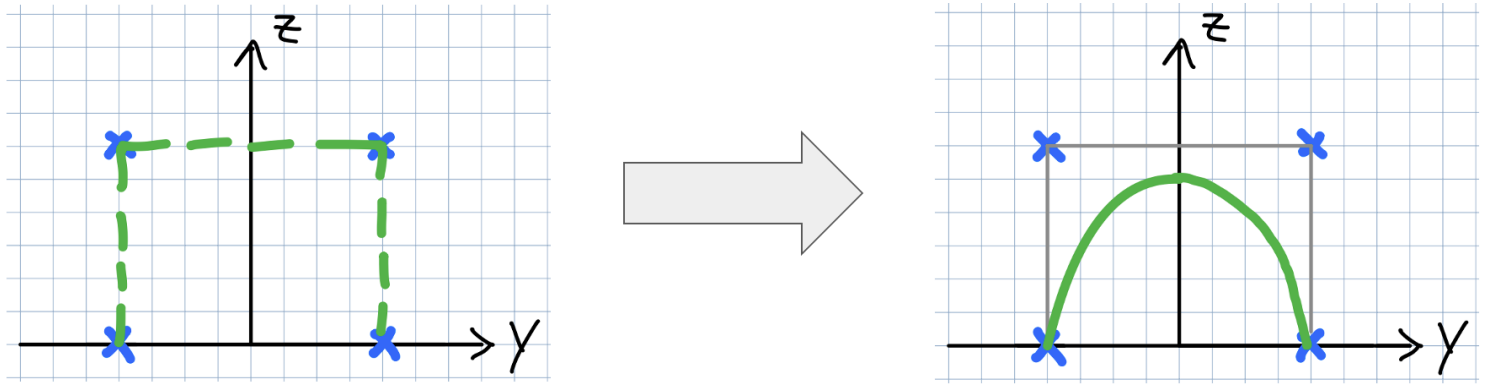
=> Weiterentwicklung der Kollisionsprüfung gegenüber der Basis eingestellt, da nicht relevant



INTERPOLATION DER BEWEGUNGSKURVE DURCH DEN ALGORITHMUS VON DE CASTELJAU

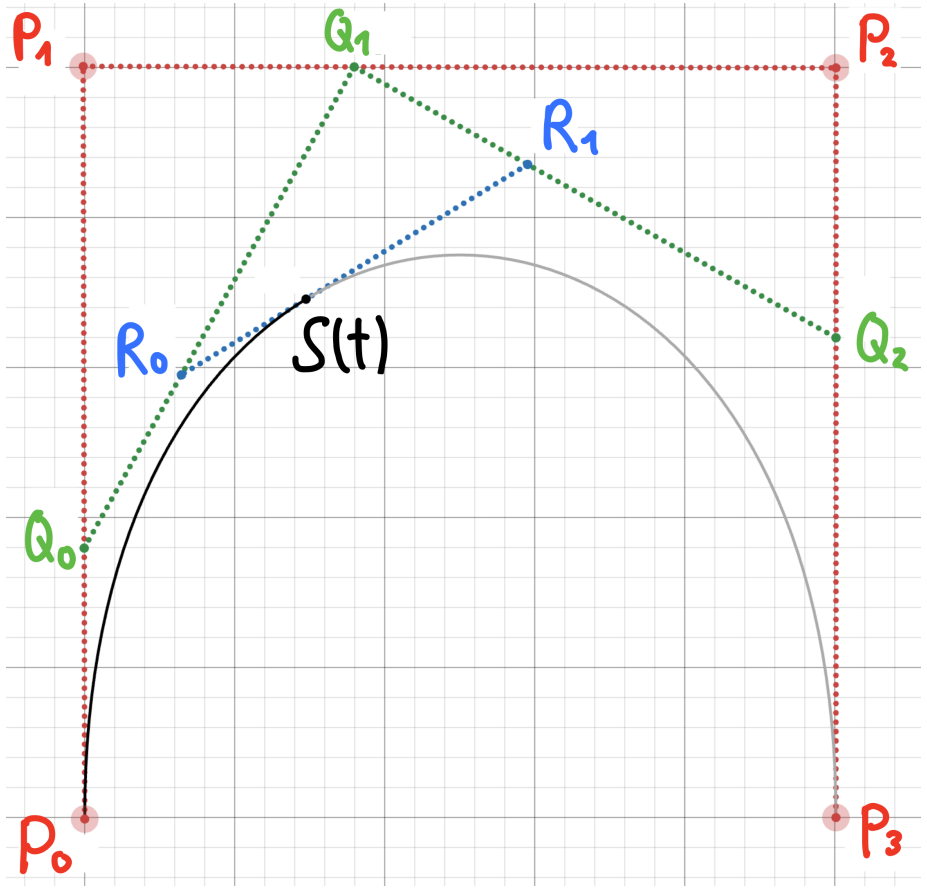
NUTZEN & MOTIVATION

- Kurvenförmige Bahn für Bewegung soll aus linearen translatorischen Bahnen interpoliert werden
- Interpolation des Translationsvektors p von Startposition zur Endposition über beliebig vielen Teilschritte i



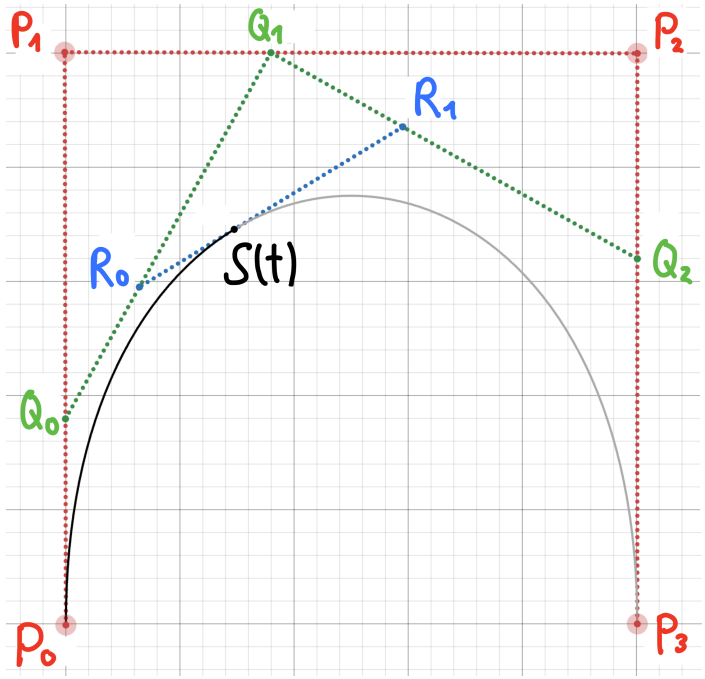
$$T_s = \begin{bmatrix} R_s & \rho_s \\ 0_{1\times 3} & 1 \end{bmatrix} \quad T_i = \begin{bmatrix} R_i & \rho_i \\ 0_{1\times 3} & 1 \end{bmatrix} \quad T_{i+1} = \begin{bmatrix} R_{i+1} & \rho_{i+1} \\ 0_{1\times 3} & 1 \end{bmatrix} \quad \dots \quad T_e = \begin{bmatrix} R_e & \rho_e \\ 0_{1\times 3} & 1 \end{bmatrix}$$

INTERPOLATION DURCH DE CASTELJAU ALGORITHMUS



- Punkte P sind gegeben: Start-, End- und Hilfspunkte
- Stufenweise lineare Interpolation zwischen Punkten der vorherigen Stufe bis ein Punkt übrig bleibt
- Zusammensetzen aller Interpolationen ergibt Positionsfunktion S(t) von letztem Punkt, welche die interpolierte Kurve darstellt

INTERPOLATION DURCH DE CASTELJAU ALGORITHMUS



$$Q_0(t) = (1-t)P_0 + tP_1 \qquad Q_1(t) = (1-t)P_1 + tP_2 \qquad Q_2(t) = (1-t)P_2 + tP_3$$

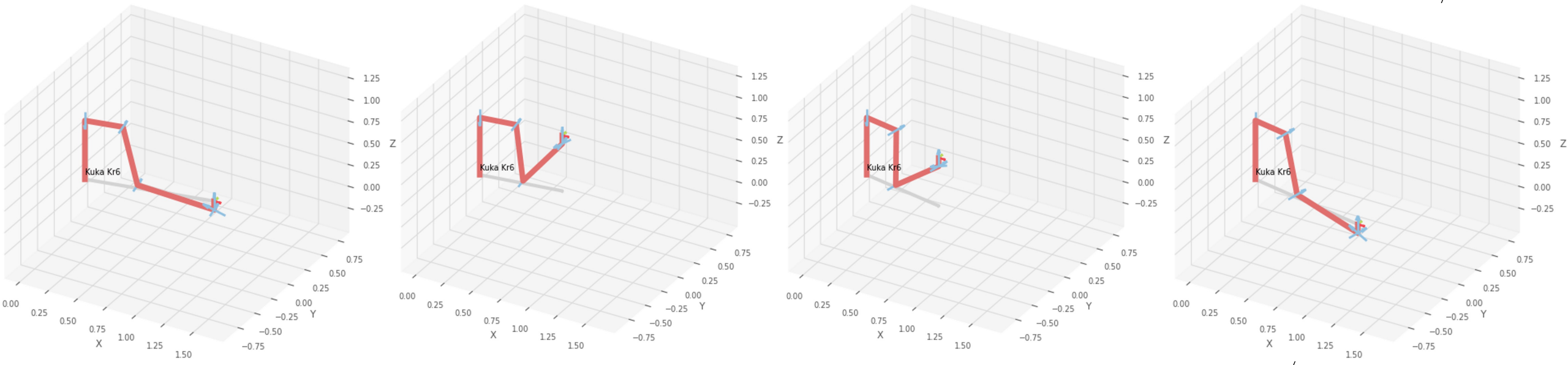
$$R_1(t) = (1-t)Q_1(t) + tQ_2(t) = (1-t)^2P_1 + 2t(1-t)P_2 + t^2P_3$$

$$R_0(t) = (1-t)Q_0(t) + tQ_1(t) = (1-t)^2P_0 + 2t(1-t)P_1 + t^2P_2$$

$$S(t) = (1-t)R_0 + tR_1 = (1-t)^3P_0 + \left(3t - 6t^2 + 3t^3\right)P_1 + 3t^2(1-t)P_2 + t^3P_3$$

- Interpolierte Kurve S(t) basiert ausschließlich auf gegebenen Punkten P mit 0 < t < 1

INTERPOLATION DURCH DE CASTELJAU ALGORITHMUS

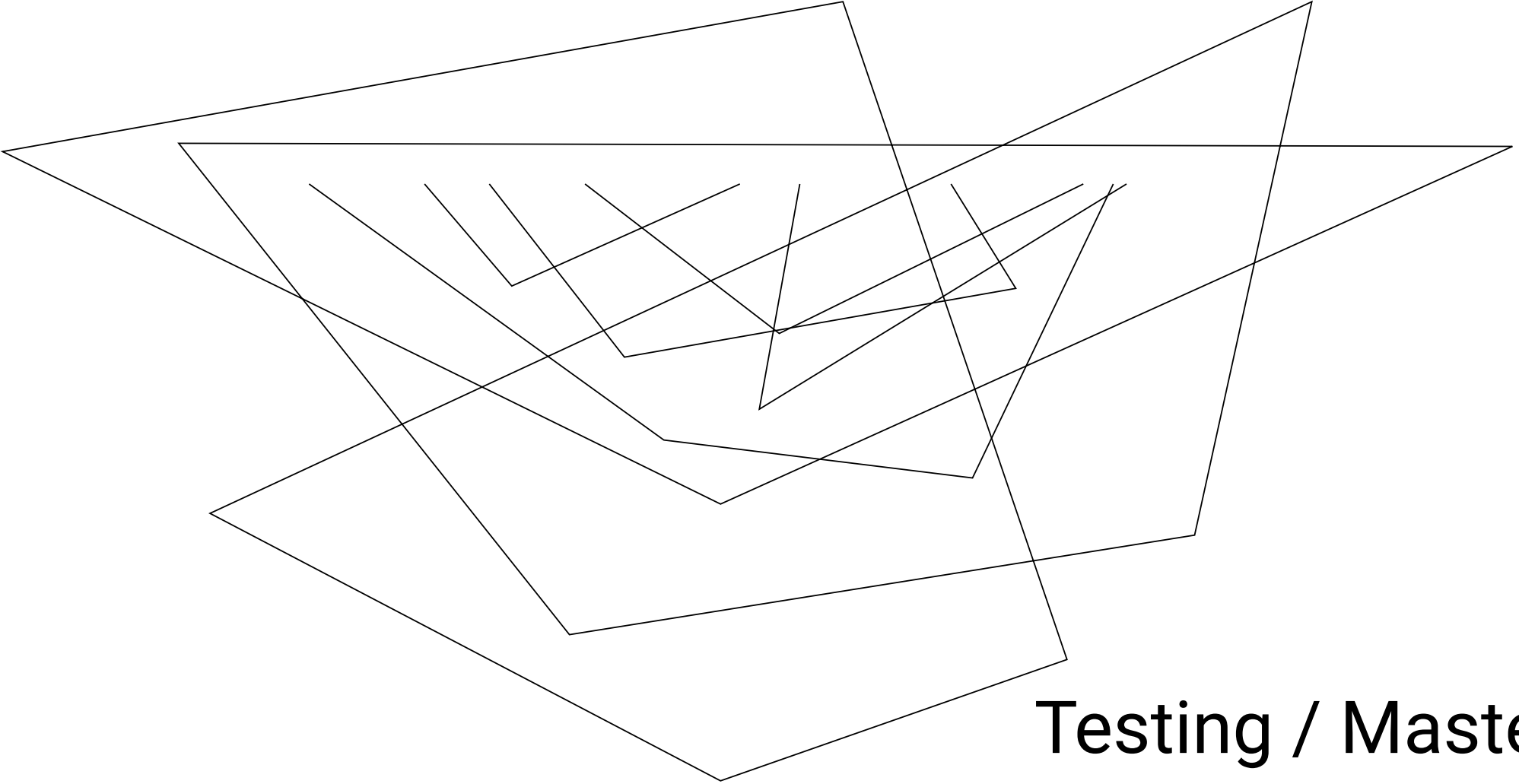


=> Resultat: Kurvenförmige Bewegung des Roboters konnten aus 4 gegebenen Punkten (≙ 3 linearen Bewegungen) interpoliert werden

VIELEN DANK FÜR IHRE AUFMERKSAMKEIT!

Johannes Decker - @000000003B9C94C8 (gitlab)

johannes.decker@hm.edu



Testing / Master Node

ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

GLIEDERUNG

- Testing
  - Mockups
  - Unit Tests
  - CI/CD
- Master Node
  - User Interface
  - Watchdog

TESTING

TESTING

Testing

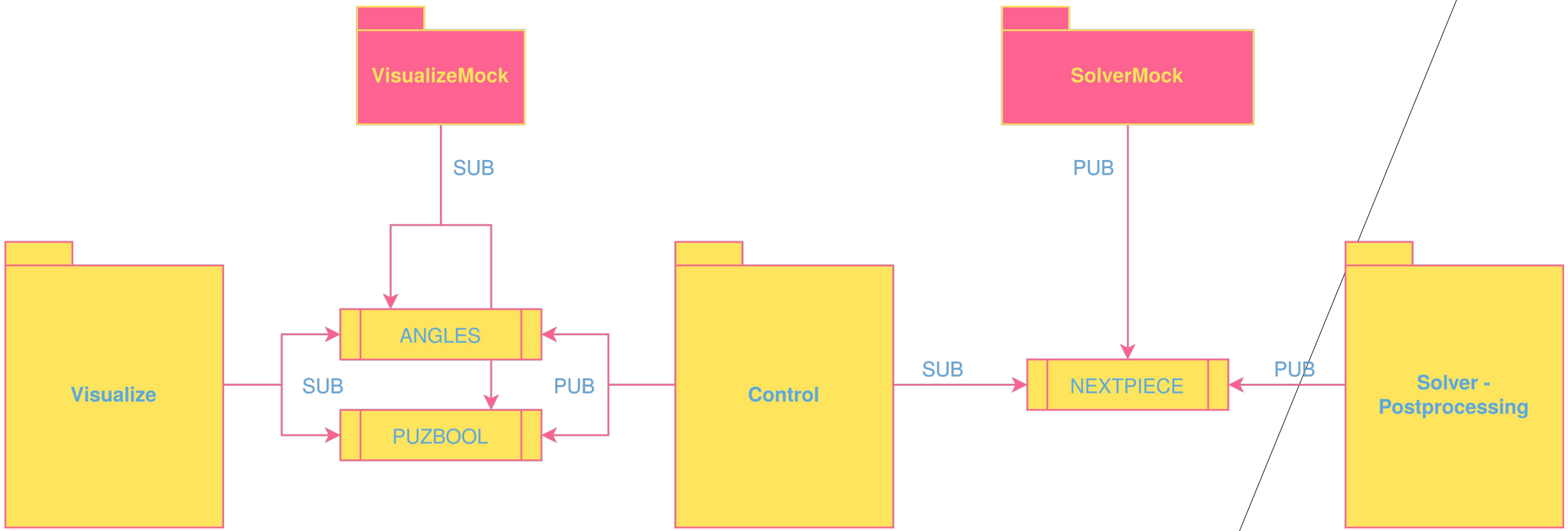
- Unabhängigkeit von anderen Nodes
- Testbarkeit der Funktionalität
- Besseres Gefühl für Verhalten im Gesamtsystem

TESTING

Mockups

- Zwei Mockups (VisualizeMock, SolverMock)
- Simulation des Verhaltens nach außen

TESTING


VisualizeMock
SUB
SolverMock
PUB
Visualize
ANGLES
SUB
PUB
PUZBOOL
Control
SUB
NEXTPIECE
PUB
Solver -
Postprocessing

TESTING

Mockups

- Solver Mock
    - Erstellt Strecke (Start- und Endpunkt)
    - Published auf *NEXTPIECE*
- Visualize Mock
    - Subscribed auf *ANGLES* und *PUZBOOL*
    - Wertet Empfangene Daten aus

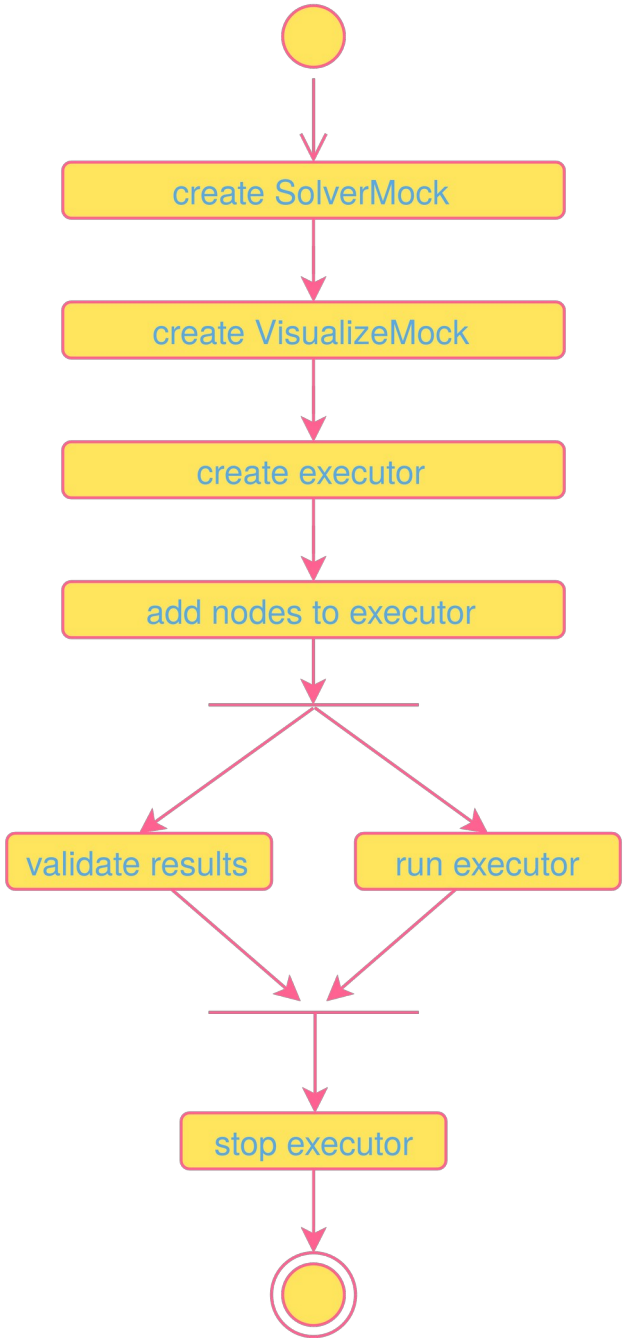
TESTING

Unit Tests

- GoogleTest (gtest)
- Automatisches Testen vordefinierter Szenarien

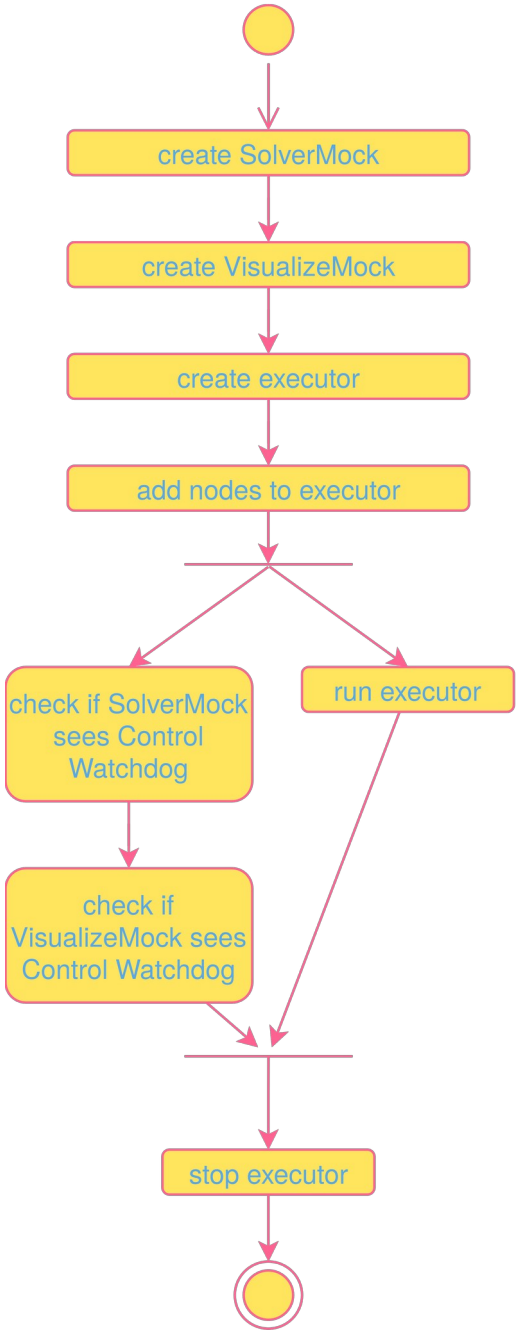
TESTING

Unit Test General



TESTING

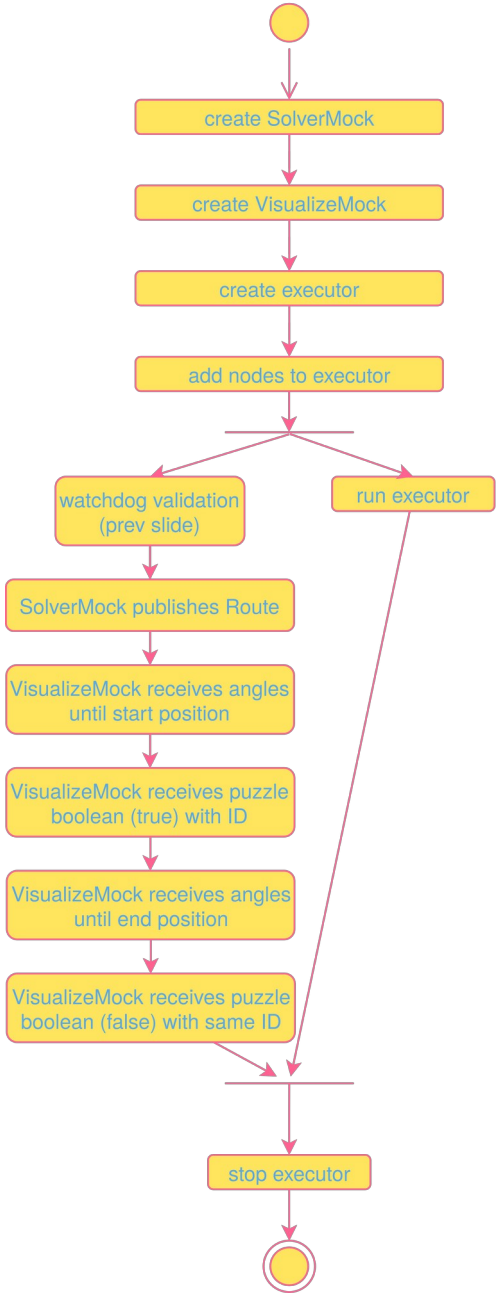
Unit Test 1



TESTING

Unit Test 2

(Not implemented)



TESTING

CI/CD

- Automatisches Ausführen von Unit Tests bei Commit
- Integriert in GitLab

TESTING

CI/CD

1. Clonen des Gesamtrepositories
2. Installieren der notwendigen Dependencies
3. Initialisieren der Subrepositories
4. Bauen des Projekts (colcon build)
5. Ausführen der Tests (colcon test --ctest-args tests)

TESTING

CI/CD

```
script:
  - git clone

  - pip --version
  - pip install --upgrade pip

  - cd robotik-ss24

  - rosdep update
  - rosdep install --from-paths src -y --ignore-src

  # - pip install spatialmath-python
  - pip install roboticstoolbox-python
  - pip install numpy --upgrade

  - cd src
  - git clone
  - git clone
  - ls
  - cd ..

  - colcon build
  - echo "BUILD SUCCESSFULL"
```

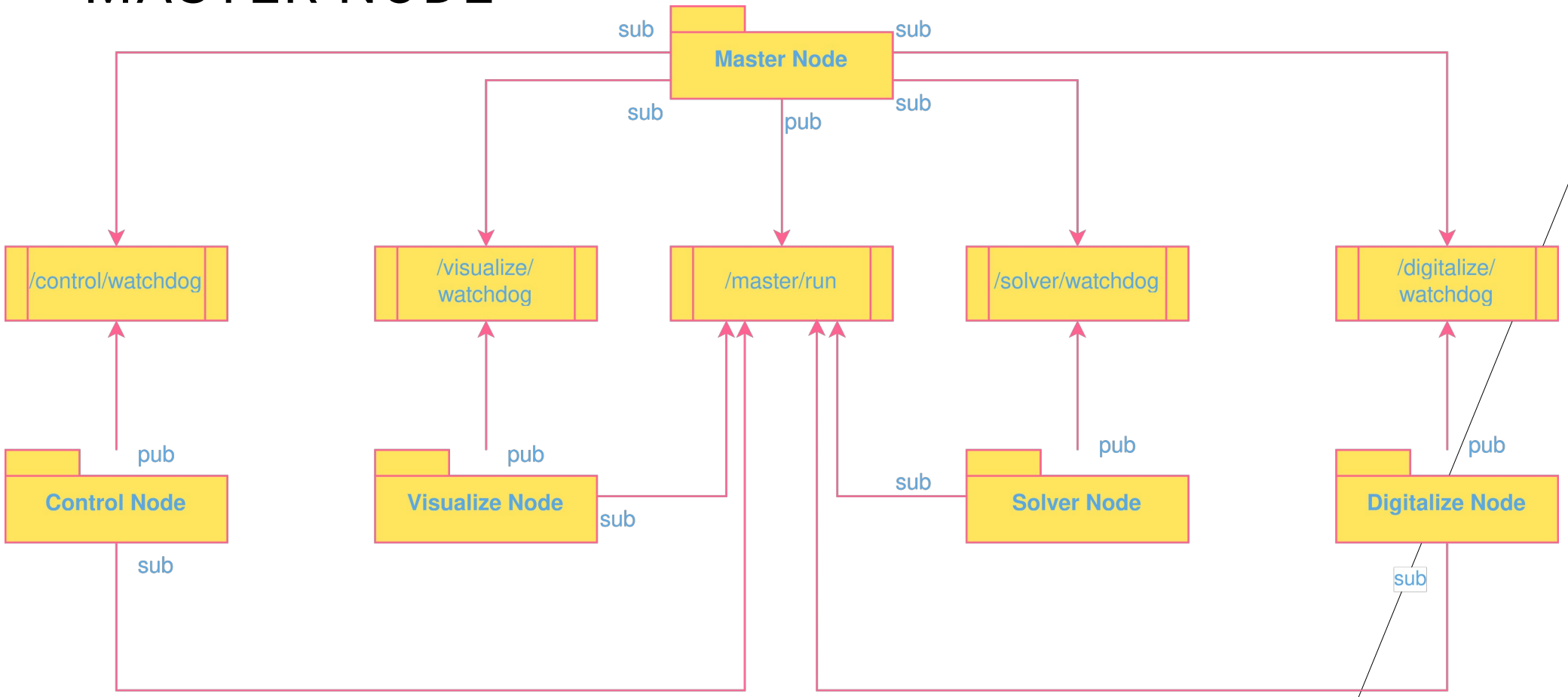
MASTER NODE

MASTER NODE

Grundidee

- Start und Stop aller Nodes synchronisieren
- Systemzustand visualisieren
    - Watchdog für jede einzelne Node
    - User Interface (QT5)

MASTER NODE


sub
Master Node
sub
sub
pub
sub
/control/watchdog
/visualize/
watchdog
/master/run
/solver/watchdog
/digitalize/
watchdog
pub
pub
pub
pub
Control Node
Visualize Node
Solver Node
Digitalize Node
sub
sub
sub
sub

MASTER NODE

User Interface



MASTER NODE

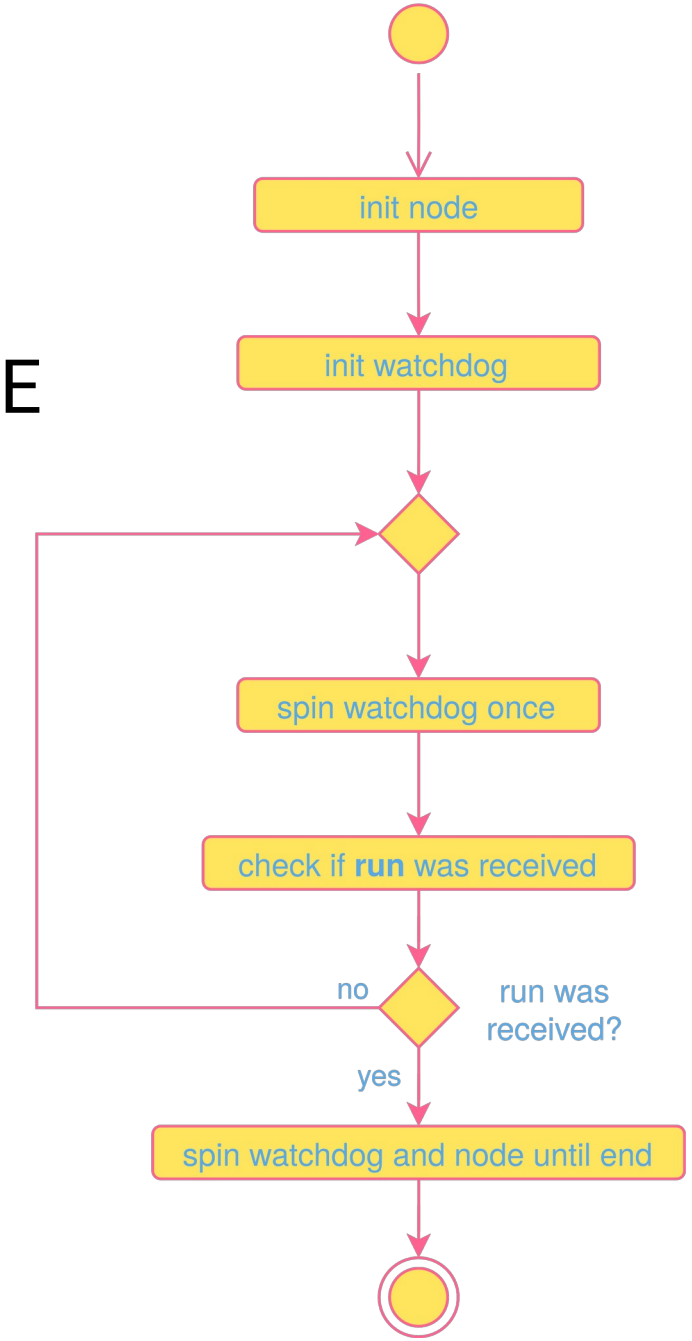
Watchdog

- Komponente die jede Sekunde Lebenszeichen sendet
- Wenn 5 Sekunden kein Watchdog kommt → Node inaktiv

MASTER NODE

Watchdog

- Startup



MASTER NODE

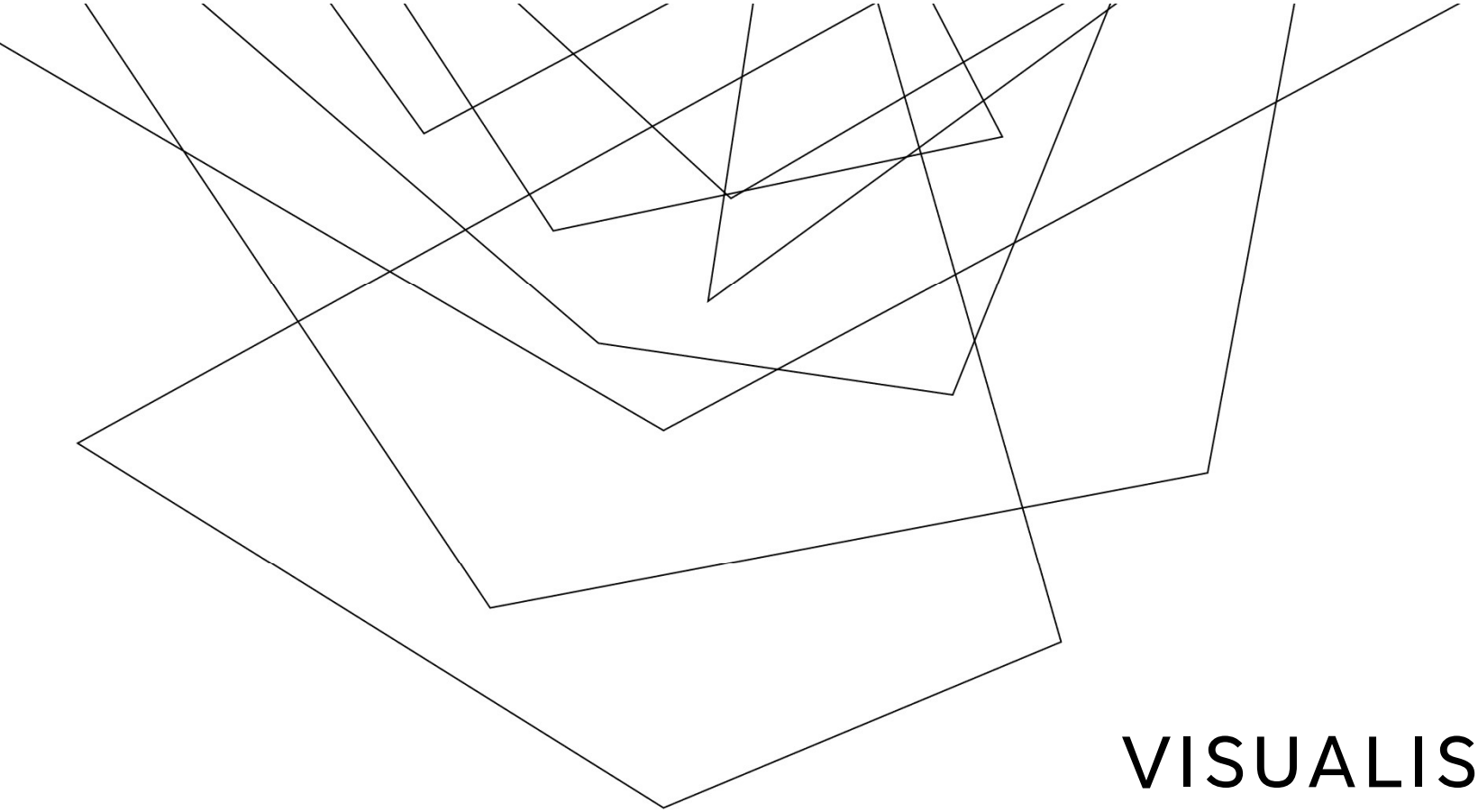
Watchdog

- Stop → Beenden aller Nodes

VIELEN DANK FÜR IHRE AUFMERKSAMKEIT!

Simon Mederer

hm-smederer@hm.edu



VISUALISIERUNG

ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

GLIEDERUNG

- Workspace
  - Docker
  - DevContainer
  - Gitlab CI
- Visualisierung
  - Kuka Kr6
  - Puzzle 3D modellieren
  - Schnittstellen Implementierung

WORKSPACE

WAS IST DOCKER ?

Docker

- Betriebsystem Virtualisierung (Kein eigener Kernel)
- Spezifiziert über Dockerfiles

Vorteile gegnüber VMs

- Schnelle Start Zeit
- Kleinerer Speicherplatzbedarf
- Kleinerer Resourcenverbrauch
- DevContainer

```
FROM osrf/ros:humble-desktop-full       You,

ENV DEBIAN_FRONTEND=noninteractive
ENV LANG=en_US.UTF-8
ENV ROS_INSTALL_PATH=/opt/ros/humble
ENV COLCON_PREFIX_PATH=/opt/ros/humble
ENV WORKSPACE=/workspace

ARG USERNAME=vscode
ARG USER_UID=1000
ARG USER_GID=$USER_UID

RUN apt update && apt upgrade -y
RUN apt install -y  x11-xserver-utils \
                    x11-apps \
                    libx11-dev \
                    libqt5widgets5 \
                    libcoin-dev \
                    libsoqt520-dev \
                    jq \
                    xinit \
                    xserver-xorg \
                    mesa-utils \
                    libegl1-mesa \
                    libegl1-mesa-dev \
                    libgbm-dev \
                    libgbm1 \
                    libgl1-mesa-dev \
                    libgl1-mesa-dri \
                    libgl1-mesa-glx \
                    libglu1-mesa \
                    libglu1-mesa-dev \
                    npm \
                    clang \
# ROS 2 ----------------------------------
```

WAS SIND DEVCONTAINER?

Devcontainer

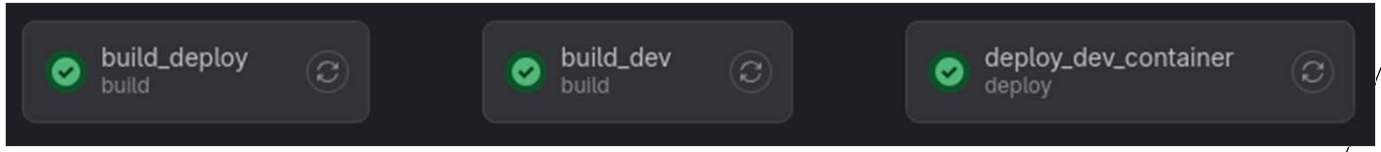
- Container die ein Entwicklungsumgebung bereitstellen
- Integration in vielen IDEs (Hauptsächlich VSCode)
- Spezifiziert über devcontainer.json

```
{
    "name": "ROS 2 Development Container for Linux",
    "privileged": true,
    "remoteUser": "vscode",
    "workspaceFolder": "/home/vscode/workspace",
    "image":
    "workspaceMount": "source=${localWorkspaceFolder},target=/home/vscode/workspace,type=bind",
    "customizations": {
        "vscode": {
            "extensions": [
                "ms-vscode.cpptools-themes",
                "twxs.cmake",
                "donjayamanne.python-extension-pack",
                "ms-iot.vscode-ros",
                "ms-vscode.cpptools",      Christoph Schwarz, 3 months ago • Devcontainer
                "ms-python.python",
                "GitLab.gitlab-workflow",
                "mhutchie.git-graph",
                "eamodio.gitlens",
                "GitHub.copilot",
                "tal7aouy.rainbow-bracket",
                "oderwat.indent-rainbow",
                "ms-vscode.cpptools-extension-pack",
                "mhutchie.git-graph",
                "seatonjiang.gitmoji-vscode",
                "bierner.markdown-mermaid",
                "tomoki1207.pdf",
                "ms-vscode.cmake-tools",
                "llvm-vs-code-extensions.vscode-clangd",
                "MS-vsliveshare.vsliveshare"
            ]
        }
    },
    "containerEnv": {
        "DISPLAY": "unix:0",
        "ROS_AUTOMATIC_DISCOVERY_RANGE": "LOCALHOST",
        "ROS_DOMAIN_ID": "42"
    },
    "runArgs": [
        "--net=host",
        "-e",
        "DISPLAY=${env:DISPLAY}"
    ],
    "postCreateCommand": "$HOME/workspace/.devcontainer/scripts/post_setup.sh",
    "postStartCommand": "$HOME/workspace/.devcontainer/scripts/post_start.sh"
}
```

WAS IST GITLAB CI?

- Bauen der Docker Images
- Pipeline Design
- Hochladen auf Registry

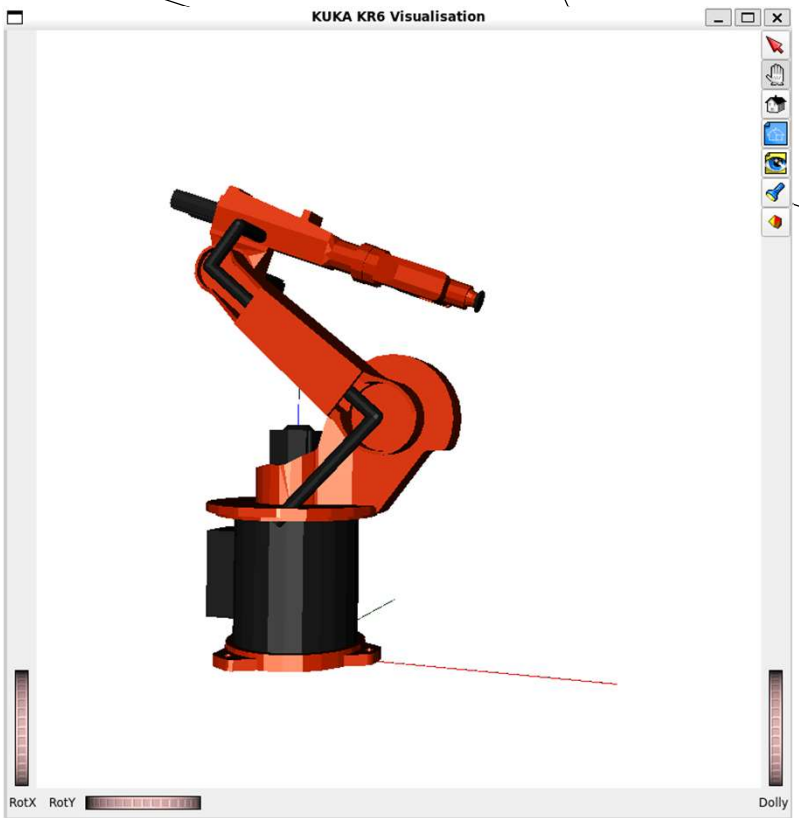
```yaml
build_deploy:
  stage: build
  tags:
    - docker_runner
  rules:
    - changes:
        - docker/Dockerfile.ROSDEPLOY
        - docker/Dockerfile.ROSDEV
  script:
    - cd ./docker
    - docker pull $CI_REGISTRY_IMAGE/ros_humble_deploy:latest || true
    - docker build -t $CI_REGISTRY_IMAGE/ros_humble_deploy:$CI_COMMIT_SHORT_SHA -f Dockerfile.ROSDEPLOY .
    - docker push $CI_REGISTRY_IMAGE/ros_humble_deploy:$CI_COMMIT_SHORT_SHA
```



VISUALISIERUNG

COIN3D 

- Coin ist eine auf OpenGL basierende 3D-Grafik-Library
- Es basiert auf der Open Inventor API
  - Erweitert um viele weitere Features die Open Inventor nicht hat
- Coin ist genau wie Open Inventor:
  - eine Szenegraph basierte, retain-mode, Rendering und Model manipulations Bibliothek
- Wichtige Konzepte/Klassen von Coin3D
  - SoSeperator
  - SoTransform (enthält Translation, Orientierung, Rotation und Skalierung)
  - SoSearchAction
  - SoIdleSensor



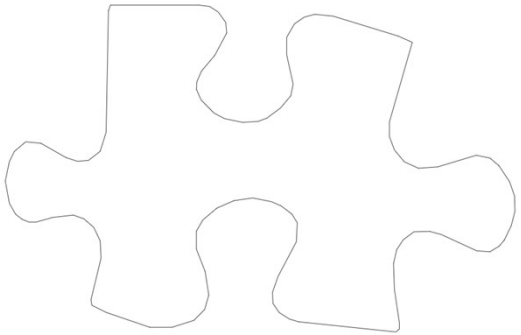
KUKA KR6 VISUALISIERUNG

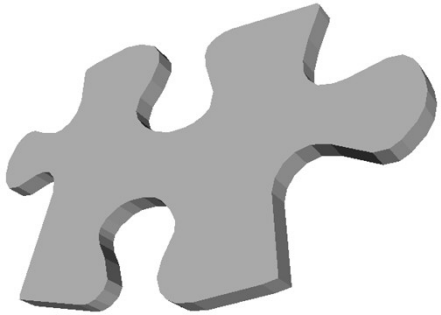
- IV-Files der einzelnen Arme
- Zusammengesetzt zu einem IV-File mit Imports
- Anordnung nach DH-Parametern
- Animation durch SoTransform und SoSeperators

PUZZLE 3D MODELLIERUNG

- Laden der Punkte des Puzzles aus der ROS message
- Verbinden der Punkte
  - 2 x für unterschiedliche Höhen (z-Achse)
- Verbinden der beiden puzzle-linien mit triangles
- Bild auf das 3D visualisierte Puzzle legen









SCHNITTSTELLEN IMPLEMENTIERUNG

SegmentedPuzzleTile

- Enthält das PNG
- Punkte des Puzzleteils (Point2d[])
- ID des Puzzleteils

PuzzlePose

- Enthält die Startpositionen eines Puzzleteils (Translation, Rotation)
- Als auch die ID des Puzzleteils

Angles

- Winkel der Arme in radiant
- Greifarm auf/zu (boolean)
- ID des Puzzleteils, welches aufgehoben/abgelegt werden soll

ABLEGEN | AUFNEHMEN

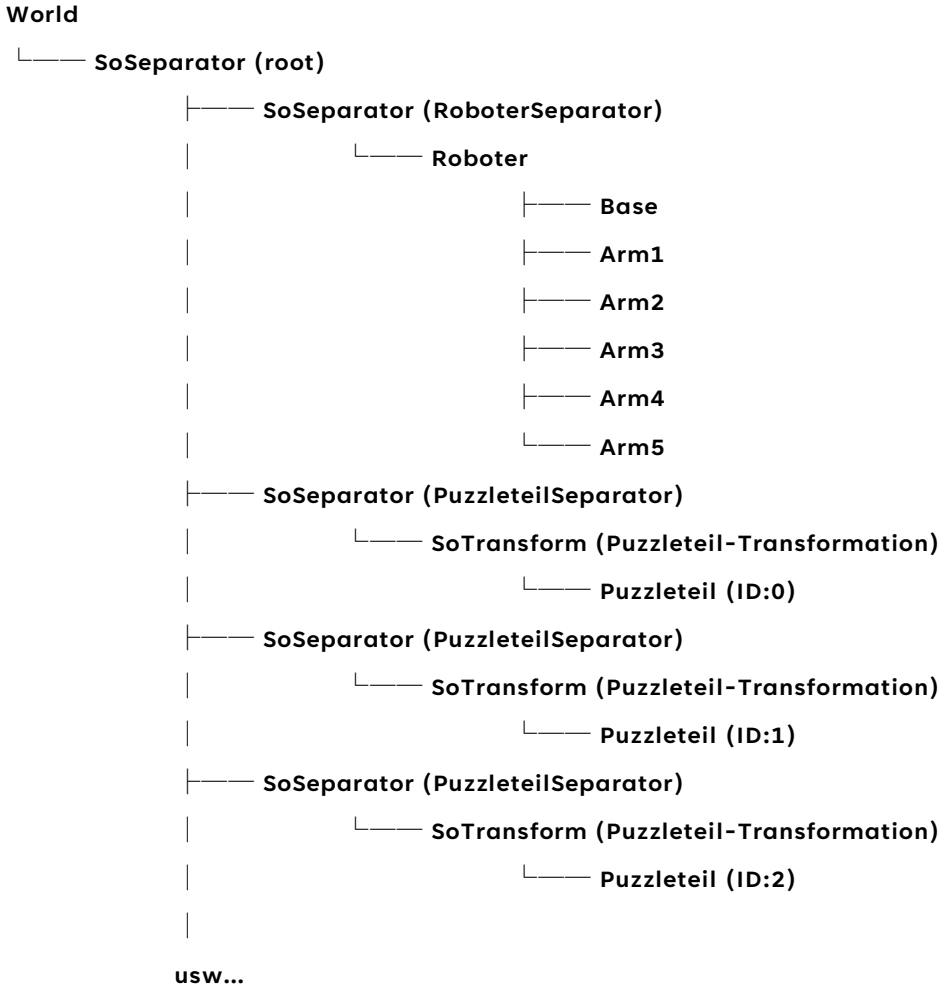
Vorgang

1. Puzzleteil nehmen
2. Puzzleteil bewegen
3. Puzzleteil ablegen

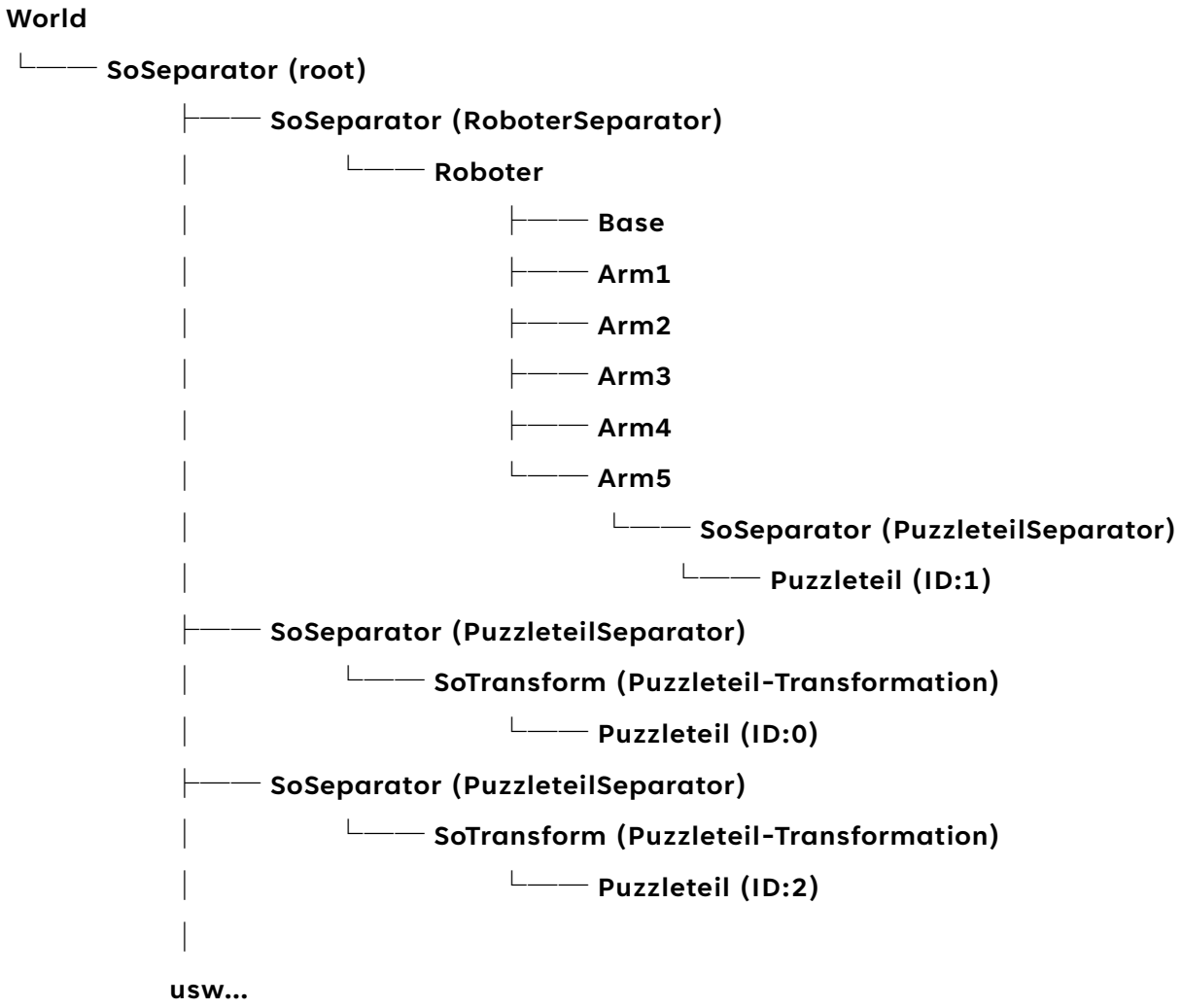
Beschreibung

1. Löschen aller SoTransform Nodes des Puzzleteils
2. Teil an den letzten Knoten des Roboters anhängen
3. Bewegen (Schnittstelle Control)
4. Ablegen
    - Berechnen der absoluten Position zum root
    - Löschen des letzten Knoten am Roboter
    - Setzten des Puzzleteils an der berechneten Position

SZENEGRAPHE (OHNE PUZZLE ATTACHED)



SZENEGRAPHE (MIT PUZZLE ATTACHED)



MERCEE, SERVUS!

Lucas Schlager

Christoph Schwarz